HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO TIỂU LUẬN 03

HỌC PHẦN: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên	**: PGS.TS. Trần Đình Quế**
Nhóm học phần	**: 05**
Họ và tên	**: Nguyễn Đức Trung**
Mã sinh viên	**: B23DCCN857**
Lớp	**: D23CNPM01**

HÀ NỘI 2026

MỤC LỤC

DANH MỤC HÌNH ẢNH 10
MỞ ĐẦU 12
1. Bối cảnh đề tài 12
2. Mục tiêu của tiểu luận 13
3. Đối tượng và phạm vi nghiên cứu 13
4. Phương pháp thực hiện 14
5. Ý nghĩa của tiểu luận 15
6. Cấu trúc của tiểu luận 15
7. Tiểu kết phần mở đầu 16
CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN TRÍ TUỆ NHÂN TẠO 17
1.1. Tổng quan về trí tuệ nhân tạo 17
1.2. Những ý tưởng ban đầu về máy thông minh 18
1.3. Alan Turing và câu hỏi về khả năng suy nghĩ của máy 18
1.4. Hội thảo Dartmouth và sự hình thành thuật ngữ Artificial Intelligence 19
1.5. AI biểu tượng và phương pháp dựa trên luật 20
1.6. Sự phát triển của hệ chuyên gia 20
1.7. Giai đoạn AI Winter 21
1.8. Sự chuyển dịch từ lập trình luật sang học máy 21
1.9. Sự phát triển của Machine Learning 21
1.9.1. Bài toán phân loại 22
1.9.2. Bài toán hồi quy 22
1.10. Sự phát triển của mạng nơ-ron nhân tạo 23
1.11. Sự hình thành và phát triển của Deep Learning 23
1.12. Sự phát triển của CNN trong xử lý hình ảnh 24
1.13. Sự phát triển của RNN trong xử lý dữ liệu tuần tự 25
1.14. Từ RNN đến LSTM và GRU 26
1.15. Sự xuất hiện của các kiến trúc AI hiện đại 26

1.16. Vai trò của dữ liệu trong sự phát triển của AI....27
1.17. Từ mô hình AI đến hệ thống AI hoàn chỉnh....27
1.18. Mối quan hệ giữa các giai đoạn phát triển của AI....28
1.19. Những bài học từ quá trình phát triển AI....29
1.20. Tiểu kết chương....29
CHƯƠNG 2. CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN....31
2.1. Tổng quan về Machine Learning....31
2.2. Học có giám sát....32
2.2.1. Bài toán phân loại....32
2.2.2. Bài toán hồi quy....32
2.3. Học không giám sát....33
2.3.1. Phân cụm K-Means....33
2.3.2. Giảm chiều dữ liệu....33
2.4. Biểu diễn dữ liệu trong Machine Learning....34
2.4.1. Biến số và biến phân loại....34
2.4.2. Biểu diễn văn bản bằng TF-IDF....35
2.5. Tiền xử lý dữ liệu....35
2.5.1. Xử lý dữ liệu thiếu....36
2.5.2. Phát hiện dữ liệu bất thường....36
2.5.3. Chuẩn hóa dữ liệu....36
2.6. Kỹ thuật hồi quy tuyến tính....37
2.7. Ridge và Lasso Regression....37
2.8. Logistic Regression....38
2.9. K-Nearest Neighbors....38
2.10. Decision Tree....39
2.11. Random Forest....39
2.12. Support Vector Machine....39
2.13. Naive Bayes....40

2.14. Đánh giá mô hình Machine Learning 40
2.14.1. Accuracy 41
2.14.2. Precision 41
2.14.3. Recall 41
2.14.4. F1-score 41
2.14.5. Confusion Matrix 41
2.14.6. MAE, MSE và RMSE 42
2.14.7. R^2 42
2.15. Overfitting, Underfitting và khả năng tổng quát hóa 42
2.16. Quy trình xây dựng một hệ thống Machine Learning 43
2.17. Minh họa thực nghiệm với các bài toán Machine Learning 43
2.17.1. Bài toán dự đoán nguy cơ bệnh tiểu đường 43
2.17.2. Bài toán dự đoán giá nhà 45
2.17.3. Bài toán phân tích hành vi khách hàng từ dữ liệu văn bản 47
2.18. So sánh kết quả các bài toán thực nghiệm 48
2.19. Cài đặt Machine Learning bằng Scratch, Keras và PyTorch 49
2.19.1. Cài đặt theo hướng Scratch 49
2.19.2. Cài đặt bằng Keras 53
2.19.3. Cài đặt bằng PyTorch 54
2.20. Lưu mô hình và triển khai ứng dụng 55
2.21. Demo dự đoán 56
2.21.1. Giao diện hệ thống dự đoán bệnh tiểu đường 56
2.21.2. Giao diện hệ thống dự đoán giá nhà 58
2.22. Nhận xét chung về các kỹ thuật Machine Learning cơ bản 60
2.23. Tiểu kết chương 60
CHƯƠNG 3. CONVOLUTIONAL NEURAL NETWORK 62
3.1. Tổng quan về Convolutional Neural Network 62
3.2. Từ mạng nơ-ron truyền thống đến CNN 63

3.3. Dữ liệu đầu vào của CNN 63
3.4. Phép Convolution 64
3.5. Kernel và Parameter Sharing 65
3.6. Stride và Padding 65
3.6.1. Stride 65
3.6.2. Padding 65
3.7. Kích thước Feature Map 66
3.8. Hàm kích hoạt ReLU 66
3.9. Pooling Layer 67
3.9.1. Max Pooling 67
3.9.2. Average Pooling 67
3.10. Fully Connected Layer 67
3.11. Hàm mất mát trong CNN 68
3.12. Backpropagation và tối ưu mô hình 68
3.13. Batch Normalization 69
3.14. Residual Connection 69
3.15. Attention trong CNN 70
3.16. Các kiến trúc CNN tiêu biểu 70
3.16.1. LeNet 70
3.16.2. AlexNet 70
3.16.3. VGG 71
3.17. Các kỹ thuật hạn chế Overfitting trong CNN 71
3.18. Các bộ dữ liệu sử dụng trong thực nghiệm 72
3.19. Dataset MNIST 72
3.20. Dataset Fashion-MNIST 72
3.21. Dataset Diabetes và CNN một chiều 73
3.22. Thiết kế các mô hình CNN thực nghiệm 73
3.23. Thiết lập quá trình huấn luyện 74

3.24. Kết quả thực nghiệm trên MNIST 75
3.25. Kết quả thực nghiệm trên Fashion-MNIST 76
3.26. Kết quả thực nghiệm trên Diabetes 78
3.27. So sánh mức cải thiện của CNN 80
3.28. Đánh giá Precision, Recall và F1-score 81
3.29. Độ phức tạp và thời gian huấn luyện 81
3.30. Phân tích kết quả 82
3.31. So sánh CNN với Machine Learning truyền thống 82
3.33. Cài đặt CNN bằng PyTorch 83
3.34. Demo nhận dạng chữ số bằng CNN 83
3.35. Tiểu kết chương 84
CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY RNN 86
4.1. Tổng quan về RNN 86
4.2. Nguyên lý hoạt động của RNN 86
4.2.1. Đặc điểm của dữ liệu tuần tự 87
4.2.2. Timestep và Sequence 87
4.2.3. Hidden State và bộ nhớ của RNN 88
4.2.4. Hiện tượng Vanishing Gradient và Exploding Gradient 88
4.2.5. Các dạng bài toán RNN 89
4.3. Dữ liệu chuỗi thời gian 89
4.3.1. Customer Behavior 89
4.3.2. AAPL Stock 90
4.4. Chuẩn bị dữ liệu cho RNN 91
4.5. Thực nghiệm với PyTorch 91
4.6. Thực nghiệm với Keras 95
4.7. Kết quả dự báo 97
4.8. Phân tích kết quả trên Customer Behavior 98
4.9. Phân tích kết quả trên AAPL 99

4.10. So sánh PyTorch và Keras 100
4.11. Hạn chế và hướng phát triển 101
4.12. Demo web 102
4.13. Tiểu kết chương 104
CHƯƠNG 5. KNOWLEDGE GRAPH VÀ RETRIEVAL-AUGMENTED GENERATION (RAG) 105
5.1. Mở đầu 105
5.1.1. Knowledge Graph và RAG: lịch sử phát triển 106
5.1.2. Vai trò và ý nghĩa của Knowledge Graph và RAG trong trí tuệ nhân tạo 109
5.2. Knowledge Graph và Knowledge Base (KB) 111
5.2.1. Khái niệm và mục tiêu xây dựng Knowledge Graph 111
5.2.2. Mô hình hóa tri thức và cấu trúc đồ thị 112
5.2.3. Lưu trữ KG bằng Neo4j và truy xuất tri thức 116
5.3. Retrieval-Augmented Generation (RAG) 119
5.3.1. Khái niệm và nguyên lý hoạt động 119
5.3.2. Chuẩn bị tài liệu và xây dựng kho RAG 119
5.3.3. Truy xuất tài liệu bằng TF-IDF và cosine similarity 120
5.3.4. Kết hợp KG và RAG bằng Reciprocal Rank Fusion 122
5.3.5. Vai trò của RAG trong hệ thống và giới hạn hiện tại 124
5.3.6. Thử nghiệm so sánh phương pháp truy xuất TF-IDF và Embedding 124
5.4. Knowledge Graph và RAG cho bộ dữ liệu bệnh tiểu đường (Diabetes) 127
5.4.1. Mục tiêu và đặc điểm dữ liệu 127
5.4.2. Xây dựng Knowledge Graph cho Diabetes 128
5.4.3. Truy xuất tri thức bằng KG và RAG 129
5.4.4. Đánh giá khả năng truy xuất trên Diabetes 130
5.4.5. Nhận xét 131
5.5. Knowledge Graph và RAG cho dữ liệu chứng khoán 131
5.5.1. Mục tiêu và đặc điểm dữ liệu 131

5.5.2. Chuẩn bị tài liệu tri thức chứng khoán 132
5.5.3. Truy xuất bằng chứng bằng KG, RAG và KG+RAG 133
5.5.4. Đánh giá khả năng truy xuất trên dữ liệu chứng khoán 134
5.5.5. Nhận xét 135
5.6. Knowledge Graph và RAG cho dữ liệu giá vàng 135
5.6.1. Mục tiêu và đặc điểm dữ liệu 135
5.6.2. Chuẩn bị tài liệu tri thức giá vàng 136
5.6.3. Truy xuất tri thức trên dữ liệu giá vàng 137
5.6.4. Đánh giá khả năng truy xuất trên dữ liệu giá vàng 138
5.6.5. Nhận xét 139
5.7. Deploy 139
5.7.1. Mục tiêu triển khai 139
5.7.2. Kiến trúc và các thành phần triển khai 140
5.7.3. Backend và các API của hệ thống 142
5.7.4. Giao diện người dùng 143
5.7.5. Quy trình chạy và sử dụng ứng dụng 144
5.7.6. Minh họa quá trình truy xuất và tổng hợp câu trả lời 146
5.7.7. Nhận xét về kết quả triển khai 147
TÀI LIỆU THAM KHẢO 150
PHỤ LỤC 151
1. Bộ dữ liệu giá nhà 151
2. Bộ dữ liệu đánh giá sản phẩm thời trang 151
3. Bộ dữ liệu chữ số viết tay MNIST 151
4. Bộ dữ liệu ảnh thời trang Fashion-MNIST 151
5. Bộ dữ liệu Pima Indians Diabetes 151
6. Bộ dữ liệu giao dịch bán lẻ trực tuyến 152
7. Bộ dữ liệu Diabetes Health Indicators 152
8. Bộ dữ liệu chứng khoán S&P 500 152

9. Bộ dữ liệu giá vàng lịch sử 152

DANH MỤC HÌNH ẢNH

Hình 1.1. Mối quan hệ giữa AI, Machine Learning và Deep Learning.....17
Hình 1.2. Mô hình minh họa Turing Test.....19
Hình 1.4. Cấu trúc cơ bản của hệ chuyên gia 20
Hình 1.5. Quy trình cơ bản của Machine Learning 22
Hình 1.6. Sự phát triển từ Machine Learning đến Deep Learning 24
Hình 1.7. Cấu trúc tổng quát của một mạng CNN.....25
Hình 1.8. Minh họa cách RNN xử lý dữ liệu tuần tự 25
Hình 2.1 – Phân bố biến mục tiêu của bộ dữ liệu Diabetes.....44
Hình 2.2 – Confusion Matrix của mô hình phân loại Diabetes 45
Hình 2.3 – Phân bố giá nhà trong tập dữ liệu 46
Hình 2.4 – So sánh giá thực tế và giá dự đoán 47
Hình 2.5 – Phân bố nhãn của các review.....48
Hình 2.6 – Giao diện web Diabetes Prediction: màn hình nhập và kết quả. 56
Hình 2.7 – Giao diện mobile Diabetes Prediction. 57
Hình 2.8 – Giao diện kết quả mobile Diabetes Prediction 57
Hình 2.9 – Giao diện web House Price Prediction. 58
Hình 2.10 – Giao diện kết quả web House Price Prediction. 58
Hình 2.11 – Giao diện mobile House Price Prediction.....59
Hình 2.12 – Giao diện kết quả mobile House Price Prediction. 59
Hình 3.1 – Một số ảnh mẫu MNIST 64
Hình 3.2 – Một số ảnh mẫu Fashion-MNIST 73
Hình 3.3 – Biểu đồ so sánh Accuracy của các mô hình trên MNIST.....75
Hình 3.4 – Training Loss và Validation Loss trên MNIST 76
Hình 3.5 – Biểu đồ Accuracy của các mô hình trên Fashion-MNIST.....77
Hình 3.6 – Training/Validation Loss trên Fashion-MNIST 78
Hình 3.7 – Biểu đồ Accuracy của M1–M4 trên Diabetes 79
Hình 3.8 – Training/Validation Loss của Diabetes 80

Hình 3.9 – Giao diện Web demo nhận dạng chữ số bằng CNN 84
Hình 3.10 – Kết quả dự đoán chữ số trên Web 84
Hình 4.1. Sơ đồ hoạt động của RNN 86
Hình 4.2. Doanh thu hằng ngày của Customer Behavior 90
Hình 4.3. Giá đóng cửa AAPL theo thời gian 90
Hình 4.4. Cấu trúc tổng quát 92
Hình 4.5. Loss train và validation của RNN PyTorch 95
Hình 4.6. Giá trị doanh thu thực tế và giá trị dự đoán bằng RNN 99
Hình 4.7. Giá AAPL thực tế và giá dự đoán bằng RNN 100
Hình 4.8. Giao diện web dự đoán AAPL Stock và Customer Behavior 103
Hình 4.9. Kết quả dự đoán với AAPL Stock 103
Hình 4.10. Kết quả dự đoán Revenue 104
Hình 5.1. Quá trình phát triển Knowledge Graph và RAG. 108
Hình 5.2. Vai trò của Knowledge Graph và RAG trong hệ thống AI hỏi đáp có truy xuất nguồn. 111
Hình 5.3. Một phần Knowledge Graph của bộ dữ liệu Diabetes trên Neo4j 118
Hình 5.4. Tìm số câu hỏi có kết quả khác nhau giữa hai phương pháp 126
Hình 5.5. Phân bố biến mục tiêu Diabetes trong bộ dữ liệu. 128
Hình 5.6. So sánh tỷ lệ truy xuất bằng chứng của các phương pháp trên Diabetes. 131
Hình 5.7. Một phần dữ liệu chứng khoán sau khi kiểm tra trong notebook. 133
Hình 5.8. So sánh Hit@5 của các phương pháp trên dữ liệu chứng khoán. 135
Hình 5.9. Thông tin dữ liệu giá vàng sau khi kiểm tra trong notebook. 137
Hình 5.10. So sánh Hit@5 của các phương pháp trên dữ liệu giá vàng. 138
Hình 5.11. Cấu trúc thư mục triển khai ứng dụng KG-RAG tích hợp Gemini. 141
Hình 5.12. Giao diện chính của hệ thống KG-RAG tích hợp Gemini 144
Hình 5.13. Kết quả truy xuất và tổng hợp câu trả lời trên dữ liệu Diabetes. 146
Hình 5.14. Kết quả truy xuất trên dữ liệu chứng khoán. 147
Hình 5.15. Kết quả truy xuất trên dữ liệu giá vàng. 147

MỞ ĐẦU

1. Bối cảnh đề tài

Trong những năm gần đây, trí tuệ nhân tạo (Artificial Intelligence – AI) đã trở thành một trong những lĩnh vực công nghệ có tốc độ phát triển nhanh và có ảnh hưởng sâu rộng đến nhiều lĩnh vực của đời sống. Từ những hệ thống xử lý dữ liệu đơn giản, AI ngày nay đã có khả năng nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên, dự đoán xu hướng, hỗ trợ ra quyết định và tương tác với con người. Sự phát triển của AI không chỉ đến từ những thuật toán mới mà còn là kết quả của sự kết hợp giữa dữ liệu lớn, năng lực tính toán ngày càng cao và sự tiến bộ của các phương pháp học máy, học sâu.

Quá trình phát triển của AI đã trải qua nhiều giai đoạn khác nhau. Ban đầu, các hệ thống AI chủ yếu dựa trên những luật được con người xây dựng và biểu diễn tri thức dưới dạng các quy tắc logic. Tuy nhiên, cách tiếp cận này gặp nhiều hạn chế khi phải xử lý những bài toán có dữ liệu lớn và phức tạp. Sự phát triển của Machine Learning đã tạo ra một hướng tiếp cận mới, trong đó máy tính có thể học các quy luật từ dữ liệu thay vì phải được lập trình trực tiếp cho từng trường hợp.

Tiếp theo, Deep Learning đã tạo ra những bước tiến đáng kể trong nhiều bài toán như nhận dạng hình ảnh, xử lý tiếng nói và xử lý ngôn ngữ. Các kiến trúc mạng nơ-ron khác nhau được phát triển nhằm giải quyết những dạng dữ liệu khác nhau. Trong đó, mạng nơ-ron tích chập (Convolutional Neural Network – CNN) đặc biệt phù hợp với dữ liệu hình ảnh, trong khi mạng nơ-ron hồi quy (Recurrent Neural Network – RNN) được phát triển để xử lý dữ liệu có tính tuần tự.

Bên cạnh khả năng học từ dữ liệu, các hệ thống AI hiện đại ngày càng cần khả năng khai thác tri thức, truy xuất thông tin và kết hợp nhiều thành phần AI để thực hiện nhiệm vụ phức tạp. Đây là cơ sở cho sự xuất hiện và phát triển của Knowledge Graph, Retrieval-Augmented Generation (RAG), Deep Graph và các hệ đa Agent. Những hướng tiếp cận này mở rộng phạm vi của AI từ việc chỉ dự đoán hoặc phân loại dữ liệu sang khả năng tổ chức tri thức, tìm kiếm thông tin liên quan, suy luận trên các mối quan hệ và phối hợp nhiều tác nhân để giải quyết nhiệm vụ.

Xuất phát từ sự phát triển liên tục đó, tiểu luận được thực hiện nhằm tìm hiểu một cách có hệ thống quá trình phát triển của AI, các kỹ thuật Machine Learning cơ bản, những kiến trúc Deep Learning tiêu biểu như CNN và RNN, đồng thời tiếp cận một số hướng phát triển hiện đại gồm Knowledge Graph và RAG, Deep Graph và hệ đa Agent.

Thông qua việc kết hợp giữa cơ sở lý thuyết và thực nghiệm trên dữ liệu, tiểu luận hướng tới việc giúp người học có cái nhìn tổng quan về quá trình hình thành, phát triển và mở rộng của AI, từ những phương pháp học máy cơ bản đến các hệ thống AI hiện đại có khả năng khai thác tri thức và phối hợp nhiều tác nhân.

2. Mục tiêu của tiểu luận

Tiểu luận được thực hiện với mục tiêu tìm hiểu các kiến thức nền tảng và một số hướng phát triển quan trọng của trí tuệ nhân tạo.

Các mục tiêu chính gồm:

- Tìm hiểu quá trình hình thành và phát triển của trí tuệ nhân tạo qua các giai đoạn khác nhau.
- Tìm hiểu những kỹ thuật Machine Learning cơ bản và cách áp dụng các kỹ thuật này vào các bài toán thực tế.
- Tìm hiểu nguyên lý hoạt động, cấu trúc và đặc điểm của mạng nơ-ron tích chập CNN.
- Tìm hiểu nguyên lý hoạt động của mạng nơ-ron hồi quy RNN và khả năng xử lý dữ liệu tuần tự.
- Tìm hiểu khái niệm Knowledge Graph và cách biểu diễn tri thức dưới dạng các thực thể và mối quan hệ.
- Tìm hiểu RAG và cách kết hợp khả năng truy xuất thông tin với mô hình sinh nhằm hỗ trợ việc tạo ra câu trả lời dựa trên nguồn tri thức bên ngoài.
- Tìm hiểu Deep Graph và khả năng áp dụng các phương pháp học sâu trên dữ liệu dạng đồ thị.
- Tìm hiểu hệ đa Agent và cách nhiều Agent có thể phối hợp để thực hiện những nhiệm vụ phức tạp.
- Thực hiện các thí nghiệm trên dữ liệu phù hợp nhằm minh họa cho những nội dung lý thuyết đã nghiên cứu.
- Đánh giá và so sánh kết quả của các mô hình thông qua những chỉ số phù hợp với từng bài toán.

Qua đó, tiểu luận không chỉ tập trung vào việc trình bày lý thuyết mà còn hướng tới việc kết nối kiến thức với quá trình xây dựng và đánh giá các mô hình AI.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của tiểu luận là các phương pháp, mô hình và kiến trúc tiêu biểu trong lĩnh vực trí tuệ nhân tạo, tập trung từ những kỹ thuật Machine Learning cơ bản đến một số hướng phát triển hiện đại của AI.

Phạm vi nghiên cứu được chia thành các nội dung chính:

- Lịch sử phát triển AI: tìm hiểu quá trình hình thành, các giai đoạn phát triển và những thay đổi trong cách tiếp cận bài toán trí tuệ nhân tạo.
- Machine Learning: nghiên cứu một số kỹ thuật học máy cơ bản, cách chuẩn bị dữ liệu, huấn luyện mô hình và đánh giá kết quả.
- CNN: nghiên cứu cấu trúc của mạng CNN, các thành phần như convolution, activation, pooling và fully connected layer, đồng thời tìm hiểu khả năng ứng dụng CNN vào xử lý dữ liệu hình ảnh.
- RNN: nghiên cứu đặc điểm của dữ liệu tuần tự, nguyên lý hoạt động của RNN và khả năng sử dụng RNN trong các bài toán dự báo hoặc xử lý chuỗi dữ liệu.
- Knowledge Graph và RAG: tìm hiểu cách biểu diễn, tổ chức và khai thác tri thức, cũng như cơ chế kết hợp truy xuất thông tin với mô hình sinh.
- Deep Graph: nghiên cứu việc áp dụng các phương pháp học sâu cho dữ liệu đồ thị và khả năng học biểu diễn trên các nút, cạnh hoặc toàn bộ đồ thị.
- Hệ đa Agent: tìm hiểu mô hình gồm nhiều Agent có khả năng thực hiện các nhiệm vụ khác nhau và phối hợp với nhau để giải quyết một bài toán chung.

Trong phạm vi của tiểu luận, các nội dung được trình bày ở mức phù hợp với mục tiêu môn học, kết hợp giữa kiến thức lý thuyết, minh họa bằng dữ liệu và thực nghiệm khi cần thiết.

4. Phương pháp thực hiện

Tiểu luận được thực hiện theo hướng kết hợp giữa nghiên cứu lý thuyết, thực nghiệm và đánh giá kết quả.

Trước hết, các khái niệm và nguyên lý cơ bản liên quan đến AI, Machine Learning và Deep Learning được tìm hiểu nhằm xây dựng nền tảng lý thuyết. Sau đó, các kỹ thuật và kiến trúc được phân tích theo đặc điểm của từng loại dữ liệu và bài toán.

Đối với phần thực nghiệm, dữ liệu được chuẩn bị và tiền xử lý trước khi đưa vào mô hình. Tùy theo từng bài toán, dữ liệu có thể được sử dụng cho các nhiệm vụ như phân loại, hồi quy, xử lý hình ảnh hoặc xử lý chuỗi.

Quá trình thực nghiệm có thể được mô tả tổng quát qua các bước:

Dữ liệu → Tiền xử lý → Xây dựng mô hình → Huấn luyện → Đánh giá → So sánh kết quả

Các công cụ lập trình và môi trường thực nghiệm được sử dụng để xây dựng, huấn luyện và đánh giá mô hình. Kết quả được thể hiện thông qua các bảng số liệu, biểu đồ hoặc hình ảnh minh họa nhằm giúp quá trình phân tích trực quan và dễ theo dõi hơn.

Đối với những mô hình có thể so sánh, các tiêu chí đánh giá phù hợp sẽ được sử dụng để phân tích ưu điểm, hạn chế và mức độ phù hợp của từng phương pháp đối với bài toán tương ứng.

5. Ý nghĩa của tiểu luận

Về mặt kiến thức, tiểu luận giúp hệ thống hóa những nội dung quan trọng trong quá trình phát triển của trí tuệ nhân tạo. Nội dung được xây dựng theo hướng từ nền tảng đến nâng cao, bắt đầu từ lịch sử phát triển AI, tiếp đến Machine Learning, CNN, RNN và sau đó mở rộng sang các hướng tiếp cận hiện đại như Knowledge Graph và RAG, Deep Graph và hệ đa Agent.

Việc nghiên cứu theo trình tự này giúp làm rõ mối quan hệ giữa các hướng phát triển của AI. Nếu Machine Learning tập trung vào khả năng học từ dữ liệu, Deep Learning mở rộng khả năng học biểu diễn thông qua mạng nơ-ron nhiều lớp, thì những phương pháp hiện đại hơn tiếp tục mở rộng khả năng của hệ thống AI thông qua việc khai thác tri thức, xử lý cấu trúc đồ thị và phối hợp nhiều Agent.

Về mặt thực hành, quá trình xây dựng các thí nghiệm giúp người thực hiện có cơ hội áp dụng kiến thức lý thuyết vào dữ liệu và mô hình cụ thể. Thông qua quá trình huấn luyện và đánh giá, có thể quan sát được ảnh hưởng của dữ liệu, kiến trúc mô hình và phương pháp học đến kết quả dự đoán.

Ngoài ra, việc tìm hiểu các hướng tiếp cận hiện đại như Knowledge Graph, RAG, Deep Graph và hệ đa Agent giúp mở rộng góc nhìn về AI, từ những mô hình dự đoán đơn lẻ sang những hệ thống có khả năng khai thác thông tin, biểu diễn quan hệ và phối hợp để giải quyết nhiệm vụ.

6. Cấu trúc của tiểu luận

Ngoài phần Mở đầu, tiểu luận gồm 7 chương, nội dung được tổ chức theo hướng từ kiến thức nền tảng đến các phương pháp và hệ thống AI hiện đại.

Chương 1: Lịch sử phát triển AI trình bày quá trình hình thành và phát triển của trí tuệ nhân tạo qua các giai đoạn. Nội dung tập trung vào những dấu mốc quan trọng, sự thay đổi trong cách tiếp cận AI và quá trình chuyển dịch từ các phương pháp dựa trên luật sang Machine Learning và Deep Learning.

Chương 2: Các kỹ thuật ML cơ bản trình bày những kỹ thuật Machine Learning cơ bản, các dạng bài toán thường gặp và quy trình xây dựng mô hình học máy. Nội dung kết hợp giữa lý thuyết, dữ liệu, thực nghiệm và đánh giá nhằm minh họa cách các kỹ thuật Machine Learning được áp dụng vào bài toán thực tế.

Chương 3: CNN tập trung vào mạng nơ-ron tích chập (Convolutional Neural Network – CNN). Nội dung trình bày cấu trúc, nguyên lý hoạt động của các thành phần trong CNN, quá trình huấn luyện và khả năng ứng dụng CNN vào các bài toán xử lý hình ảnh. Các mô hình và kết quả thực nghiệm được sử dụng để minh họa khả năng của CNN.

Chương 4: RNN trình bày mạng nơ-ron hồi quy (Recurrent Neural Network – RNN), tập trung vào đặc điểm của dữ liệu tuần tự và cơ chế duy trì thông tin qua các bước thời gian. Chương cũng tìm hiểu cách sử dụng RNN cho các bài toán dự báo chuỗi và đánh giá kết quả của mô hình thông qua thực nghiệm.

Chương 5: Knowledge Graph và RAG giới thiệu Knowledge Graph và Retrieval-Augmented Generation (RAG). Nội dung tập trung vào cách biểu diễn và tổ chức tri thức dưới dạng đồ thị, cơ chế truy xuất thông tin và cách RAG kết hợp nguồn dữ liệu bên ngoài với mô hình sinh nhằm nâng cao khả năng cung cấp thông tin của hệ thống AI.

Chương 6: Deep Graph tìm hiểu các phương pháp Deep Graph và việc áp dụng kỹ thuật học sâu cho dữ liệu có cấu trúc đồ thị. Nội dung tập trung vào khả năng học biểu diễn của các nút và mối quan hệ trong đồ thị, cũng như các bài toán có thể giải quyết bằng phương pháp học sâu trên đồ thị.

Chương 7: Hệ đa Agent trình bày khái niệm hệ đa Agent và cách nhiều Agent có thể phối hợp với nhau để giải quyết một nhiệm vụ. Nội dung tập trung vào vai trò của từng Agent, cơ chế tương tác và phối hợp giữa các Agent, qua đó cho thấy xu hướng phát triển từ một mô hình AI đơn lẻ sang hệ thống gồm nhiều thành phần có khả năng cộng tác.

7. Tiểu kết phần mở đầu

Tiểu luận được xây dựng nhằm khảo sát quá trình phát triển của trí tuệ nhân tạo từ những giai đoạn đầu đến các phương pháp và hệ thống hiện đại. Nội dung được tổ chức theo một lộ trình từ nền tảng đến nâng cao: lịch sử phát triển AI → Machine Learning → CNN → RNN → Knowledge Graph và RAG → Deep Graph → Hệ đa Agent.

Cách tổ chức này giúp tạo ra một mạch nội dung thống nhất, trong đó mỗi chương tiếp tục mở rộng từ những kiến thức được trình bày ở các chương trước. Bên cạnh phần lý thuyết, các nội dung thực nghiệm và minh họa được sử dụng để làm rõ cách các phương pháp AI hoạt động trên dữ liệu thực tế.

CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN TRÍ TUỆ NHÂN TẠO

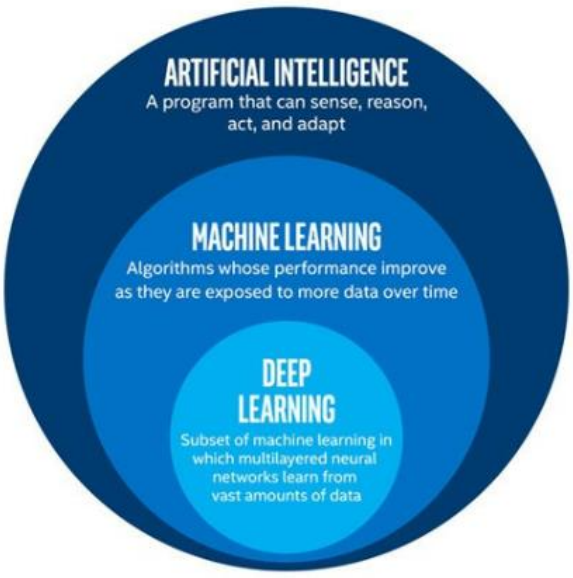
1.1. Tổng quan về trí tuệ nhân tạo

Trí tuệ nhân tạo, thường được gọi là AI (Artificial Intelligence), là lĩnh vực nghiên cứu nhằm xây dựng các hệ thống máy tính có khả năng thực hiện những nhiệm vụ đòi hỏi một mức độ thông minh nhất định. Những nhiệm vụ này có thể bao gồm suy luận, học tập, nhận dạng, dự đoán, lập kế hoạch, xử lý ngôn ngữ và ra quyết định.

Khái niệm "thông minh" trong AI không nhất thiết có nghĩa là máy móc phải suy nghĩ giống hoàn toàn con người. Trong nhiều trường hợp, một hệ thống AI chỉ cần thực hiện tốt một nhiệm vụ cụ thể bằng cách phân tích dữ liệu và đưa ra kết quả phù hợp. Chẳng hạn, một mô hình có thể nhận diện chữ số viết tay, dự đoán giá của một căn nhà, phân loại một email hoặc dự đoán giá trị tiếp theo của một chuỗi thời gian.

AI là một lĩnh vực có tính liên ngành cao. Nó có mối quan hệ với khoa học máy tính, toán học, thống kê, khoa học dữ liệu, xử lý tín hiệu, tâm lý học nhận thức và nhiều lĩnh vực khác. Trong quá trình phát triển, AI đã thay đổi đáng kể về phương pháp tiếp cận. Các hệ thống ban đầu thường dựa vào luật và tri thức được con người xây dựng trực tiếp, trong khi các hệ thống hiện đại có xu hướng học quy luật từ dữ liệu.

Có thể xem quá trình phát triển của AI là quá trình chuyển dần từ "lập trình cho máy biết phải làm gì" sang "cung cấp dữ liệu để máy học cách thực hiện nhiệm vụ". Sự thay đổi này là nền tảng dẫn đến sự phát triển mạnh mẽ của học máy và học sâu trong những thập niên gần đây.



Hình 1.1. Mối quan hệ giữa AI, Machine Learning và Deep Learning

Trong đó, học máy là một nhánh của AI tập trung vào khả năng học từ dữ liệu. Học sâu là một nhánh của học máy sử dụng các mạng nơ-ron có nhiều lớp để học các biểu diễn phức tạp từ dữ liệu.

1.2. Những ý tưởng ban đầu về máy thông minh

Ý tưởng về việc tạo ra những cỗ máy có khả năng thực hiện các nhiệm vụ trí tuệ đã xuất hiện từ rất lâu trước khi máy tính điện tử hiện đại ra đời. Con người từ lâu đã sử dụng các công cụ cơ học để hỗ trợ tính toán và xử lý thông tin. Sự phát triển của toán học, logic và cơ học dần tạo tiền đề cho việc hình thành tư tưởng rằng một số hoạt động suy luận của con người có thể được mô tả bằng các quy tắc.

Một bước tiến quan trọng là sự phát triển của các thiết bị tính toán cơ học. Những thiết bị này chưa thể được xem là AI nhưng đã cho thấy khả năng chuyển một số thao tác tính toán từ con người sang máy móc. Khi máy tính điện tử xuất hiện, khả năng xử lý thông tin của máy được mở rộng rất lớn.

Trong bối cảnh đó, một câu hỏi quan trọng được đặt ra: nếu hoạt động suy luận có thể được biểu diễn bằng các quy tắc và máy tính có thể thực hiện các quy tắc đó, liệu máy tính có thể thực hiện những nhiệm vụ vốn được xem là cần đến trí thông minh hay không?

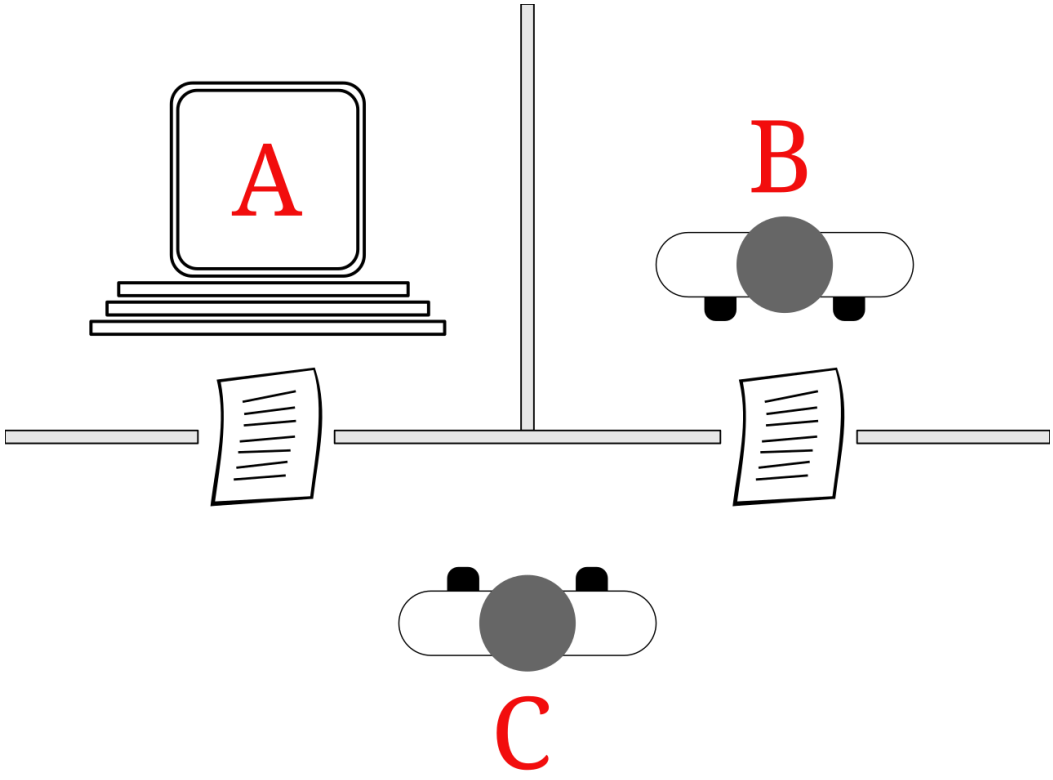
Câu hỏi này đã trở thành một trong những cơ sở quan trọng cho sự hình thành của lĩnh vực trí tuệ nhân tạo.

1.3. Alan Turing và câu hỏi về khả năng suy nghĩ của máy

Alan Turing là một trong những nhà khoa học có ảnh hưởng lớn đến nền tảng lý thuyết của khoa học máy tính và AI. Một trong những đóng góp quan trọng của ông là đặt ra vấn đề về khả năng máy móc có thể thể hiện hành vi thông minh.

Thay vì cố gắng đưa ra một định nghĩa tuyệt đối về "máy có biết suy nghĩ hay không", Turing đưa ra một cách tiếp cận dựa trên khả năng tương tác. Ý tưởng này về sau được biết đến với tên gọi Turing Test.

Trong một mô hình đơn giản, một người thực hiện giao tiếp với một đối tượng mà không biết đối tượng đó là con người hay máy. Nếu thông qua quá trình giao tiếp, người đánh giá không thể phân biệt được máy với con người dựa trên câu trả lời, hệ thống có thể được xem là thể hiện một mức độ hành vi thông minh.



Hình 1.2. Mô hình minh họa Turing Test

Turing Test có ý nghĩa quan trọng bởi nó chuyển câu hỏi "máy có thực sự suy nghĩ hay không?" thành một vấn đề có thể quan sát và đánh giá thông qua hành vi. Mặc dù ngày nay Turing Test không phải là tiêu chí duy nhất để đánh giá một hệ thống AI, tư tưởng này vẫn có giá trị lịch sử rất lớn đối với quá trình hình thành ngành trí tuệ nhân tạo.

1.4. Hội thảo Dartmouth và sự hình thành thuật ngữ Artificial Intelligence

Năm 1956 được xem là một cột mốc quan trọng trong lịch sử AI. Tại hội thảo Dartmouth, các nhà nghiên cứu đã thảo luận về khả năng mô phỏng các khía cạnh của trí thông minh bằng máy tính. Thuật ngữ Artificial Intelligence được sử dụng để mô tả lĩnh vực nghiên cứu này.

Từ thời điểm đó, AI dần hình thành như một lĩnh vực nghiên cứu riêng biệt. Các nhà khoa học bắt đầu nghiên cứu những vấn đề như biểu diễn tri thức, suy luận tự động, giải quyết bài toán, lập kế hoạch và xử lý ngôn ngữ.

Quan điểm chủ đạo trong giai đoạn đầu cho rằng nhiều hoạt động trí tuệ có thể được mô tả bằng các ký hiệu và quy tắc. Nếu tri thức được biểu diễn dưới dạng phù hợp, máy tính có thể sử dụng các quy tắc để suy luận và đưa ra kết luận.

Đây là cơ sở cho sự phát triển của hướng tiếp cận được gọi là AI biểu tượng (Symbolic AI).

1.5. AI biểu tượng và phương pháp dựa trên luật

Trong giai đoạn đầu, nhiều hệ thống AI được xây dựng dựa trên các ký hiệu, luật và tri thức do con người cung cấp. Một hệ thống có thể được thiết kế với tập luật dạng:

- Nếu điều kiện A xảy ra và điều kiện B đúng thì thực hiện kết luận C.
- Ví dụ, trong một hệ thống chẩn đoán đơn giản, nếu một người có một nhóm triệu chứng nhất định thì hệ thống có thể đưa ra một kết luận dựa trên các luật đã được xây dựng trước.

Ưu điểm của phương pháp này là cách hoạt động tương đối dễ giải thích. Khi hệ thống đưa ra một kết luận, người phát triển có thể kiểm tra những luật nào đã được kích hoạt. Điều này đặc biệt hữu ích trong các bài toán mà việc giải thích quyết định có ý nghĩa quan trọng.

Tuy nhiên, phương pháp dựa trên luật cũng có những hạn chế rõ rệt. Trong thực tế, nhiều bài toán có số lượng tình huống rất lớn và khó có thể mô tả toàn bộ bằng các luật thủ công. Khi dữ liệu thay đổi hoặc xuất hiện tình huống mới, hệ thống có thể cần bổ sung hoặc điều chỉnh nhiều luật.

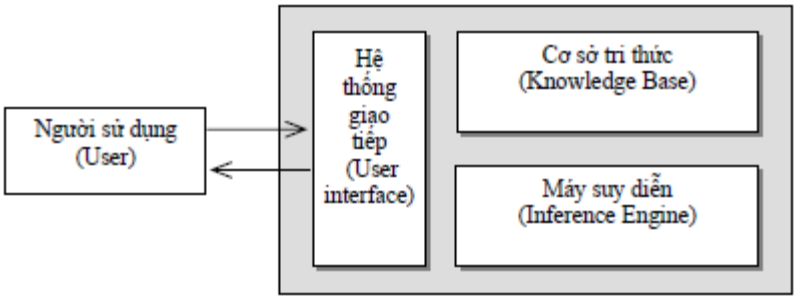
Hạn chế này dần thúc đẩy sự phát triển của các phương pháp có khả năng học trực tiếp từ dữ liệu.

1.6. Sự phát triển của hệ chuyên gia

Trong thập niên 1970 và 1980, hệ chuyên gia trở thành một hướng nghiên cứu và ứng dụng quan trọng của AI. Hệ chuyên gia được xây dựng với mục tiêu mô phỏng một phần kiến thức và cách suy luận của chuyên gia trong một lĩnh vực cụ thể.

Một hệ chuyên gia thường gồm hai thành phần quan trọng là cơ sở tri thức và bộ suy diễn. Cơ sở tri thức lưu trữ các sự kiện và luật, trong khi bộ suy diễn sử dụng những thông tin này để tạo ra kết luận.

Mô hình tổng quát có thể được minh họa như sau:



Hình 1.4. Cấu trúc cơ bản của hệ chuyên gia

Hệ chuyên gia cho thấy máy tính có thể thực hiện một số nhiệm vụ có tính chất chuyên môn nếu tri thức của con người được biểu diễn phù hợp. Tuy nhiên, quá trình xây dựng và duy trì cơ sở tri thức thường tốn nhiều công sức. Đây cũng là một trong những nguyên nhân khiến hướng tiếp cận dựa trên tri thức gặp khó khăn khi mở rộng sang những bài toán phức tạp.

1.7. Giai đoạn AI Winter

Trong lịch sử phát triển của AI, không phải giai đoạn nào cũng có tốc độ phát triển liên tục. Có những thời kỳ kỳ vọng dành cho AI rất cao nhưng kết quả thực tế không đạt được như mong muốn. Khi nguồn tài trợ và sự quan tâm giảm xuống, hoạt động nghiên cứu AI cũng bị ảnh hưởng. Những giai đoạn suy giảm này thường được gọi là AI Winter.

Một nguyên nhân quan trọng là năng lực tính toán và dữ liệu trong thời kỳ đầu còn hạn chế. Các thuật toán có thể hoạt động tốt trong những bài toán nhỏ nhưng gặp khó khăn khi phải xử lý vấn đề có quy mô lớn. Bên cạnh đó, việc biểu diễn tri thức và xây dựng luật thủ công không dễ dàng mở rộng.

AI Winter cho thấy một bài học quan trọng: sự phát triển của trí tuệ nhân tạo không chỉ phụ thuộc vào ý tưởng thuật toán mà còn phụ thuộc vào dữ liệu, năng lực tính toán, phương pháp huấn luyện và khả năng triển khai trong thực tế.

1.8. Sự chuyển dịch từ lập trình luật sang học máy

Một thay đổi quan trọng trong lịch sử AI là sự chuyển dịch từ việc mô tả trực tiếp các quy tắc sang việc sử dụng dữ liệu để máy tự học.

Trong phương pháp truyền thống, con người phải xác định quy tắc: Dữ liệu + Luật do con người xây dựng → Kết quả. Trong học máy, quá trình được thay đổi thành: Dữ liệu + Kết quả mong muốn → Mô hình học được quy luật

Sự thay đổi này có ý nghĩa rất lớn. Thay vì phải xây dựng thủ công hàng loạt luật cho mọi tình huống, mô hình có thể tìm kiếm các mối quan hệ thống kê trong dữ liệu.

Ví dụ, trong bài toán dự đoán giá nhà, thay vì viết một tập luật cố định như “nhà có diện tích lớn thì giá cao”, mô hình có thể được huấn luyện trên dữ liệu gồm diện tích, số phòng, vị trí, tuổi nhà và giá thực tế. Từ đó, mô hình học được mối quan hệ giữa các đặc trưng và giá nhà.

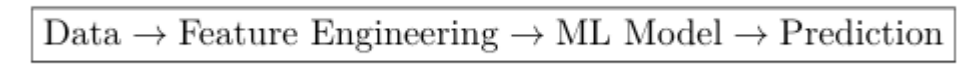
Đây chính là nền tảng của học máy hiện đại.

1.9. Sự phát triển của Machine Learning

Machine Learning là một nhánh của AI tập trung vào việc xây dựng các thuật toán có khả năng học từ dữ liệu. Thay vì chỉ thực hiện các quy tắc được lập trình sẵn, mô hình sử dụng dữ liệu huấn luyện để tìm ra các tham số hoặc cấu trúc phù hợp với bài toán.

Quá trình học máy thường bao gồm các bước:

- Thu thập dữ liệu.
- Làm sạch và tiền xử lý dữ liệu.
- Lựa chọn đặc trưng.
- Chia dữ liệu thành các tập phù hợp.
- Lựa chọn thuật toán.
- Huấn luyện mô hình.
- Đánh giá mô hình.
- Điều chỉnh mô hình.
- Sử dụng mô hình trên dữ liệu mới.



Hình 1.5. Quy trình cơ bản của Machine Learning

Sự phát triển của học máy đã mở rộng đáng kể khả năng ứng dụng AI vào các bài toán thực tế.

1.9.1. Bài toán phân loại

Phân loại là bài toán trong đó mô hình cần xác định một đối tượng thuộc về lớp nào. Ví dụ, dữ liệu có thể được phân thành hai lớp hoặc nhiều lớp khác nhau.

Trong phân loại nhị phân, kết quả thường có hai khả năng. Mô hình được huấn luyện dựa trên dữ liệu đã biết nhãn và sau đó dự đoán nhãn cho các mẫu mới.

Các chỉ số thường được sử dụng để đánh giá mô hình phân loại gồm Accuracy, Precision, Recall và F1-score.

Accuracy phản ánh tỷ lệ dự đoán đúng trên tổng số mẫu. Precision phản ánh mức độ chính xác của những dự đoán thuộc một lớp nhất định. Recall phản ánh khả năng mô hình phát hiện đúng các mẫu thực sự thuộc lớp đó. F1-score kết hợp Precision và Recall thành một chỉ số duy nhất.

1.9.2. Bài toán hồi quy

Khác với phân loại, hồi quy nhằm dự đoán một giá trị liên tục. Ví dụ có thể dự đoán giá nhà, doanh thu, nhiệt độ hoặc một giá trị số trong tương lai.

Một số mô hình hồi quy cơ bản có thể được sử dụng để tìm mối quan hệ giữa biến đầu vào và biến mục tiêu. Chất lượng mô hình thường được đánh giá bằng các chỉ số như MAE, RMSE và R^2.

Sự phát triển của các phương pháp phân loại và hồi quy đã tạo nền tảng quan trọng cho nhiều ứng dụng AI hiện nay.

1.10. Sự phát triển của mạng nơ-ron nhân tạo

Bên cạnh các thuật toán học máy truyền thống, mạng nơ-ron nhân tạo (Artificial Neural Network – ANN) được phát triển dựa trên ý tưởng mô phỏng một cách đơn giản hoạt động của mạng nơ-ron sinh học.

Một nơ-ron nhân tạo nhận các giá trị đầu vào, nhân với các trọng số tương ứng, cộng thêm bias và đưa kết quả qua hàm kích hoạt. Nhiều nơ-ron được kết nối thành các lớp để tạo thành mạng nơ-ron.

Một mạng nơ-ron cơ bản thường gồm:

- Lớp đầu vào.
- Một hoặc nhiều lớp ẩn.
- Lớp đầu ra.

Trong quá trình huấn luyện, mô hình điều chỉnh các trọng số nhằm giảm sai số giữa kết quả dự đoán và giá trị thực tế. Phương pháp lan truyền ngược (Backpropagation) kết hợp với các thuật toán tối ưu đã trở thành cơ chế quan trọng trong quá trình huấn luyện mạng nơ-ron.

Khi số lượng lớp tăng lên, mạng có thể học được các biểu diễn phức tạp hơn. Từ đây, khái niệm Deep Learning dần trở thành một hướng phát triển quan trọng.

1.11. Sự hình thành và phát triển của Deep Learning

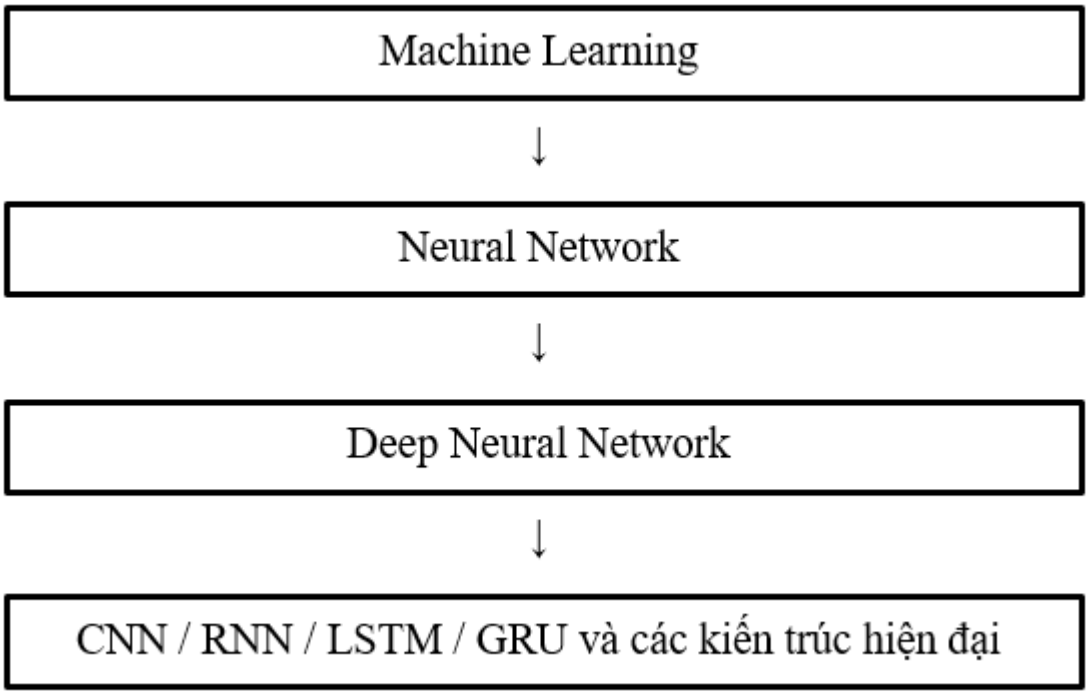
Deep Learning có thể được xem là sự phát triển mạnh của mạng nơ-ron nhân tạo với nhiều lớp. Điểm quan trọng của học sâu là mô hình có thể tự học các biểu diễn dữ liệu ở nhiều mức độ khác nhau.

Ví dụ, trong xử lý hình ảnh, các lớp đầu tiên có thể học những đặc trưng đơn giản như cạnh hoặc đường nét. Các lớp tiếp theo có thể kết hợp những đặc trưng này để nhận biết các cấu trúc phức tạp hơn. Ở những lớp sâu hơn, mô hình có thể sử dụng các biểu diễn đã học để thực hiện nhiệm vụ phân loại.

Sự phát triển của Deep Learning được thúc đẩy bởi nhiều yếu tố:

- Số lượng dữ liệu ngày càng lớn.
- Năng lực tính toán tăng mạnh.
- Sự phát triển của GPU.
- Các thuật toán tối ưu được cải thiện.
- Các framework hỗ trợ xây dựng mạng nơ-ron ngày càng hoàn thiện.

Nhờ những yếu tố trên, các mô hình học sâu có khả năng đạt kết quả cao trong nhiều bài toán nhận dạng và dự đoán.



Hình 1.6. Sự phát triển từ Machine Learning đến Deep Learning

1.12. Sự phát triển của CNN trong xử lý hình ảnh

Một trong những kiến trúc học sâu quan trọng là Convolutional Neural Network – CNN. CNN được thiết kế đặc biệt để xử lý dữ liệu có cấu trúc không gian, tiêu biểu là hình ảnh.

Một hình ảnh có thể được biểu diễn dưới dạng ma trận các điểm ảnh. Nếu sử dụng mạng nơ-ron truyền thống, việc đưa toàn bộ điểm ảnh vào các lớp kết nối đầy đủ có thể tạo ra số lượng tham số rất lớn. CNN giải quyết vấn đề này bằng cách sử dụng phép tích chập để tìm kiếm các đặc trưng cục bộ.

Các thành phần thường gặp trong CNN gồm:

- Convolution Layer.
- Activation Function.
- Pooling Layer.
- Fully Connected Layer.
- Output Layer.

Trong quá trình phát triển, nhiều kiến trúc CNN khác nhau đã được đề xuất nhằm cải thiện độ chính xác và khả năng huấn luyện. Các kiến trúc sâu hơn có thể học được những đặc trưng phức tạp hơn từ hình ảnh.

CNN đã trở thành một trong những phương pháp quan trọng trong nhận dạng hình ảnh, phân loại ảnh, phát hiện đối tượng và nhiều bài toán thị giác máy tính.

$$X \rightarrow \text{Conv1D} \rightarrow \text{ReLU} \rightarrow \text{Conv1D} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Flatten} \rightarrow \text{Dense} \rightarrow \hat{y}$$

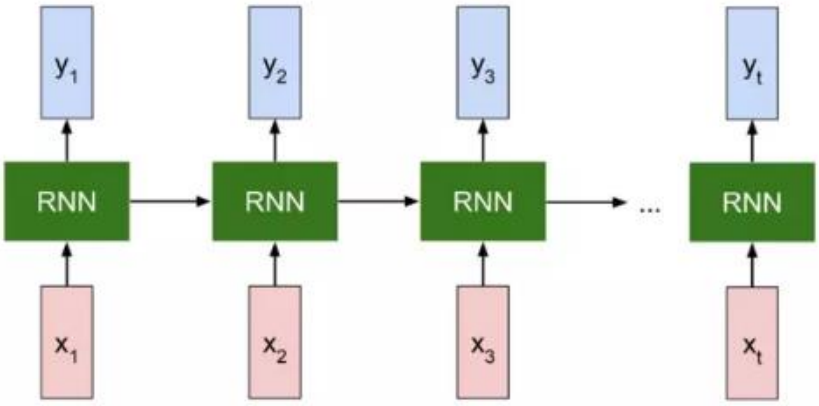
Hình 1.7. Cấu trúc tổng quát của một mạng CNN

Nội dung về CNN sẽ được trình bày chi tiết hơn trong Chương 3.

1.13. Sự phát triển của RNN trong xử lý dữ liệu tuần tự

Không phải tất cả dữ liệu đều có cấu trúc dạng ảnh. Nhiều loại dữ liệu có tính tuần tự, trong đó thứ tự xuất hiện của các phần tử mang ý nghĩa quan trọng. Ví dụ gồm dữ liệu giá cổ phiếu, doanh số theo thời gian, nhiệt độ theo ngày, chuỗi văn bản hoặc tín hiệu.

Mạng nơ-ron hồi quy (Recurrent Neural Network – RNN) được phát triển để xử lý nhóm dữ liệu này. Điểm đặc trưng của RNN là mô hình sử dụng thông tin từ các bước trước đó trong quá trình xử lý bước hiện tại.



Hình 1.8. Minh họa cách RNN xử lý dữ liệu tuần tự

Trong đó, trạng thái ẩn được truyền từ bước trước sang bước tiếp theo, giúp mô hình lưu giữ một phần thông tin của quá trình đã xử lý.

RNN mở ra khả năng xử lý nhiều bài toán mà mạng nơ-ron thông thường khó giải quyết do không xem xét thứ tự của dữ liệu.

1.14. Từ RNN đến LSTM và GRU

Một hạn chế của RNN cơ bản là khó duy trì thông tin trong thời gian dài khi chuỗi dữ liệu có nhiều bước. Trong quá trình lan truyền ngược qua nhiều bước thời gian, gradient có thể trở nên quá nhỏ hoặc quá lớn, gây khó khăn cho quá trình học.

Để giải quyết vấn đề này, các kiến trúc như Long Short-Term Memory – LSTM và Gated Recurrent Unit – GRU được phát triển.

LSTM sử dụng các cơ chế cổng để kiểm soát thông tin được lưu giữ, cập nhật hoặc loại bỏ. Nhờ đó, mạng có khả năng duy trì những thông tin quan trọng trong khoảng thời gian dài hơn.

GRU có cấu trúc đơn giản hơn LSTM nhưng vẫn sử dụng cơ chế cổng để kiểm soát trạng thái. Trong một số bài toán, GRU có thể mang lại hiệu quả tương đương với LSTM trong khi có cấu trúc gọn hơn.

Sự phát triển của RNN, LSTM và GRU cho thấy quá trình tiến hóa của AI không chỉ là việc tăng số lượng lớp mà còn là quá trình thiết kế những cơ chế phù hợp với đặc điểm của từng loại dữ liệu.

1.15. Sự xuất hiện của các kiến trúc AI hiện đại

Khi nhu cầu xử lý dữ liệu ngày càng lớn và phức tạp, các kiến trúc AI tiếp tục phát triển. Một bước tiến đáng chú ý là sự xuất hiện của kiến trúc Transformer, sử dụng cơ chế attention để xác định mức độ quan trọng của các phần tử trong dữ liệu.

Khác với RNN, Transformer không cần xử lý dữ liệu hoàn toàn theo từng bước tuần tự. Cơ chế attention cho phép mô hình xem xét mối quan hệ giữa nhiều phần tử trong chuỗi, từ đó cải thiện khả năng xử lý các quan hệ dài.

Sự phát triển của Transformer đã tạo ra những thay đổi lớn trong xử lý ngôn ngữ tự nhiên và sau đó được mở rộng sang nhiều lĩnh vực khác.

Điều này cho thấy lịch sử AI vẫn đang tiếp tục phát triển. Các mô hình ngày càng có khả năng xử lý dữ liệu đa dạng, kết hợp nhiều loại thông tin và thực hiện các nhiệm vụ phức tạp hơn.

1.16. Vai trò của dữ liệu trong sự phát triển của AI

Nếu các thuật toán là phương pháp giúp máy học thì dữ liệu chính là cơ sở để quá trình học diễn ra. Sự phát triển của AI hiện đại gắn liền với sự gia tăng về cả số lượng và chất lượng dữ liệu.

Dữ liệu tốt cần có độ phù hợp với bài toán, đủ số lượng và được xử lý hợp lý. Dữ liệu chứa nhiều lỗi, giá trị thiếu hoặc phân bố không phù hợp có thể làm giảm chất lượng của mô hình.

Một quy trình AI vì vậy không nên bắt đầu ngay từ việc chọn mô hình. Trước tiên cần hiểu dữ liệu:

- Dữ liệu có bao nhiêu mẫu?
- Có bao nhiêu thuộc tính?
- Kiểu dữ liệu là gì?
- Có giá trị thiếu hay không?
- Các lớp có cân bằng hay không?
- Các biến có cần chuẩn hóa hay không?
- Dữ liệu có chứa nhiễu hoặc ngoại lệ hay không?

Đây là lý do tại sao quá trình phân tích và tiền xử lý dữ liệu thường chiếm một phần quan trọng trong việc xây dựng hệ thống AI.

1.17. Từ mô hình AI đến hệ thống AI hoàn chỉnh

Một trong những thay đổi quan trọng của AI hiện đại là chuyển từ việc chỉ nghiên cứu thuật toán sang xây dựng hệ thống có khả năng hoạt động trong thực tế.

Một mô hình sau khi huấn luyện cần được lưu trữ để có thể sử dụng lại. Khi triển khai, hệ thống cần tiếp nhận dữ liệu đầu vào, thực hiện đúng các bước tiền xử lý đã sử dụng trong quá trình huấn luyện, đưa dữ liệu vào mô hình và trả về kết quả dự đoán.

Quy trình tổng quát có thể mô tả như sau:

Người dùng → Giao diện ứng dụng → Dữ liệu đầu vào → Tiền xử lý → Mô hình AI → Kết quả dự đoán

Một ứng dụng web có thể được sử dụng như lớp giao tiếp giữa người dùng và mô hình. Người dùng nhập thông tin thông qua giao diện, hệ thống thực hiện xử lý và mô hình đưa ra kết quả.

Điều này cho thấy một hệ thống AI thực tế không chỉ bao gồm mô hình học máy hoặc học sâu. Nó còn bao gồm dữ liệu, tiền xử lý, mô hình, giao diện, cơ chế lưu trữ và quy trình triển khai.

1.18. Mối quan hệ giữa các giai đoạn phát triển của AI

Nhìn lại lịch sử phát triển, có thể thấy các phương pháp AI không hoàn toàn thay thế lẫn nhau mà thường bổ sung và kế thừa lẫn nhau.

Giai đoạn đầu tập trung vào biểu diễn tri thức và xây dựng luật. Khi các bài toán trở nên phức tạp hơn, việc xây dựng thủ công toàn bộ tri thức trở nên khó khăn và học máy trở thành một hướng tiếp cận quan trọng.

Sau đó, sự phát triển của mạng nơ-ron và học sâu cho phép mô hình học được các đặc trưng phức tạp từ dữ liệu. CNN trở thành kiến trúc quan trọng cho dữ liệu hình ảnh, RNN và các biến thể phù hợp với dữ liệu tuần tự, trong khi những kiến trúc hiện đại hơn tiếp tục mở rộng khả năng xử lý nhiều dạng dữ liệu.

Có thể khái quát quá trình này:

AI biểu tượng

↓

Hệ chuyên gia

↓

Machine Learning

↓

Neural Network

↓

Deep Learning

↓

CNN / RNN / LSTM / GRU

↓

Các kiến trúc AI hiện đại

Mỗi giai đoạn giải quyết một nhóm hạn chế của giai đoạn trước và đồng thời mở ra những vấn đề nghiên cứu mới.

1.19. Những bài học từ quá trình phát triển AI

Lịch sử phát triển AI cho thấy một số yếu tố có vai trò đặc biệt quan trọng.

Thứ nhất, thuật toán cần phù hợp với dữ liệu. Không có một mô hình duy nhất phù hợp với mọi bài toán. Dữ liệu hình ảnh có những đặc điểm khác với dữ liệu bảng và dữ liệu chuỗi thời gian.

Thứ hai, chất lượng dữ liệu ảnh hưởng trực tiếp đến chất lượng mô hình. Một mô hình phức tạp nhưng được huấn luyện trên dữ liệu kém chất lượng chưa chắc cho kết quả tốt.

Thứ ba, năng lực tính toán có ảnh hưởng lớn đến khả năng phát triển mô hình. Nhiều ý tưởng về mạng nơ-ron đã xuất hiện từ khá sớm nhưng chỉ thực sự phát triển mạnh khi phần cứng và dữ liệu đáp ứng được yêu cầu.

Thứ tư, đánh giá mô hình cần sử dụng các tiêu chí phù hợp. Không nên chỉ dựa vào một chỉ số duy nhất. Đối với phân loại, cần xem xét thêm Precision, Recall và F1-score trong những trường hợp phù hợp. Đối với hồi quy, cần quan tâm đến mức độ sai lệch giữa giá trị dự đoán và giá trị thực tế.

Cuối cùng, một mô hình tốt cần có khả năng được sử dụng trong thực tế. Vì vậy, quá trình triển khai và duy trì mô hình cũng là một phần quan trọng của hệ thống AI.

1.20. Tiểu kết chương

Chương 1 đã trình bày quá trình hình thành và phát triển của trí tuệ nhân tạo từ những ý tưởng ban đầu về khả năng suy luận của máy tính đến các phương pháp AI hiện đại.

Ban đầu, AI chủ yếu dựa trên các luật và tri thức được con người xây dựng. Sự phát triển của hệ chuyên gia cho thấy máy tính có thể mô phỏng một phần khả năng suy luận trong những lĩnh vực cụ thể. Tuy nhiên, hạn chế của phương pháp dựa trên luật đã thúc đẩy sự chuyển dịch sang học máy, trong đó mô hình có khả năng học các quy luật trực tiếp từ dữ liệu.

Sự phát triển tiếp theo của mạng nơ-ron và Deep Learning tạo ra khả năng xử lý những loại dữ liệu phức tạp hơn. CNN trở thành kiến trúc quan trọng đối với dữ liệu hình ảnh, trong khi RNN, LSTM và GRU được phát triển để xử lý dữ liệu tuần tự. Sau đó, những kiến trúc hiện đại như Transformer tiếp tục mở rộng phạm vi ứng dụng của AI.

Qua quá trình phát triển này có thể thấy AI là kết quả của sự kết hợp giữa nhiều yếu tố gồm thuật toán, dữ liệu, phần cứng và phương pháp triển khai. Việc nghiên cứu AI vì vậy cần được tiếp cận một cách tổng thể, từ việc hiểu dữ liệu, lựa chọn phương pháp, xây dựng mô hình cho đến đánh giá và triển khai.

Những kiến thức nền tảng về lịch sử và quá trình phát triển AI trong chương này là cơ sở để tiếp tục nghiên cứu các kỹ thuật học máy cụ thể ở chương tiếp theo, trước khi đi sâu vào các kiến trúc học sâu như CNN và RNN.

CHƯƠNG 2. CÁC KỸ THUẬT MACHINE LEARNING CƠ BẢN

2.1. Tổng quan về Machine Learning

Machine Learning (ML) hay học máy là một nhánh quan trọng của trí tuệ nhân tạo, tập trung vào việc xây dựng các phương pháp cho phép máy tính học được quy luật từ dữ liệu thay vì phải được lập trình cụ thể cho từng trường hợp. Thay vì xây dựng một tập hợp các luật cố định, người phát triển cung cấp cho mô hình dữ liệu cùng với thông tin cần thiết, sau đó mô hình tìm ra mối quan hệ giữa dữ liệu đầu vào và kết quả mong muốn.

Có thể hình dung một bài toán Machine Learning đơn giản gồm ba thành phần chính: dữ liệu đầu vào, mô hình học và kết quả dự đoán. Nếu dữ liệu đầu vào được ký hiệu là X và giá trị cần dự đoán là y, quá trình học nhằm tìm ra một hàm f sao cho:

$$\hat{y} = f(X)$$

Trong đó:

- X là tập đặc trưng đầu vào.
- y là giá trị thực tế hoặc nhãn của dữ liệu.
- ŷ là giá trị mà mô hình dự đoán.
- f là hàm được mô hình học từ dữ liệu.

Điểm quan trọng của Machine Learning là mô hình không chỉ cần ghi nhớ dữ liệu huấn luyện mà phải có khả năng tổng quát hóa cho những dữ liệu chưa từng xuất hiện trong quá trình huấn luyện. Vì vậy, việc đánh giá mô hình trên tập dữ liệu độc lập là một bước bắt buộc.

Machine Learning hiện được sử dụng trong nhiều lĩnh vực như nhận dạng hình ảnh, phát hiện thư rác, dự báo giá, phân tích khách hàng, phát hiện gian lận, dự đoán rủi ro và xử lý ngôn ngữ tự nhiên.

Tùy theo cách dữ liệu được cung cấp và mục tiêu của bài toán, Machine Learning thường được chia thành một số nhóm chính:

- Học có giám sát (Supervised Learning).
- Học không giám sát (Unsupervised Learning).
- Học bán giám sát (Semi-supervised Learning).
- Học tăng cường (Reinforcement Learning).

Trong phạm vi chương này, trọng tâm được đặt vào các kỹ thuật Machine Learning cơ bản thường được sử dụng trong các bài toán phân loại, hồi quy và phân tích dữ liệu.

2.2. Học có giám sát

Học có giám sát là nhóm kỹ thuật Machine Learning trong đó dữ liệu huấn luyện đã có sẵn kết quả cần dự đoán. Mỗi mẫu dữ liệu thường bao gồm một tập đặc trưng X và một giá trị mục tiêu y.

Mục tiêu của quá trình huấn luyện là xây dựng mô hình có khả năng học mối quan hệ: **X → y**

Sau khi được huấn luyện, mô hình có thể nhận dữ liệu X mới và đưa ra giá trị dự đoán ŷ.

Học có giám sát thường được chia thành hai bài toán quan trọng là **classification** và **regression**.

2.2.1. Bài toán phân loại

Phân loại là bài toán trong đó biến mục tiêu thuộc về một hoặc nhiều lớp xác định.

Ví dụ:

- Dự đoán một người có mắc bệnh hay không.
- Phân loại email thành spam và không spam.
- Phân loại khách hàng thành các nhóm.
- Nhận dạng chữ số từ hình ảnh.

Nếu bài toán chỉ có hai lớp, đó là **binary classification**. Nếu có nhiều hơn hai lớp, đó là **multi-class classification**.

Ví dụ với bài toán dự đoán bệnh tiểu đường:

- Lớp 0: Không mắc bệnh.
- Lớp 1: Mắc bệnh.

Mô hình có thể trả về xác suất thuộc lớp 1. Sau đó một ngưỡng được sử dụng để chuyển xác suất thành nhãn dự đoán.

2.2.2. Bài toán hồi quy

Hồi quy được sử dụng khi biến mục tiêu là một giá trị số liên tục.

Ví dụ:

- Dự đoán giá nhà.
- Dự đoán doanh thu.
- Dự đoán nhiệt độ.
- Dự đoán nhu cầu sản phẩm.

Một mô hình hồi quy tuyến tính đơn giản có thể biểu diễn dưới dạng:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Trong đó:

- $\hat{y}$ là giá trị dự đoán.
- $x_1, x_2, ..., x_n$ là các đặc trưng đầu vào.
- β_0 là hệ số chặn.
- $\beta_1, \beta_2, ..., \beta_n$ là các hệ số của mô hình.

Mục tiêu của mô hình là tìm các hệ số sao cho giá trị dự đoán gần với giá trị thực tế nhất.

2.3. Học không giám sát

Khác với học có giám sát, học không giám sát sử dụng dữ liệu không có nhãn mục tiêu rõ ràng. Mô hình phải tự tìm ra cấu trúc, quy luật hoặc nhóm dữ liệu trong tập dữ liệu.

Một trong những kỹ thuật phổ biến nhất là clustering – phân cụm.

2.3.1. Phân cụm K-Means

K-Means chia dữ liệu thành K nhóm dựa trên mức độ tương đồng giữa các điểm dữ liệu.

Quá trình cơ bản gồm:

- Chọn số cụm K.
- Khởi tạo K tâm cụm.
- Gán mỗi điểm dữ liệu vào tâm cụm gần nhất.
- Cập nhật lại tâm của từng cụm.
- Lặp lại quá trình cho đến khi kết quả ổn định.

Khoảng cách Euclid giữa hai điểm có thể được tính theo:

$$d(x, y) = \sqrt{[(x_1 - y_1)^2 + (x_2 - y_2)^2 + ... + (x_n - y_n)^2]}$$

K-Means có thể được sử dụng trong phân nhóm khách hàng, phân tích hành vi người dùng hoặc khám phá cấu trúc dữ liệu.

2.3.2. Giảm chiều dữ liệu

Một kỹ thuật khác của học không giám sát là giảm số chiều dữ liệu. Khi một tập dữ liệu có rất nhiều đặc trưng, việc trực quan hóa hoặc xử lý trực tiếp có thể trở nên khó khăn.

PCA (Principal Component Analysis) là một phương pháp phổ biến nhằm tìm ra các hướng biến thiên chính của dữ liệu và biểu diễn dữ liệu trong không gian có số chiều thấp hơn.

Giảm chiều có thể giúp:

- Giảm số lượng đặc trưng.

- Giảm chi phí tính toán.
- Hỗ trợ trực quan hóa dữ liệu.
- Loại bỏ một phần thông tin dư thừa.

Tuy nhiên, việc giảm chiều cũng có thể làm mất một phần thông tin, vì vậy cần lựa chọn số lượng thành phần phù hợp.

2.4. Biểu diễn dữ liệu trong Machine Learning

Một trong những vấn đề quan trọng nhất của Machine Learning là dữ liệu phải được biểu diễn dưới dạng mà mô hình có thể xử lý**.**

Máy tính không trực tiếp hiểu các khái niệm như "nhà đẹp", "khách hàng hài lòng" hay "bệnh nhân có nguy cơ". Các thông tin đó phải được chuyển thành các giá trị số.

Một observation có thể được biểu diễn dưới dạng vector:

$$\mathbf{x} = [x_1, x_2, ..., x_n]$$

Trong đó mỗi thành phần đại diện cho một đặc trưng.

Ví dụ một đối tượng nhà có thể được biểu diễn bởi: **x = [diện tích, số phòng ngủ, số phòng tắm, tuổi nhà, ...]**

Đối với dữ liệu dạng bảng, mỗi hàng thường tương ứng với một observation và mỗi cột tương ứng với một feature.

Đối với dữ liệu văn bản, thông tin cần được chuyển thành dạng vector số thông qua các kỹ thuật như Bag-of-Words hoặc TF-IDF.

2.4.1. Biến số và biến phân loại

Feature trong dữ liệu có thể thuộc nhiều dạng khác nhau.

Dữ liệu số có thể được sử dụng trực tiếp hoặc cần chuẩn hóa.

Ví dụ:

- Tuổi.
- Thu nhập.
- Diện tích.
- Số lượng sản phẩm.

Dữ liệu phân loại thường phải được chuyển sang dạng số.

Ví dụ:

- Giới tính.

- Loại nhà.
- Khu vực.
- Loại sản phẩm.

Một phương pháp phổ biến là One-Hot Encoding.

Ví dụ một feature “Loại nhà” có ba giá trị:

- Apartment
- House
- Villa

có thể được chuyển thành ba cột nhị phân tương ứng.

2.4.2. Biểu diễn văn bản bằng TF-IDF

Đối với dữ liệu văn bản, một phương pháp cơ bản nhưng rất hiệu quả là TF-IDF.

TF-IDF đánh giá mức độ quan trọng của một từ trong một văn bản so với toàn bộ tập văn bản.

TF có thể hiểu là mức độ xuất hiện của từ trong một tài liệu:

TF(t, d) = số lần từ t xuất hiện trong tài liệu d / tổng số từ trong tài liệu d

IDF thể hiện mức độ hiếm của từ:

$$\mathbf{IDF(t) = \log(N / DF(t))}$$

Trong đó:

- N là tổng số tài liệu.
- DF(t) là số tài liệu chứa từ t.

TF-IDF được tính bằng:

$$\mathbf{TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)}$$

Một từ xuất hiện nhiều trong một tài liệu nhưng không xuất hiện phổ biến trong các tài liệu khác sẽ có trọng số TF-IDF cao hơn.

Trong thực tế, TF-IDF thường được sử dụng cho các bài toán phân loại văn bản, tìm kiếm văn bản và phân tích đánh giá của khách hàng.

2.5. Tiền xử lý dữ liệu

Dữ liệu thực tế thường chưa thể đưa trực tiếp vào mô hình. Có thể tồn tại giá trị thiếu, dữ liệu trùng lặp, dữ liệu sai định dạng hoặc các đặc trưng có thang đo rất khác nhau.

Do đó, tiền xử lý dữ liệu là một bước quan trọng của pipeline Machine Learning.

2.5.1. Xử lý dữ liệu thiếu

Missing value có thể xuất hiện do quá trình thu thập dữ liệu hoặc do người dùng không cung cấp thông tin.

Một số phương pháp xử lý phổ biến:

- Xóa các dòng chứa quá nhiều giá trị thiếu.
- Thay thế bằng mean.
- Thay thế bằng median.
- Thay thế bằng mode.
- Sử dụng một mô hình khác để ước lượng giá trị thiếu.

Việc lựa chọn phương pháp phụ thuộc vào đặc điểm của dữ liệu.

2.5.2. Phát hiện dữ liệu bất thường

Outlier là các điểm dữ liệu có giá trị khác biệt đáng kể so với phần còn lại.

Một phương pháp phổ biến là IQR:

$$\mathbf{IQR = Q_3 - Q_1}$$

Trong đó Q_1 là tứ phân vị thứ nhất và Q_3 là tứ phân vị thứ ba.

Có thể xác định khoảng giá trị thông thường:

- $\mathbf{Lower\ Bound = Q_1 - 1.5 \times IQR}$
- $\mathbf{Upper\ Bound = Q_3 + 1.5 \times IQR}$

Tuy nhiên, không phải mọi outlier đều là dữ liệu sai. Một số trường hợp có thể phản ánh đúng hiện tượng thực tế. Vì vậy, việc loại bỏ outlier cần dựa trên ý nghĩa của bài toán.

2.5.3. Chuẩn hóa dữ liệu

Các thuật toán như KNN, SVM hoặc một số mô hình dựa trên khoảng cách thường nhạy cảm với thang đo của feature.

Standardization đưa dữ liệu về dạng có trung bình gần 0 và độ lệch chuẩn gần 1:

$$\mathbf{z = (x - \mu) / \sigma}$$

Trong đó:

- x là giá trị ban đầu.
- μ là giá trị trung bình.
- σ là độ lệch chuẩn.
- z là giá trị sau chuẩn hóa.

Một phương pháp khác là Min-Max Scaling:

$$x' = (x - x_min) / (x_max - x_min)$$

Kết quả thường nằm trong khoảng từ 0 đến 1.

2.6. Kỹ thuật hồi quy tuyến tính

Linear Regression là một trong những thuật toán Machine Learning cơ bản nhất.

Với một feature, mô hình có dạng:

$$\hat{y} = \beta_0 + \beta_1 x$$

Với nhiều feature:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Mục tiêu là tìm các hệ số β để tổng sai số giữa giá trị thực tế và giá trị dự đoán là nhỏ nhất.

Linear Regression có ưu điểm:

- Dễ hiểu.
- Dễ triển khai.
- Tốc độ huấn luyện nhanh.
- Có khả năng giải thích tương đối rõ vai trò của các feature.

Tuy nhiên, mô hình giả định mối quan hệ gần tuyến tính giữa feature và target. Khi dữ liệu có quan hệ phức tạp, Linear Regression có thể không đạt kết quả tốt.

2.7. Ridge và Lasso Regression

Khi số lượng feature lớn hoặc các feature có tương quan cao, hồi quy tuyến tính có thể gặp vấn đề về overfitting.

Ridge Regression bổ sung thành phần phạt dựa trên bình phương hệ số:

$$Loss = MSE + \alpha \times \Sigma \beta_i^2$$

Trong đó α kiểm soát mức độ regularization.

Ridge có xu hướng làm giảm độ lớn của các hệ số nhưng thường không đưa chúng hoàn toàn về 0.

Lasso Regression sử dụng tổng trị tuyệt đối:

$$Loss = MSE + \alpha \times \Sigma |\beta_i|$$

Do đó, Lasso có thể đưa một số hệ số về đúng 0 và có khả năng thực hiện lựa chọn feature.

Hai phương pháp này là những kỹ thuật quan trọng khi cần hạn chế overfitting trong mô hình hồi quy.

2.8. Logistic Regression

Mặc dù có tên là Regression, Logistic Regression thường được sử dụng cho bài toán phân loại.

Mô hình trước hết tính một giá trị tuyến tính:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Sau đó sử dụng hàm sigmoid:

$$\sigma(z) = 1 / (1 + e^{-z})$$

Giá trị sigmoid nằm trong khoảng từ 0 đến 1 và có thể được hiểu là xác suất thuộc lớp dương.

Ví dụ:

- $\sigma(z) = 0.91$: mô hình có độ tin tưởng cao rằng mẫu thuộc lớp 1.
- $\sigma(z) = 0.23$: mô hình có xu hướng dự đoán lớp 0.

Logistic Regression có ưu điểm là đơn giản, tốc độ nhanh và dễ giải thích. Đây là một mô hình baseline quan trọng cho nhiều bài toán classification.

2.9. K-Nearest Neighbors

K-Nearest Neighbors (KNN) là thuật toán dự đoán dựa trên các điểm dữ liệu gần nhất.

Khi cần dự đoán một mẫu mới, thuật toán:

- Tính khoảng cách từ mẫu mới đến các mẫu trong tập huấn luyện.
- Chọn K mẫu gần nhất.
- Với classification, lựa chọn lớp xuất hiện nhiều nhất.
- Với regression, có thể lấy trung bình giá trị của các điểm gần nhất.

KNN có ưu điểm là đơn giản và không yêu cầu xây dựng một mô hình tham số phức tạp.

Tuy nhiên, KNN có một số hạn chế:

- Chi phí dự đoán tăng khi dữ liệu lớn.
- Nhạy cảm với thang đo feature.
- Việc lựa chọn K ảnh hưởng trực tiếp đến kết quả.
- Hiệu quả giảm khi số chiều dữ liệu quá lớn.

2.10. Decision Tree

Decision Tree biểu diễn quá trình dự đoán dưới dạng một cây quyết định.

Mỗi nút trong cây thực hiện một điều kiện trên feature. Quá trình phân chia tiếp tục cho đến khi đạt được các nút lá chứa kết quả dự đoán.

Ví dụ trong bài toán đánh giá rủi ro, cây có thể thực hiện: **Tuổi > 50? → Có → Kiểm tra BMI → ...**

Decision Tree có ưu điểm:

- Dễ trực quan hóa.
- Có thể xử lý cả feature số và feature phân loại.
- Không yêu cầu chuẩn hóa dữ liệu trong nhiều trường hợp.
- Có khả năng mô hình hóa quan hệ phi tuyến.

Tuy nhiên, cây quá sâu có thể học quá sát dữ liệu huấn luyện và gây overfitting.

Các tham số như max_depth, min_samples_split và min_samples_leaf thường được sử dụng để kiểm soát độ phức tạp của cây.

2.11. Random Forest

Random Forest là phương pháp ensemble kết hợp nhiều Decision Tree.

Thay vì chỉ sử dụng một cây quyết định, Random Forest xây dựng nhiều cây khác nhau rồi kết hợp kết quả. Đối với classification, kết quả thường được xác định dựa trên biểu quyết của các cây. Đối với regression, kết quả có thể được tính bằng trung bình dự đoán của các cây.

Random Forest có một số ưu điểm:

- Có khả năng mô hình hóa quan hệ phi tuyến.
- Ít nhạy với overfitting hơn một Decision Tree đơn.
- Có thể xử lý dữ liệu có nhiều feature.
- Có thể cung cấp thông tin về mức độ quan trọng của feature.

Nhược điểm là mô hình có thể lớn hơn và khó giải thích trực tiếp như một Decision Tree đơn.

Random Forest phù hợp với nhiều bài toán dữ liệu dạng bảng và thường được sử dụng như một mô hình baseline mạnh.

2.12. Support Vector Machine

Support Vector Machine (SVM) tìm một siêu phẳng có khả năng phân tách các lớp dữ liệu.

Trong trường hợp dữ liệu có thể phân chia tuyến tính, mô hình tìm đường biên sao cho khoảng cách từ đường biên đến các điểm gần nhất của hai lớp là lớn nhất.

Các điểm gần đường biên được gọi là support vectors.

SVM có thể mở rộng cho dữ liệu không tuyến tính thông qua kernel. Một số kernel phổ biến gồm:

- Linear.
- Polynomial.
- Radial Basis Function (RBF).

SVM thường hoạt động tốt với dữ liệu có số chiều tương đối cao, đặc biệt khi kết hợp với biểu diễn TF-IDF trong bài toán văn bản.

Một hạn chế là chi phí tính toán có thể tăng khi số lượng mẫu rất lớn.

2.13. Naive Bayes

Naive Bayes là nhóm thuật toán phân loại dựa trên định lý Bayes.

Định lý Bayes có thể biểu diễn:

$$P(A|B) = P(B|A)P(A) / P(B)$$

Trong bài toán phân loại, mô hình ước lượng xác suất một mẫu thuộc từng lớp dựa trên các feature.

Tên gọi "Naive" xuất phát từ giả định đơn giản rằng các feature độc lập có điều kiện với nhau khi biết lớp. Giả định này không phải lúc nào cũng đúng trong thực tế, nhưng Naive Bayes vẫn có hiệu quả tốt trong một số bài toán, đặc biệt là phân loại văn bản.

Ưu điểm của Naive Bayes là:

- Đơn giản.
- Huấn luyện nhanh.
- Hoạt động tốt với dữ liệu có số chiều lớn.
- Thường phù hợp với các bài toán text classification.

2.14. Đánh giá mô hình Machine Learning

Sau khi huấn luyện, mô hình cần được đánh giá bằng dữ liệu mà mô hình chưa sử dụng trong quá trình học.

Thông thường dữ liệu được chia thành:

- Training set: dùng để huấn luyện.
- Validation set: dùng để lựa chọn hoặc điều chỉnh mô hình.
- Test set: dùng để đánh giá cuối cùng.

Với các bài toán đơn giản, có thể sử dụng train/test split. Với các bài toán cần đánh giá ổn định hơn, có thể sử dụng cross-validation.

2.14.1. Accuracy

Accuracy là tỷ lệ dự đoán đúng trên tổng số mẫu:

Accuracy = số dự đoán đúng / tổng số mẫu

Accuracy dễ hiểu nhưng không phải lúc nào cũng phù hợp với dữ liệu mất cân bằng.

2.14.2. Precision

Precision cho biết trong số các mẫu được dự đoán là dương tính, có bao nhiêu mẫu thực sự dương tính:

Precision = TP / (TP + FP)

2.14.3. Recall

Recall cho biết trong số các mẫu thực sự dương tính, mô hình phát hiện được bao nhiêu:

Recall = TP / (TP + FN)

Recall đặc biệt quan trọng trong các bài toán mà việc bỏ sót một trường hợp dương tính có hậu quả lớn.

2.14.4. F1-score

F1-score là trung bình điều hòa giữa Precision và Recall:

F1 = 2 × Precision × Recall / (Precision + Recall)

F1-score thường được sử dụng khi cần cân bằng giữa Precision và Recall.

2.14.5. Confusion Matrix

Confusion Matrix mô tả số lượng dự đoán đúng và sai của mô hình theo từng lớp.

Bốn thành phần cơ bản là:

- TP: True Positive.
- TN: True Negative.
- FP: False Positive.
- FN: False Negative.

Confusion Matrix giúp phát hiện trường hợp mô hình có Accuracy cao nhưng hoạt động kém trên một lớp cụ thể.

2.14.6. MAE, MSE và RMSE

Đối với regression, một số metric phổ biến là MAE, MSE và RMSE.

MAE: $\mathbf{MAE = (1/n) \times \Sigma|y_i - \hat{y}_i|}$

MSE: $\mathbf{MSE = (1/n) \times \Sigma(y_i - \hat{y}_i)^2}$

RMSE: $\mathbf{RMSE = \sqrt{MSE}}$

MAE dễ diễn giải vì giữ nguyên đơn vị của target. MSE phạt mạnh các sai số lớn. RMSE cũng giữ cùng đơn vị với target nhưng nhạy hơn với các sai số lớn.

2.14.7. R^2

R^2 đo mức độ mô hình giải thích được biến thiên của biến mục tiêu:

$$\mathbf{R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]}$$

Trong đó $\bar{y}$ là giá trị trung bình của target.

R^2 càng gần 1 thì mô hình thường càng giải thích tốt dữ liệu trong phạm vi đánh giá. Tuy nhiên, R^2 không nên được sử dụng riêng lẻ mà cần xem xét cùng các metric khác.

2.15. Overfitting, Underfitting và khả năng tổng quát hóa

Một vấn đề quan trọng khi xây dựng Machine Learning là sự cân bằng giữa khả năng học dữ liệu và khả năng tổng quát hóa.

Underfitting xảy ra khi mô hình quá đơn giản và không thể mô tả đầy đủ mối quan hệ trong dữ liệu. Khi đó, kết quả trên cả training set và test set đều có thể thấp.

Overfitting xảy ra khi mô hình học quá sát dữ liệu huấn luyện, bao gồm cả nhiễu và các đặc điểm riêng của training set. Khi đó:

- Training performance rất cao.
- Test performance thấp hơn đáng kể.

Một số phương pháp hạn chế overfitting:

- Giảm độ phức tạp mô hình.
- Regularization.
- Sử dụng nhiều dữ liệu hơn.
- Feature selection.
- Cross-validation.
- Điều chỉnh hyperparameter.
- Early stopping đối với một số mô hình.

Khả năng tổng quát hóa là yếu tố quan trọng để đánh giá một mô hình Machine Learning có thực sự hữu ích hay không.

2.16. Quy trình xây dựng một hệ thống Machine Learning

Từ các kỹ thuật trên, một quy trình Machine Learning có thể được tổ chức thành các bước:

Xác định bài toán → Thu thập dữ liệu → Khám phá dữ liệu → Tiền xử lý → Biểu diễn dữ liệu → Chia dữ liệu → Huấn luyện → Đánh giá → Lựa chọn mô hình → Lưu mô hình → Triển khai

Trong đó, việc lựa chọn thuật toán chỉ là một phần của toàn bộ quy trình.

Chất lượng của hệ thống phụ thuộc đồng thời vào:

- Chất lượng dữ liệu.
- Cách biểu diễn dữ liệu.
- Cách tiền xử lý.
- Lựa chọn feature.
- Thuật toán.
- Hyperparameter.
- Metric đánh giá.
- Khả năng tổng quát hóa.
- Cách triển khai mô hình.

Điều này cho thấy một mô hình có thuật toán phức tạp chưa chắc tốt hơn một mô hình đơn giản nếu dữ liệu và representation không phù hợp.

2.17. Minh họa thực nghiệm với các bài toán Machine Learning

Các kỹ thuật được trình bày ở trên được minh họa thông qua ba bài toán có đặc điểm khác nhau: phân loại dữ liệu y tế, hồi quy dữ liệu giá nhà và phân loại kết hợp dữ liệu dạng bảng với văn bản.

Mục đích của phần này không phải giới thiệu lại toàn bộ quá trình xây dựng từng ứng dụng mà nhằm cho thấy các kỹ thuật Machine Learning cơ bản được áp dụng như thế nào trong những tình huống thực tế.

2.17.1. Bài toán dự đoán nguy cơ bệnh tiểu đường

Bài toán đầu tiên là phân loại nhị phân. Dữ liệu chứa các thuộc tính liên quan đến tình trạng sức khỏe và biến mục tiêu thể hiện trạng thái chẩn đoán bệnh.

Các feature được biểu diễn dưới dạng vector số để đưa vào mô hình Machine Learning.

Một số bước xử lý quan trọng gồm:

Kiểm tra cấu trúc dữ liệu.

Kiểm tra giá trị thiếu.

Kiểm tra dữ liệu trùng lặp.

Phân tích phân bố target.

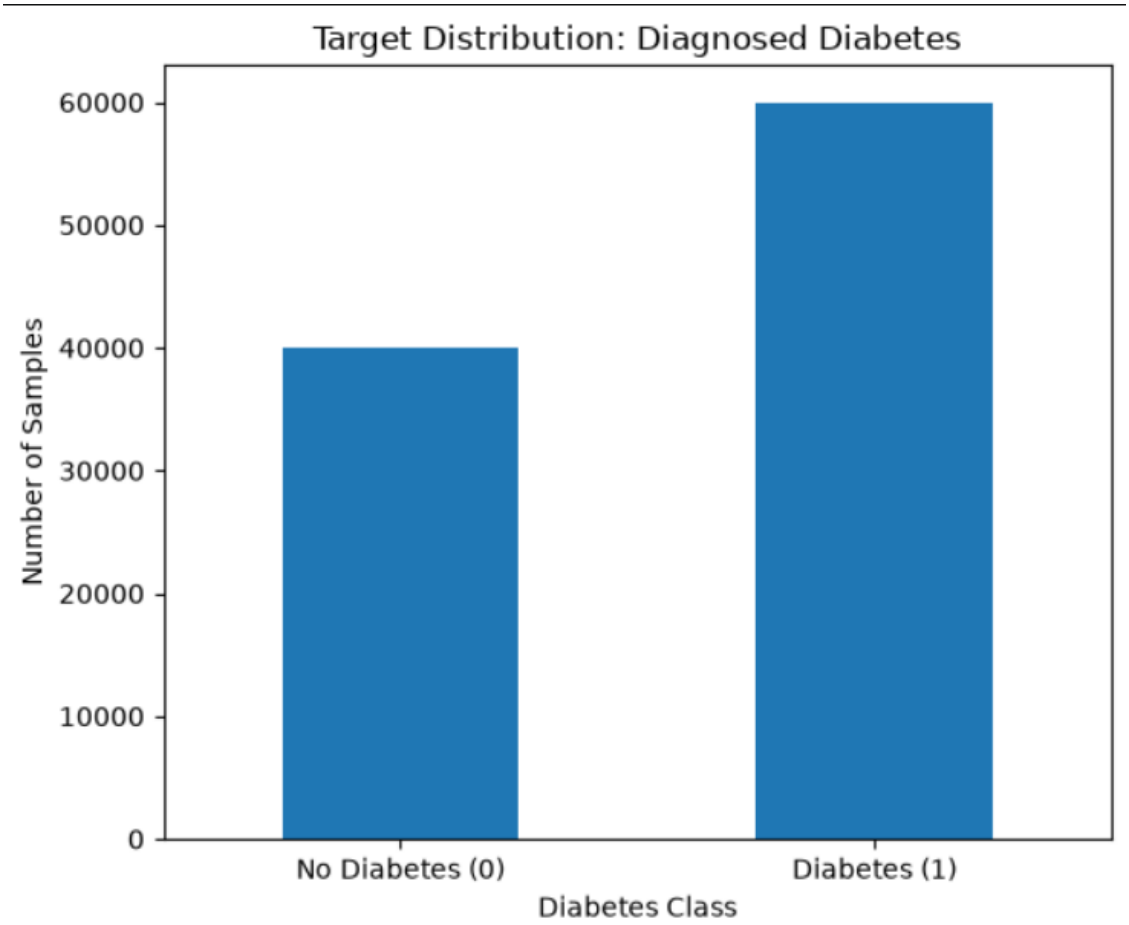
Kiểm tra các feature có khả năng gây leakage.

Chia dữ liệu thành tập huấn luyện và kiểm tra.

Huấn luyện các mô hình phân loại.

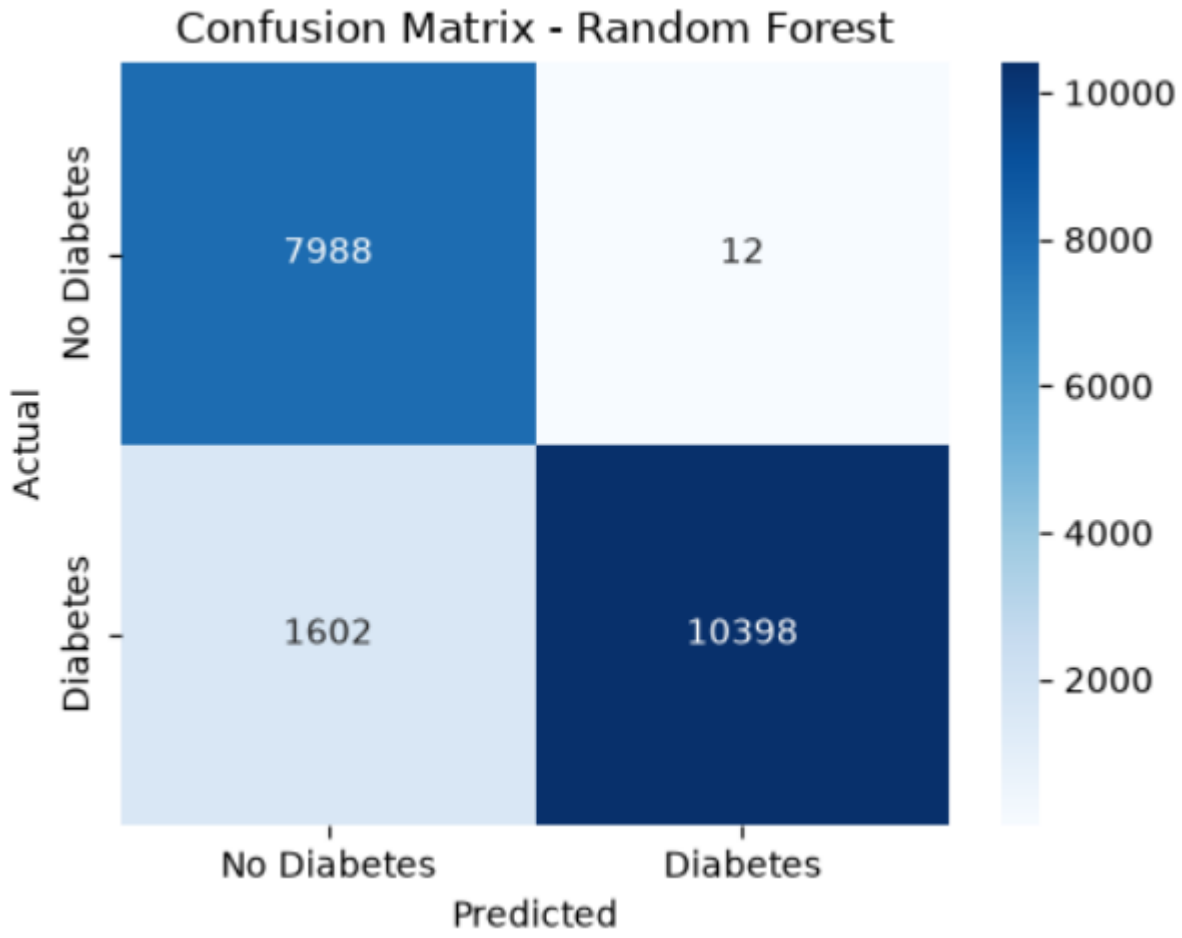
Đánh giá bằng Accuracy, Precision, Recall, F1-score và ROC-AUC.

Trong thực nghiệm, Random Forest cho kết quả tốt với Accuracy khoảng 0,9193, F1-score khoảng 0,9280 và ROC-AUC khoảng 0,9406. Kết quả này cho thấy mô hình có khả năng phân biệt tương đối tốt giữa các nhóm dữ liệu.



Hình 2.1 – Phân bố biến mục tiêu của bộ dữ liệu Diabetes

Nhận xét: Biểu đồ cho thấy sự phân bố của hai lớp trong dữ liệu, từ đó giúp đánh giá mức độ cân bằng của bài toán trước khi lựa chọn metric và huấn luyện mô hình.



Hình 2.2 – Confusion Matrix của mô hình phân loại Diabetes

Nhận xét: Confusion Matrix cho phép quan sát trực tiếp số lượng mẫu được dự đoán đúng và sai ở từng lớp, qua đó bổ sung thông tin cho Accuracy và F1-score.

2.17.2. Bài toán dự đoán giá nhà

Bài toán thứ hai thuộc nhóm regression. Thay vì dự đoán một nhãn rời rạc, mô hình cần dự đoán một giá trị liên tục là giá nhà.

Dữ liệu đầu vào gồm các đặc trưng mô tả căn nhà. Các đặc trưng này được chuyển thành ma trận số trước khi đưa vào mô hình.

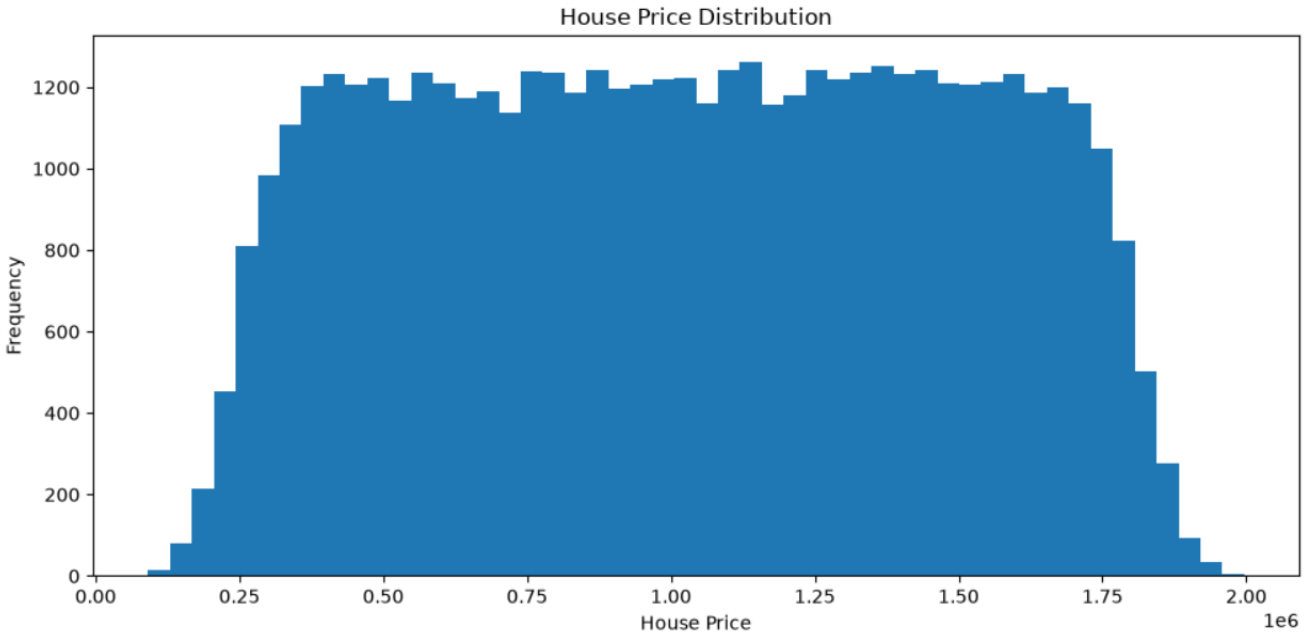
Các mô hình hồi quy có thể được sử dụng gồm:

- Linear Regression.
- Ridge Regression.

- Lasso Regression.
- Decision Tree Regression.
- Random Forest Regression.
- Gradient Boosting Regression.

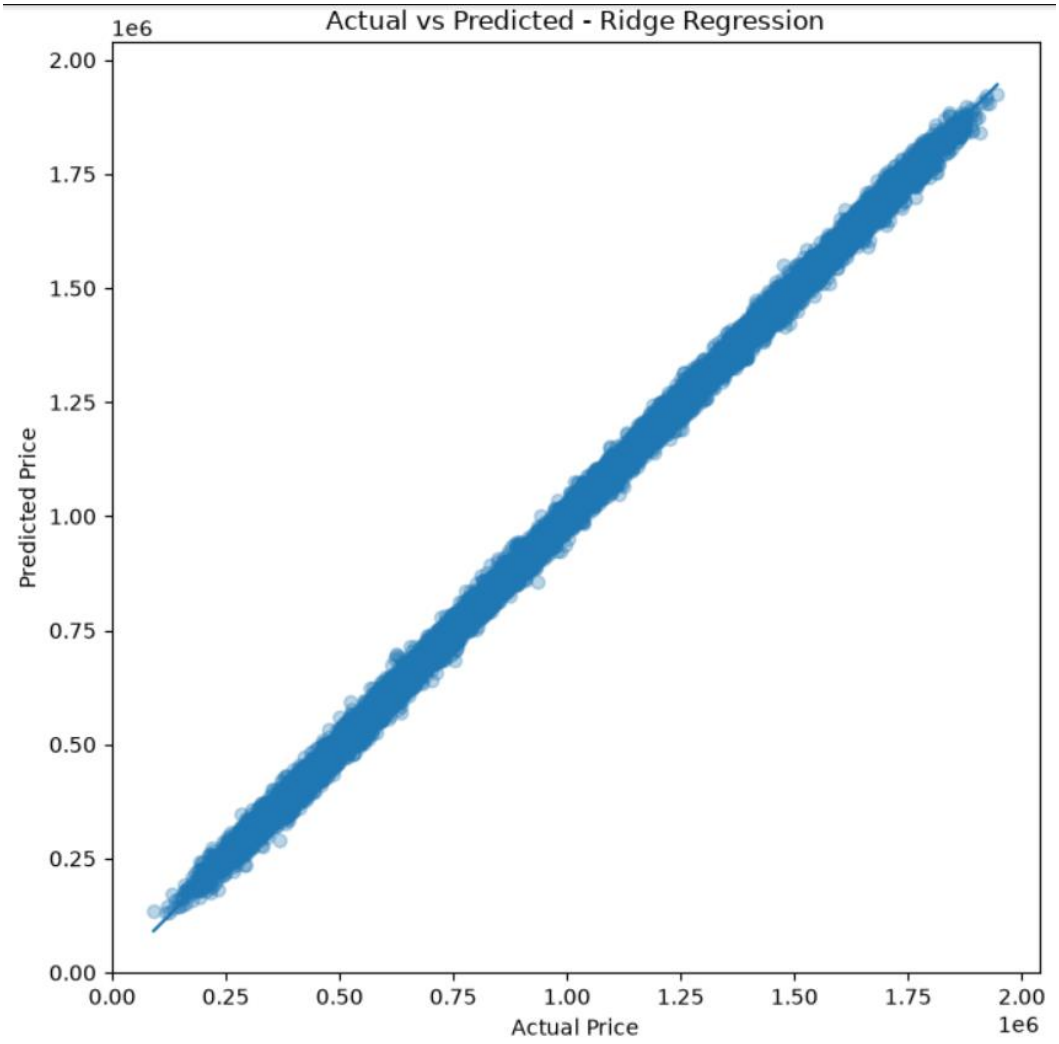
Trong thực nghiệm, Ridge Regression đạt kết quả tốt với MAE khoảng 15.954,11, RMSE khoảng 19.940,75 và R^2 khoảng 0,9981.

Kết quả cho thấy những mô hình tuyến tính được regularization có thể hoạt động rất tốt trên bộ dữ liệu này. Tuy nhiên, không nên suy rộng kết quả này thành kết luận rằng Ridge luôn tốt hơn các thuật toán khác. Hiệu quả phụ thuộc vào đặc điểm của từng dataset.



Hình 2.3 – Phân bố giá nhà trong tập dữ liệu

Nhận xét: Biểu đồ giúp quan sát phạm vi giá và hình dạng phân bố của biến mục tiêu, từ đó hỗ trợ lựa chọn phương pháp tiền xử lý và đánh giá mô hình.



Hình 2.4 – So sánh giá thực tế và giá dự đoán

Nhận xét: Các điểm càng nằm gần đường chéo thì giá trị dự đoán càng gần giá trị thực tế. Đây là cách trực quan để quan sát chất lượng của mô hình hồi quy.

2.17.3. Bài toán phân tích hành vi khách hàng từ dữ liệu văn bản

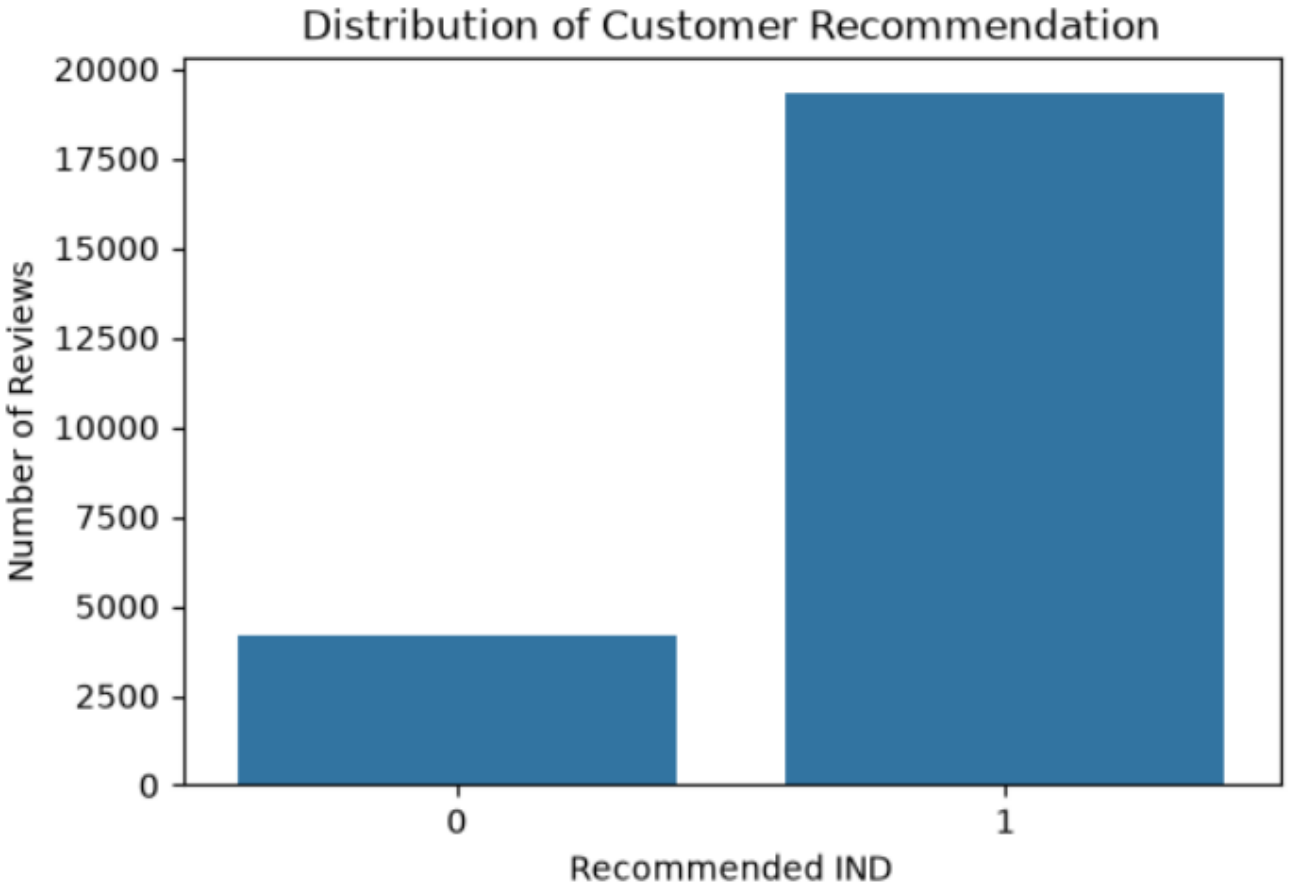
Bài toán thứ ba minh họa trường hợp Machine Learning không chỉ sử dụng dữ liệu dạng bảng mà còn xử lý dữ liệu văn bản.

Dữ liệu đánh giá sản phẩm chứa các trường thông tin dạng bảng kết hợp với nội dung review của khách hàng. Văn bản được chuyển thành vector số bằng TF-IDF trước khi đưa vào mô hình.

Quá trình này minh họa rõ vai trò của **data representation**. Nếu chỉ sử dụng các feature dạng bảng, mô hình có thể bỏ qua phần lớn thông tin được thể hiện trong nội dung đánh giá. Khi bổ sung representation của review bằng TF-IDF, mô hình có thêm thông tin về từ ngữ và nội dung của khách hàng.

Trong thực nghiệm, Text Linear SVM đạt F1-score khoảng 0,9386 và ROC-AUC khoảng 0,9440.

Kết quả cho thấy một thuật toán đơn giản kết hợp với representation phù hợp vẫn có thể đạt hiệu quả cao.



Hình 2.5 – Phân bố nhãn của các review

Nhận xét: Biểu đồ giúp quan sát đặc điểm của dữ liệu văn bản trước khi thực hiện vector hóa.

2.18. So sánh kết quả các bài toán thực nghiệm

Ba bài toán trên đại diện cho ba tình huống Machine Learning khác nhau:

Bài toán	Loại bài toán	Dạng dữ liệu	Mô hình nổi bật	Metric tiêu biểu
Diabetes	Classification	Dữ liệu dạng bảng	Random Forest	F1 = 0,9280
House Price	Regression	Dữ liệu dạng bảng	Ridge Regression	R^2 = 0,9981

Customer Behavior	Classification	Bảng + văn bản	Linear SVM	F1 = 0,9386

Bảng trên cho thấy không có một thuật toán duy nhất phù hợp với mọi loại dữ liệu.

Với dữ liệu dạng bảng và bài toán classification, các phương pháp như Logistic Regression, Decision Tree, Random Forest, KNN hoặc SVM có thể được thử nghiệm.

Với regression, Linear Regression, Ridge, Lasso, Decision Tree Regression, Random Forest Regression và Gradient Boosting là những lựa chọn phổ biến.

Với văn bản, representation đóng vai trò rất lớn. Các phương pháp như TF-IDF kết hợp Linear SVM hoặc Naive Bayes là những baseline quan trọng trước khi chuyển sang các phương pháp Deep Learning phức tạp hơn.

Do đó, việc xây dựng Machine Learning nên bắt đầu từ **đặc điểm của dữ liệu và bản chất bài toán**, sau đó mới lựa chọn thuật toán.

2.19. Cài đặt Machine Learning bằng Scratch, Keras và PyTorch

Một mục tiêu quan trọng khi học Machine Learning là không chỉ biết gọi thư viện mà còn hiểu được các thành phần cơ bản của quá trình học.

2.19.1. Cài đặt theo hướng Scratch

Cách tiếp cận Scratch nhằm minh họa các phép tính và quá trình cập nhật tham số ở mức cơ bản.

Ví dụ với một mô hình tuyến tính:

$$\hat{y} = wx + b$$

Sai số giữa giá trị dự đoán và giá trị thực tế được sử dụng để xây dựng hàm mất mát. Sau đó các tham số w và b được cập nhật thông qua gradient descent.

Cập nhật tham số có dạng:

$$w \leftarrow w - \eta \times \partial L/\partial w$$

$$b \leftarrow b - \eta \times \partial L/\partial b$$

Trong đó:

- η là learning rate.
- L là loss function.
- $\partial L/\partial w$ và $\partial L/\partial b$ là gradient.

Cách cài đặt này giúp hiểu bản chất của quá trình học nhưng không thuận tiện khi xây dựng hệ thống lớn.

Code Scratch minh họa một thuật toán Machine Learning:

```
# Implement binary Logistic Regression from scratch using NumPy.

import numpy as np

class ScratchLogisticRegression:

    def __init__(
        self,
        learning_rate=0.01,
        epochs=1000
    ):

        self.learning_rate = learning_rate
        self.epochs = epochs

        self.weights = None
        self.bias = None

        self.loss_history = []


    def sigmoid(self, z):

        z = np.clip(
            z,
            -500,
            500
        )

        return 1 / (
            1 + np.exp(-z)
        )


    def fit(self, X, y):

        n_samples, n_features = X.shape

        self.weights = np.zeros(
            n_features
        )
```

```
        self.bias = 0.0

        y = np.asarray(y)


        for epoch in range(
            self.epochs
        ):

            linear_output = (
                np.dot(
                    X,
                    self.weights
                )
                + self.bias
            )

            predictions = self.sigmoid(
                linear_output
            )


            epsilon = 1e-15

            predictions = np.clip(
                predictions,
                epsilon,
                1 - epsilon
            )


            loss = -np.mean(
                y * np.log(predictions)
                +
                (1 - y)
                * np.log(1 - predictions)
            )

            self.loss_history.append(
                loss
            )


            dw = np.dot(
                X.T,
                predictions - y
            ) / n_samples
```

```
            db = np.mean(
                predictions - y
            )


            self.weights -= (
                self.learning_rate
                * dw
            )

            self.bias -= (
                self.learning_rate
                * db
            )

        return self


    def predict_proba(self, X):

        linear_output = (
            np.dot(
                X,
                self.weights
            )
            + self.bias
        )

        return self.sigmoid(
            linear_output
        )


    def predict(self, X):

        probabilities = (
            self.predict_proba(X)
        )

        return (
            probabilities >= 0.5
        ).astype(int)
```

2.19.2. Cài đặt bằng Keras

Keras cung cấp giao diện cấp cao giúp xây dựng mô hình nhanh hơn.

Một quy trình cơ bản gồm:

- Chuẩn bị dữ liệu.
- Xây dựng model.
- Compile model.
- Huấn luyện.
- Đánh giá.
- Dự đoán.

Ưu điểm của Keras là cú pháp ngắn gọn và phù hợp cho việc thử nghiệm nhiều mô hình.

Code xây dựng và huấn luyện mô hình bằng Keras:

```
# Build and train Logistic Regression using Keras.

import tensorflow as tf

keras_model = tf.keras.Sequential([
    tf.keras.layers.Input(
        shape=(X_train_processed.shape[1],)
    ),

    tf.keras.layers.Dense(
        1,
        activation="sigmoid"
    )
])

keras_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

keras_history = keras_model.fit(
    X_train_processed,
    y_train.values,
    validation_split=0.1,
    epochs=30,
    batch_size=64,
    verbose=1
)
```

2.19.3. Cài đặt bằng PyTorch

PyTorch cung cấp khả năng kiểm soát chi tiết hơn quá trình xây dựng và huấn luyện mô hình.

Một quy trình thông thường gồm:

Dataset → DataLoader → Model → Loss → Optimizer → Training Loop → Evaluation

PyTorch đặc biệt hữu ích khi cần tự xây dựng training loop hoặc nghiên cứu các kiến trúc mô hình phức tạp.

Code xây dựng và train mô hình bằng PyTorch:

```
# Build and train Logistic Regression using PyTorch.

import torch
import torch.nn as nn

torch_X_train = torch.tensor(
    X_train_processed,
    dtype=torch.float32
)

torch_y_train = torch.tensor(
    y_train.values,
    dtype=torch.float32
).reshape(-1, 1)

torch_X_test = torch.tensor(
    X_test_processed,
    dtype=torch.float32
)

torch_y_test = torch.tensor(
    y_test.values,
    dtype=torch.float32
).reshape(-1, 1)


class TorchLogisticRegression(
    nn.Module
):

    def __init__(
        self,
        input_size
    ):

        super().__init__()
```

```
        self.linear = nn.Linear(
            input_size,
            1
        )


    def forward(self, x):

        return self.linear(x)


torch_model = TorchLogisticRegression(
    X_train_processed.shape[1]
)

criterion = nn.BCEWithLogitsLoss()

optimizer = torch.optim.Adam(
    torch_model.parameters(),
    lr=0.001
)
```

Việc sử dụng ba cách tiếp cận giúp làm rõ sự khác biệt giữa việc hiểu thuật toán ở mức nền tảng và sử dụng framework để phát triển hệ thống thực tế.

2.20. Lưu mô hình và triển khai ứng dụng

Sau khi huấn luyện và đánh giá, mô hình có thể được lưu lại để sử dụng cho dữ liệu mới mà không cần huấn luyện lại.

Đây là bước quan trọng để chuyển từ một thí nghiệm Machine Learning sang một ứng dụng có khả năng sử dụng thực tế.

Không chỉ model cần được lưu, các bước preprocessing cũng cần được duy trì nhất quán. Nếu dữ liệu lúc inference được xử lý khác với lúc training, kết quả dự đoán có thể bị sai lệch.

Một pipeline triển khai có thể được hiểu đơn giản là:

Dữ liệu đầu vào → Preprocessing → Model → Prediction → Kết quả

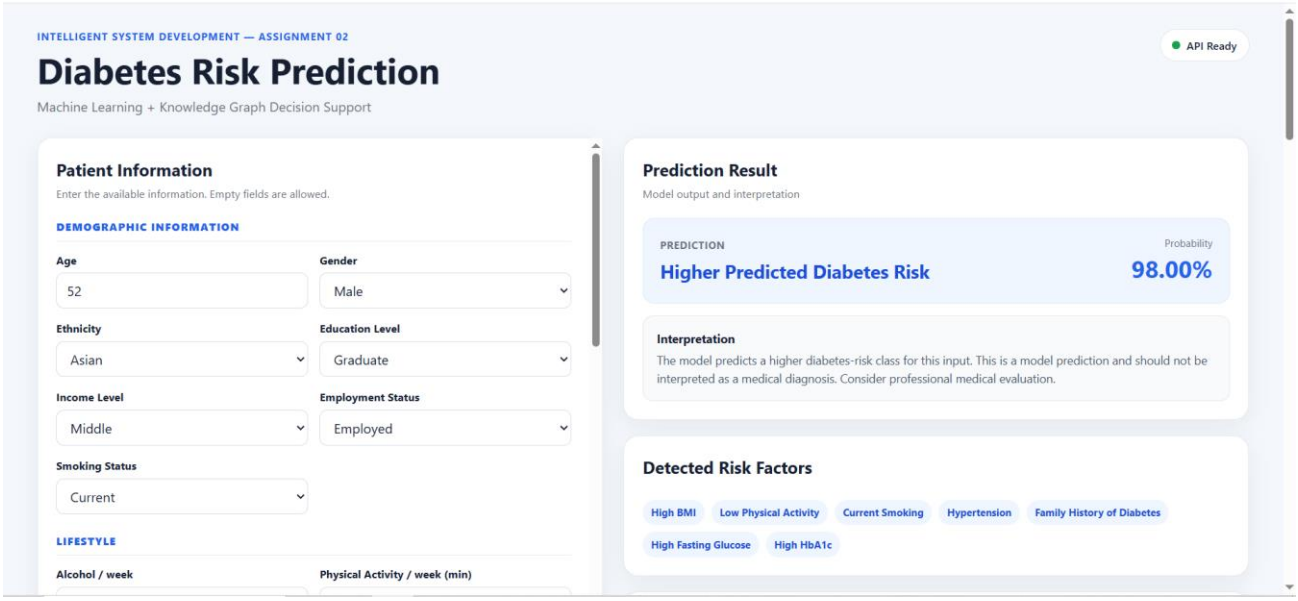
Trong trường hợp sử dụng Flask, mô hình được nạp vào ứng dụng Python. Người dùng nhập dữ liệu thông qua giao diện Web, Web gửi request đến server, Flask xử lý dữ liệu, thực hiện preprocessing và đưa dữ liệu vào model để nhận kết quả dự đoán.

2.21. Demo dự đoán

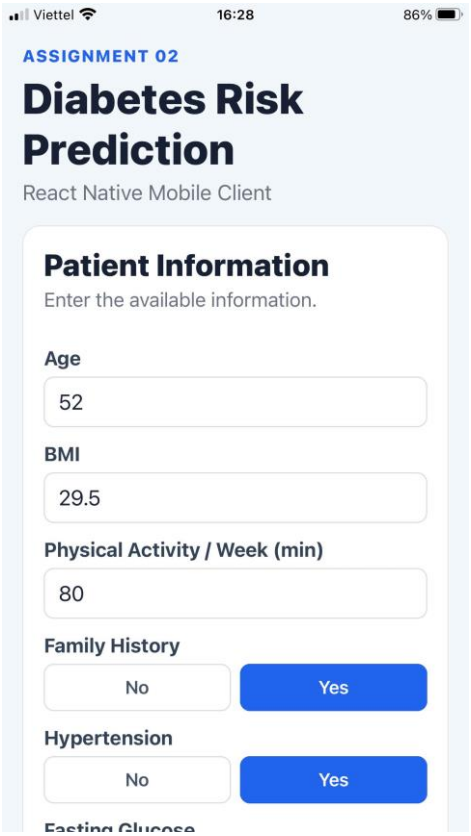
Việc triển khai Web giúp kiểm chứng khả năng sử dụng mô hình ngoài môi trường notebook. Người dùng có thể nhập dữ liệu trên giao diện và nhận kết quả dự đoán từ mô hình.

2.21.1. Giao diện hệ thống dự đoán bệnh tiểu đường

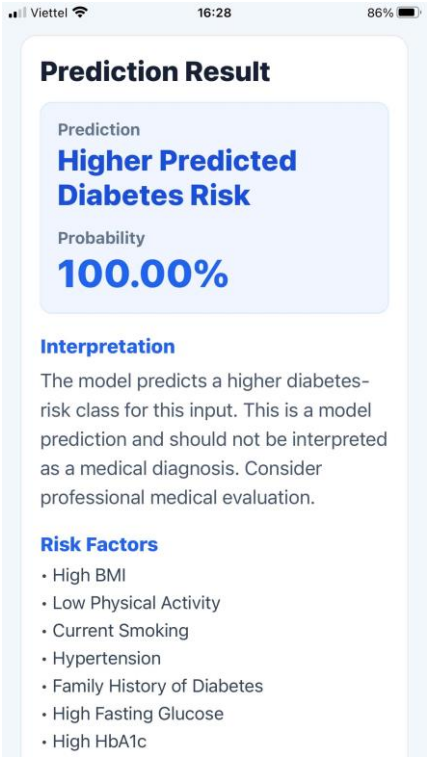
Hệ thống dự đoán bệnh tiểu đường được triển khai cả ở dạng web và mobile:



Hình 2.6 – Giao diện web Diabetes Prediction: màn hình nhập và kết quả.

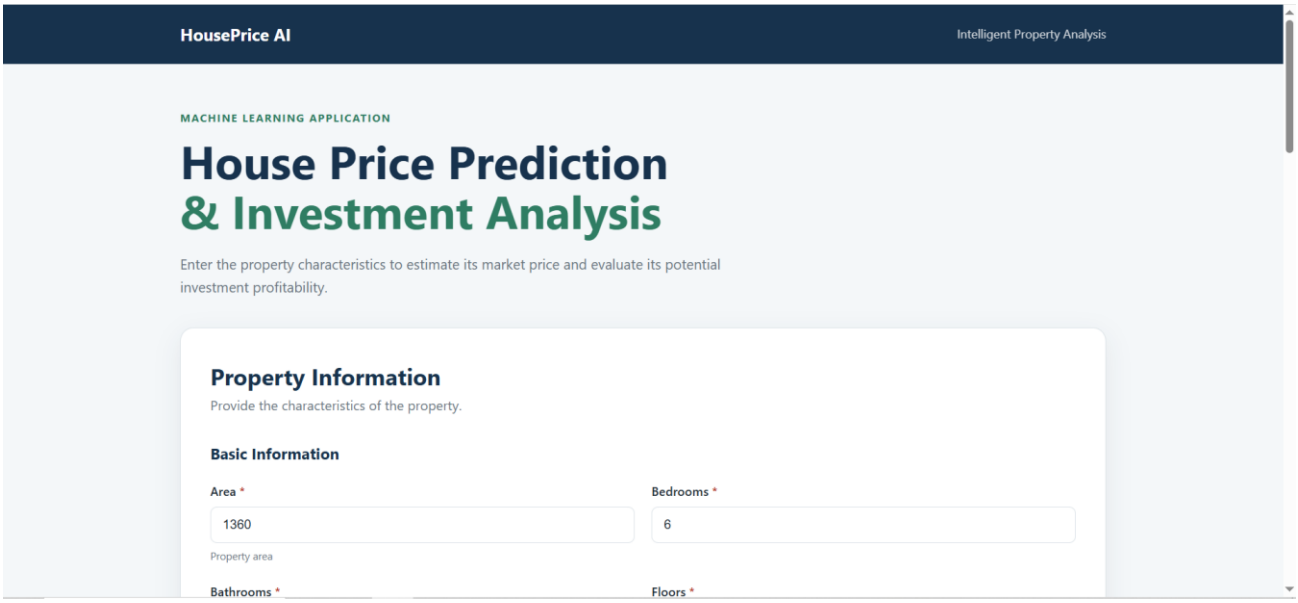


Hình 2.7 – Giao diện mobile Diabetes Prediction.

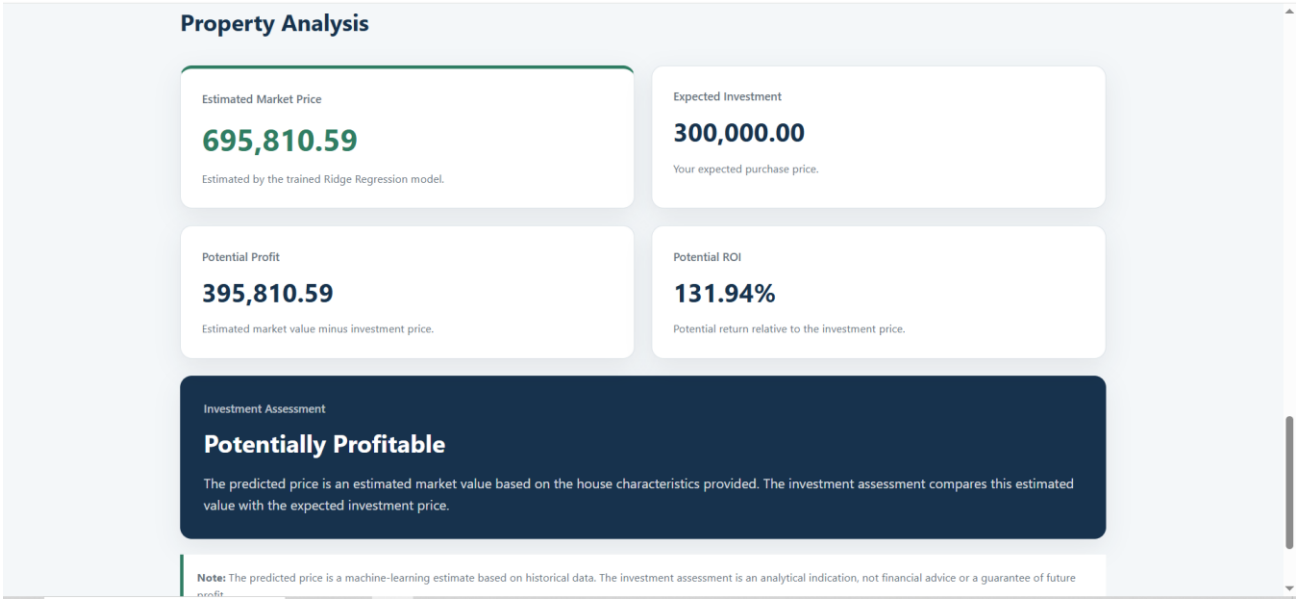


Hình 2.8 – Giao diện kết quả mobile Diabetes Prediction

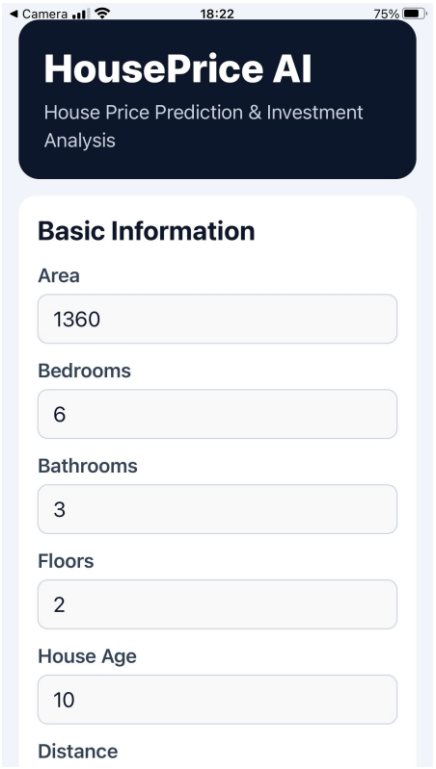
2.21.2. Giao diện hệ thống dự đoán giá nhà



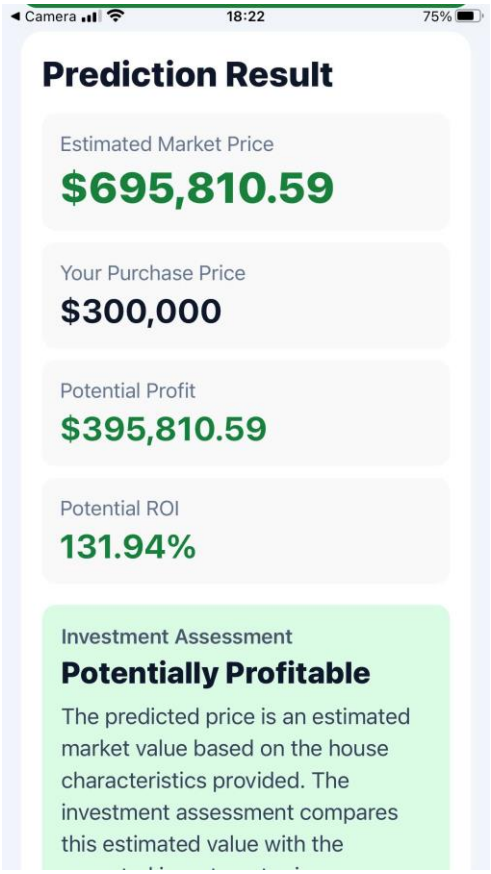
Hình 2.9 – Giao diện web House Price Prediction.



Hình 2.10 – Giao diện kết quả web House Price Prediction.



Hình 2.11 – Giao diện mobile House Price Prediction.



Hình 2.12 – Giao diện kết quả mobile House Price Prediction.

Nhận xét: Ứng dụng cho phép chuyển mô hình hồi quy từ môi trường notebook thành một công cụ tương tác, trong đó người dùng cung cấp các đặc trưng của căn nhà và nhận giá trị dự đoán từ mô hình.

2.22. Nhận xét chung về các kỹ thuật Machine Learning cơ bản

Qua phần lý thuyết và thực nghiệm có thể thấy Machine Learning bao gồm nhiều nhóm kỹ thuật khác nhau, trong đó mỗi nhóm phù hợp với một loại bài toán và một dạng dữ liệu nhất định.

Linear Regression và các biến thể Ridge, Lasso phù hợp với các bài toán dự đoán giá trị liên tục. Logistic Regression, KNN, Decision Tree, Random Forest và SVM là những thuật toán cơ bản quan trọng đối với bài toán classification.

Đối với dữ liệu không có nhãn, các kỹ thuật như K-Means và PCA hỗ trợ khám phá cấu trúc dữ liệu.

Bên cạnh thuật toán, **data representation** là một yếu tố có ảnh hưởng rất lớn đến kết quả. Dữ liệu dạng bảng, dữ liệu phân loại và dữ liệu văn bản cần những cách biểu diễn khác nhau. Trường hợp dữ liệu văn bản cho thấy việc chuyển đổi review thành vector TF-IDF có thể cung cấp thêm thông tin hữu ích cho mô hình.

Các thực nghiệm cũng cho thấy việc đánh giá mô hình phải sử dụng metric phù hợp với bài toán. Classification không nên chỉ xem Accuracy mà cần xem thêm Precision, Recall, F1-score và Confusion Matrix. Regression cần kết hợp MAE, RMSE và R^2 để có cái nhìn đầy đủ hơn.

Cuối cùng, một mô hình Machine Learning chỉ là một thành phần của hệ thống hoàn chỉnh. Sau quá trình huấn luyện và đánh giá, mô hình cần được lưu trữ, kiểm thử inference và có thể được triển khai thành API hoặc Web để người dùng tương tác.

2.23. Tiểu kết chương

Chương 2 đã trình bày những kỹ thuật Machine Learning cơ bản và quy trình xây dựng một hệ thống học máy từ dữ liệu đến triển khai. Nội dung tập trung vào hai nhóm bài toán chính là classification và regression, đồng thời giới thiệu thêm các kỹ thuật học không giám sát như K-Means và PCA.

Các thuật toán cơ bản gồm Linear Regression, Ridge, Lasso, Logistic Regression, KNN, Decision Tree, Random Forest, SVM và Naive Bayes đã được trình bày cùng với ưu điểm, hạn chế và phạm vi sử dụng. Bên cạnh đó, chương cũng đề cập đến các vấn đề quan trọng như

biểu diễn dữ liệu, tiền xử lý, chuẩn hóa, TF-IDF, đánh giá mô hình, overfitting và khả năng tổng quát hóa.

Ba bài toán thực nghiệm về dự đoán nguy cơ bệnh tiểu đường, dự đoán giá nhà và phân tích hành vi khách hàng được sử dụng để minh họa cho các nhóm kỹ thuật trên. Kết quả cho thấy hiệu quả của một hệ thống Machine Learning không chỉ phụ thuộc vào thuật toán mà còn phụ thuộc lớn vào chất lượng dữ liệu, cách biểu diễn dữ liệu và quy trình tiền xử lý.

Quá trình từ dữ liệu đến mô hình và triển khai có thể được khái quát:

Dữ liệu → Tiền xử lý → Biểu diễn → Huấn luyện → Đánh giá → Lưu mô hình → Triển khai → Dự đoán

Những kiến thức về Machine Learning truyền thống trong chương này là nền tảng để chuyển sang các phương pháp Deep Learning. Trong chương tiếp theo, nội dung sẽ được mở rộng từ việc sử dụng các feature được thiết kế trước sang khả năng học đặc trưng tự động thông qua Convolutional Neural Network (CNN)**.**

CHƯƠNG 3. CONVOLUTIONAL NEURAL NETWORK

3.1. Tổng quan về Convolutional Neural Network

Convolutional Neural Network (CNN) là một kiến trúc mạng nơ-ron nhân tạo thuộc nhóm Deep Learning, được phát triển đặc biệt hiệu quả đối với dữ liệu có cấu trúc không gian như hình ảnh. So với các mạng nơ-ron truyền thống, CNN có khả năng khai thác mối quan hệ cục bộ giữa các phần tử dữ liệu thông qua phép tích chập (convolution).

Đối với một hình ảnh, các pixel nằm gần nhau thường có mối quan hệ với nhau. Những cấu trúc nhỏ như cạnh, góc, đường nét có thể kết hợp để tạo thành các cấu trúc lớn hơn như hình dạng của vật thể. CNN tận dụng đặc điểm này bằng cách sử dụng các kernel hoặc filter để quét trên ảnh và tự động học các đặc trưng quan trọng.

Một CNN điển hình có thể bao gồm:

- Convolution layer.
- Activation function.
- Pooling layer.
- Batch Normalization.
- Fully Connected layer.
- Một số cơ chế cải tiến như Residual Connection hoặc Attention.

Quá trình xử lý có thể được khái quát:

Ảnh đầu vào → Convolution → Activation → Pooling → Trích xuất đặc trưng → Fully Connected → Dự đoán

Điểm quan trọng của CNN là các đặc trưng không nhất thiết phải được thiết kế thủ công. Trong quá trình huấn luyện, mạng tự điều chỉnh các filter để tìm ra những pattern có ích cho bài toán.

CNN được sử dụng rộng rãi trong:

- Nhận dạng chữ viết.
- Phân loại hình ảnh.
- Nhận diện vật thể.
- Phân tích ảnh y tế.
- Nhận dạng khuôn mặt.
- Phân tích ảnh vệ tinh.
- Phân loại sản phẩm.
- Kiểm tra lỗi trong sản xuất.

3.2. Từ mạng nơ-ron truyền thống đến CNN

Trong mạng nơ-ron truyền thống, dữ liệu hình ảnh thường phải được chuyển thành một vector một chiều trước khi đưa vào Fully Connected layer.

Ví dụ, một ảnh grayscale kích thước 28 × 28 có: 28 × 28 = 784 pixel

Nếu chuyển toàn bộ ảnh thành vector, đầu vào sẽ có 784 giá trị.

Cách làm này có một số hạn chế. Các pixel vốn có quan hệ về vị trí trong ảnh nhưng khi trải phẳng thành vector, thông tin về cấu trúc không gian không còn được khai thác trực tiếp.

CNN giải quyết vấn đề này bằng cách thực hiện convolution trực tiếp trên ma trận ảnh.

Ví dụ: Ảnh 28 × 28 → Convolution → Feature Map

Thay vì kết nối tất cả pixel với tất cả neuron, CNN chỉ quan sát một vùng cục bộ của ảnh tại mỗi vị trí. Nhờ đó số lượng tham số có thể giảm đáng kể và mô hình có khả năng học các đặc trưng không gian.

3.3. Dữ liệu đầu vào của CNN

CNN thường xử lý dữ liệu dưới dạng tensor.

Đối với ảnh grayscale, dữ liệu thường có dạng: **C × H × W**

Trong đó:

- C là số channel.
- H là chiều cao.
- W là chiều rộng.

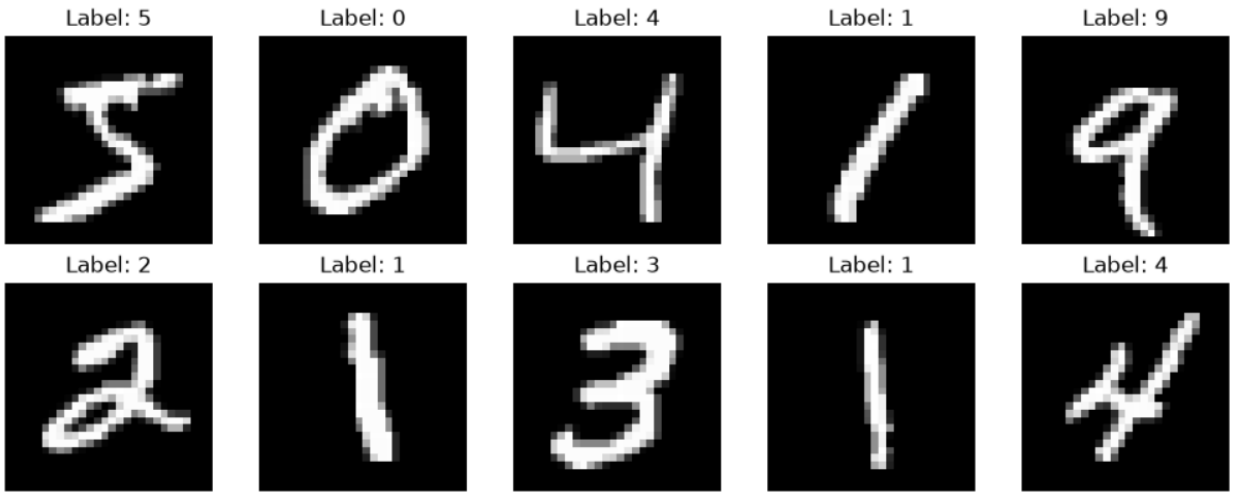
Ảnh grayscale có một channel nên ảnh MNIST kích thước 28 × 28 có dạng: **1 × 28 × 28**

Đối với ảnh màu RGB: **3 × H × W**

Trong quá trình huấn luyện theo batch, tensor thường có thêm chiều batch: **N × C × H × W**

Trong đó N là số lượng ảnh trong một batch.

Việc hiểu đúng kích thước tensor rất quan trọng khi xây dựng CNN, bởi vì mỗi layer yêu cầu một định dạng đầu vào nhất định.



Hình 3.1 – Một số ảnh mẫu MNIST

Nhận xét: MNIST chứa các ảnh chữ số viết tay kích thước 28 × 28 pixel. Dữ liệu có cấu trúc không gian rõ ràng nên phù hợp để minh họa khả năng học đặc trưng của CNN.

3.4. Phép Convolution

Convolution là thành phần quan trọng nhất của CNN.

Một kernel có kích thước nhỏ, ví dụ 3 × 3, được dịch chuyển trên ảnh. Tại mỗi vị trí, các giá trị trong kernel được nhân với các pixel tương ứng rồi cộng lại để tạo ra một giá trị trong feature map.

Với kernel K và vùng ảnh X, phép tính có thể được biểu diễn đơn giản: **Output(i,j) = ΣΣ X(i+m,j+n) × K(m,n)**

Trong đó:

- X là vùng dữ liệu đầu vào.
- K là kernel.
- i, j xác định vị trí.
- m, n xác định vị trí bên trong kernel.

Các kernel khác nhau có thể học các đặc trưng khác nhau.

Ở những layer đầu tiên, CNN thường học các đặc trưng đơn giản như:

- Cạnh ngang.
- Cạnh dọc.
- Góc.
- Đường nét.

Ở các layer sâu hơn, những đặc trưng đơn giản có thể được kết hợp thành:

- Hình dạng.
- Bộ phận của vật thể.
- Pattern phức tạp hơn.

Do đó, CNN tạo ra quá trình học đặc trưng theo nhiều tầng.

3.5. Kernel và Parameter Sharing

Một đặc điểm quan trọng của CNN là **parameter sharing**.

Thay vì sử dụng một tập trọng số riêng cho từng vị trí của ảnh, cùng một kernel được sử dụng khi quét qua toàn bộ ảnh.

Ví dụ một kernel 3×3 chỉ có: $3 \times 3 = 9$ trọng số và một bias.

Các trọng số này được sử dụng ở nhiều vị trí khác nhau.

Điều này mang lại hai lợi ích:

- Giảm đáng kể số lượng tham số.
- Cho phép mô hình phát hiện cùng một đặc trưng ở những vị trí khác nhau.

Đây là một trong những lý do CNN phù hợp với dữ liệu hình ảnh.

3.6. Stride và Padding

Hai tham số quan trọng của convolution là stride và padding.

3.6.1. Stride

Stride xác định số pixel mà kernel dịch chuyển sau mỗi lần tính.

Nếu stride bằng 1, kernel dịch chuyển từng pixel.

Nếu stride bằng 2, kernel dịch chuyển hai pixel mỗi lần.

Stride lớn hơn giúp giảm kích thước feature map nhưng cũng làm mất nhiều thông tin không gian hơn.

3.6.2. Padding

Padding bổ sung thêm các giá trị xung quanh biên của ảnh.

Một trường hợp phổ biến là padding bằng 0.

Padding giúp:

- Giữ kích thước không gian của feature map.
- Hạn chế mất thông tin ở vùng biên.

- Cho phép xây dựng nhiều convolution layer liên tiếp mà kích thước không giảm quá nhanh.

Với convolution có kernel 3 × 3, stride bằng 1 và padding bằng 1, kích thước chiều cao và chiều rộng có thể được giữ nguyên.

3.7. Kích thước Feature Map

Kích thước đầu ra của một convolution layer có thể được tính theo công thức:

$$\textbf{Output} = \text{floor}[(W - K + 2P) / S] + 1$$

Trong đó:

- W là kích thước đầu vào.
- K là kích thước kernel.
- P là padding.
- S là stride.

Ví dụ với ảnh 28 × 28, kernel 3 × 3, padding 1 và stride 1:

$$\textbf{Output} = [(28 - 3 + 2 \times 1) / 1] + 1 = 28$$

Do đó feature map vẫn có kích thước 28 × 28.

Nếu stride bằng 2:

$$\textbf{Output} = [(28 - 3 + 2 \times 1) / 2] + 1 = 14$$

Kích thước không gian giảm xuống còn khoảng một nửa.

Việc kiểm soát kích thước feature map là một phần quan trọng trong thiết kế CNN.

3.8. Hàm kích hoạt ReLU

Sau convolution, CNN thường sử dụng hàm kích hoạt để đưa tính phi tuyến vào mô hình.

Một hàm phổ biến là ReLU: $\textbf{ReLU(x) = max(0, x)}$

Điều này có nghĩa:

- Nếu $x < 0$ thì kết quả bằng 0.
- Nếu $x \geq 0$ thì kết quả bằng x.

ReLU có ưu điểm là đơn giản và tính toán nhanh. Đồng thời, nó giúp mạng có khả năng học các quan hệ phi tuyến phức tạp.

Nếu chỉ sử dụng các phép biến đổi tuyến tính liên tiếp, nhiều layer vẫn có thể tương đương với một phép biến đổi tuyến tính. Activation function giúp phá vỡ tính tuyến tính đó.

3.9. Pooling Layer

Pooling được sử dụng để giảm kích thước không gian của feature map.

Hai phương pháp phổ biến là:

- Max Pooling.
- Average Pooling.

3.9.1. Max Pooling

Max Pooling chọn giá trị lớn nhất trong một vùng.

Ví dụ với cửa sổ 2 × 2:

[1 3]
[2 4]

kết quả là: 4

Max Pooling có tác dụng giữ lại các tín hiệu nổi bật trong vùng.

3.9.2. Average Pooling

Average Pooling tính giá trị trung bình trong vùng.

Nếu vùng 2 × 2 có các giá trị 1, 2, 3, 4 thì kết quả: (1 + 2 + 3 + 4) / 4 = 2,5

Pooling giúp:

- Giảm kích thước dữ liệu.
- Giảm chi phí tính toán.
- Giảm số lượng tham số ở các layer sau.

Tăng một mức độ bất biến với những thay đổi nhỏ về vị trí.

3.10. Fully Connected Layer

Sau nhiều convolution và pooling layer, feature map thường được chuyển thành vector trước khi đưa vào Fully Connected layer.

Ví dụ:

Feature Map → Flatten → Vector → Fully Connected → Output

Fully Connected layer thực hiện phép biến đổi giữa toàn bộ các neuron đầu vào và neuron đầu ra.

Trong bài toán phân loại 10 chữ số MNIST, layer cuối có thể có 10 giá trị tương ứng với 10 lớp từ 0 đến 9.

Kết quả đầu ra có thể được đưa qua Softmax để chuyển thành xác suất:

$$P(y=i) = e^{z_i} / \Sigma e^{z_j}$$

Trong đó z là vector đầu ra của layer cuối.

Lớp có xác suất cao nhất thường được chọn làm kết quả dự đoán.

3.11. Hàm mất mát trong CNN

CNN được huấn luyện bằng cách tối ưu một hàm mất mát (loss function).

Đối với classification nhiều lớp, Cross Entropy Loss là một lựa chọn phổ biến.

Công thức có thể viết:

$$L = -\Sigma\, y_i \log(p_i)$$

Trong đó:

y_i là nhãn thực tế.

p_i là xác suất mô hình dự đoán cho lớp i.

Mục tiêu của quá trình huấn luyện là làm giảm giá trị loss.

Quá trình này được thực hiện thông qua backpropagation và optimizer.

3.12. Backpropagation và tối ưu mô hình

Trong quá trình huấn luyện, CNN thực hiện các bước cơ bản: **Forward → Tính Loss → Backward → Cập nhật trọng số**

Ở bước forward, dữ liệu đi qua mạng để tạo ra prediction.

Sau đó loss được tính dựa trên prediction và target.

Ở bước backward, gradient của loss đối với các tham số được tính toán.

Optimizer sử dụng gradient để cập nhật trọng số.

Một cách biểu diễn đơn giản:

$$\theta \leftarrow \theta - \eta \times \nabla L(\theta)$$

Trong đó:

θ là tham số của mô hình.

η là learning rate.

$\nabla L(\theta)$ là gradient của loss.

Adam là một optimizer phổ biến trong Deep Learning và cũng được sử dụng trong các thực nghiệm CNN của nghiên cứu này.

3.13. Batch Normalization

Batch Normalization là một kỹ thuật thường được sử dụng để ổn định quá trình huấn luyện mạng. Thay vì để activation có phân bố thay đổi quá mạnh giữa các bước huấn luyện, Batch Normalization chuẩn hóa dữ liệu trong batch.

Một dạng biểu diễn đơn giản:

$$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$$

Sau đó sử dụng hai tham số có thể học là γ và β:

$$y = \gamma\hat{x} + \beta$$

Batch Normalization có thể giúp:

- Quá trình huấn luyện ổn định hơn.
- Tốc độ hội tụ tốt hơn.
- Cho phép sử dụng learning rate phù hợp hơn.
- Trong một số trường hợp hỗ trợ giảm overfitting.

Trong thực nghiệm CNN, việc thêm Batch Normalization tạo thành một phiên bản cải tiến từ mô hình cơ sở.

3.14. Residual Connection

Khi mạng trở nên sâu hơn, việc tối ưu có thể trở nên khó khăn. Residual Network đưa ra ý tưởng sử dụng một đường tắt gọi là skip connection.

Thay vì học trực tiếp một hàm H(x), block có thể học phần residual F(x):

$$H(x) = F(x) + x$$

Trong đó x được truyền trực tiếp qua shortcut rồi cộng với đầu ra của một số convolution layer.

Ý tưởng này giúp thông tin và gradient có thể truyền qua mạng thuận lợi hơn.

Residual connection trở thành một trong những ý tưởng quan trọng trong sự phát triển của CNN hiện đại.

Trong thực nghiệm của chương, mô hình M3 Residual được xây dựng để kiểm tra ảnh hưởng của cơ chế này.

3.15. Attention trong CNN

Attention là cơ chế cho phép mô hình học mức độ quan trọng của các thành phần trong feature representation. Trong CNN, attention có thể được áp dụng theo không gian hoặc theo channel.

Một hướng đơn giản là channel attention, trong đó mô hình học cách tăng trọng số cho những channel quan trọng và giảm ảnh hưởng của những channel ít quan trọng. Trong thực nghiệm, cơ chế SE được sử dụng trong M4 Attention.

Tuy nhiên, việc bổ sung một cơ chế phức tạp hơn không đảm bảo mô hình luôn có accuracy cao hơn. Hiệu quả phụ thuộc vào:

- Kích thước dataset.
- Độ khó của bài toán.
- Kiến trúc nền.
- Cách huấn luyện.
- Hyperparameter.

Đây là vấn đề sẽ được kiểm chứng thông qua các thực nghiệm ở phần sau.

3.16. Các kiến trúc CNN tiêu biểu

CNN đã phát triển qua nhiều kiến trúc khác nhau.

3.16.1. LeNet

LeNet là một trong những kiến trúc CNN kinh điển, được thiết kế cho bài toán nhận dạng chữ số viết tay.

Kiến trúc tương đối đơn giản, gồm convolution, pooling và fully connected layer.

LeNet có ưu điểm:

- Nhẹ.
- Dễ hiểu.
- Dễ huấn luyện.

Tuy nhiên, kiến trúc này có giới hạn khi áp dụng cho những bài toán phức tạp hơn.

3.16.2. AlexNet

AlexNet là một kiến trúc CNN có ảnh hưởng lớn đến sự phát triển của Deep Learning trong xử lý ảnh.

So với các kiến trúc CNN trước đó, AlexNet sử dụng mạng sâu hơn và số lượng tham số lớn hơn.

Trong thực nghiệm, một phiên bản AlexNet được điều chỉnh để phù hợp với ảnh grayscale kích thước 28×28 thay vì sử dụng nguyên cấu hình dành cho các dataset ảnh lớn.

3.16.3. VGG

VGG sử dụng nhiều convolution layer với kernel nhỏ, thường là 3×3, để xây dựng mạng sâu.

Ý tưởng quan trọng là sử dụng nhiều layer convolution nhỏ thay vì một convolution lớn.

Trong thực nghiệm, VGG cũng được điều chỉnh để phù hợp với kích thước và đặc điểm của MNIST và Fashion-MNIST.

Do đó, các mô hình Adapted AlexNet và Adapted VGG trong thực nghiệm cần được hiểu là **phiên bản thích nghi**, không phải cấu hình nguyên bản của các kiến trúc lịch sử.

3.17. Các kỹ thuật hạn chế Overfitting trong CNN

CNN có thể có rất nhiều tham số, đặc biệt khi mạng sâu và có nhiều Fully Connected layer. Vì vậy, overfitting là vấn đề quan trọng.

Một số phương pháp phổ biến gồm:

Dropout

Dropout tạm thời loại bỏ một phần neuron trong quá trình training. Điều này giúp mạng không phụ thuộc quá mức vào một nhóm neuron cụ thể.

Data Augmentation

Data Augmentation tạo ra các biến thể mới từ dữ liệu hiện có.

Đối với ảnh có thể thực hiện:

- Xoay.
- Dịch chuyển.
- Phóng to hoặc thu nhỏ.
- Lật ảnh trong trường hợp phù hợp.
- Thay đổi một số đặc điểm hình ảnh.

Data Augmentation giúp mô hình tiếp xúc với nhiều biến thể hơn của dữ liệu.

Early Stopping

Theo dõi performance trên validation set và dừng quá trình huấn luyện khi mô hình không còn cải thiện.

Regularization

Regularization thêm thành phần phạt vào hàm loss để hạn chế trọng số quá lớn.

Những kỹ thuật này có thể được kết hợp tùy thuộc vào bài toán.

3.18. Các bộ dữ liệu sử dụng trong thực nghiệm

Để đánh giá CNN trên các dạng dữ liệu khác nhau, ba dataset được sử dụng gồm: MNIST, Fashion-MNIST, Diabetes.

Hai dataset đầu là dữ liệu ảnh và phù hợp trực tiếp với Conv2d. Dataset Diabetes có dạng bảng nên được sử dụng để minh họa cách điều chỉnh convolution theo chiều một thông qua Conv1d.

Cách lựa chọn này giúp so sánh khả năng hoạt động của CNN trên **Dữ liệu ảnh → Conv2d** và **Dữ liệu dạng vector → Conv1d**

3.19. Dataset MNIST

MNIST là bộ dữ liệu gồm các ảnh chữ số viết tay từ 0 đến 9. Mỗi ảnh có kích thước 28 × 28 pixel và được biểu diễn dưới dạng grayscale.

Bài toán của MNIST là classification với 10 lớp.

MNIST thường được sử dụng làm dataset cơ bản để kiểm tra các thuật toán xử lý ảnh vì:

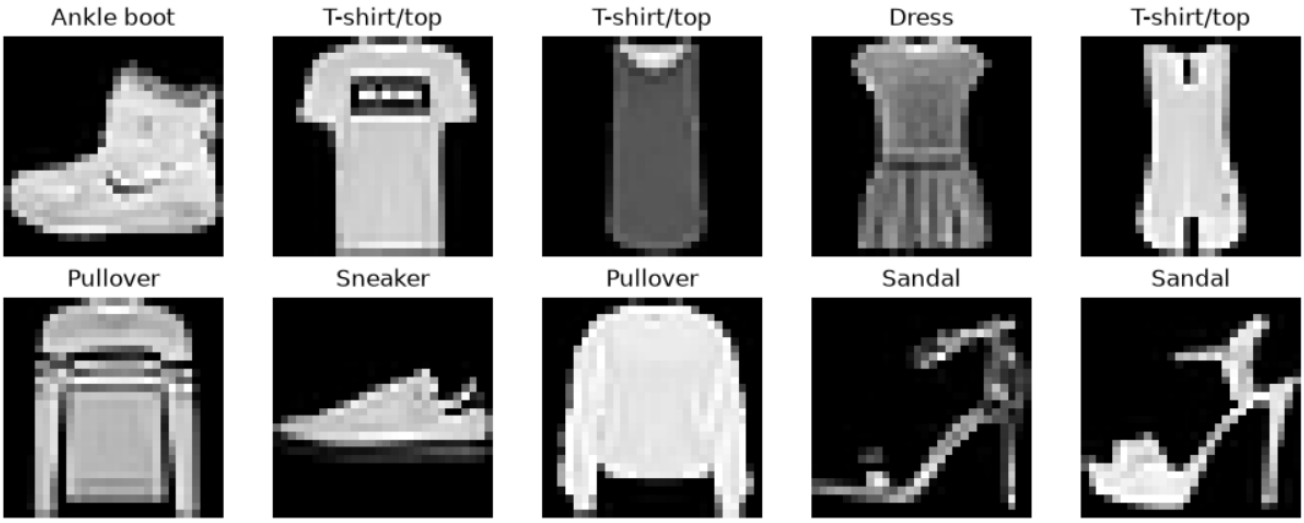
- Dữ liệu có cấu trúc rõ ràng.
- Kích thước ảnh nhỏ.
- Có 10 lớp tương đối cân bằng.
- Dễ triển khai và huấn luyện.

3.20. Dataset Fashion-MNIST

Fashion-MNIST có cấu trúc tương tự MNIST nhưng thay chữ số bằng hình ảnh các sản phẩm thời trang.

Mỗi ảnh có kích thước: 28 × 28 × 1 và bài toán cũng gồm 10 lớp. Các lớp bao gồm các loại sản phẩm thời trang như áo, giày, túi và các nhóm sản phẩm khác. Fashion-MNIST khó hơn MNIST bởi một số lớp có hình ảnh tương đối giống nhau. Ví dụ các loại áo có thể có hình dạng gần nhau.

Điều này khiến Fashion-MNIST phù hợp để đánh giá khả năng phân biệt các pattern phức tạp hơn của CNN.



Hình 3.2 – Một số ảnh mẫu Fashion-MNIST

Nhận xét: Các hình ảnh có cấu trúc phức tạp hơn chữ số viết tay. Sự tương đồng giữa một số lớp khiến bài toán khó hơn MNIST.

3.21. Dataset Diabetes và CNN một chiều

Dataset Diabetes có dạng dữ liệu bảng thay vì hình ảnh.

Trong thực nghiệm, dữ liệu gồm các thuộc tính số và biến mục tiêu nhị phân. Thay vì coi các feature như một ảnh hai chiều, vector feature được tổ chức thành dạng phù hợp với Conv1d.

Có thể hình dung: **Feature vector → 1 × N → Conv1d → Feature representation**

Cách làm này chủ yếu nhằm minh họa khả năng sử dụng convolution trên dữ liệu một chiều. Nó không có nghĩa rằng mọi bài toán dữ liệu bảng đều phù hợp với CNN.

3.22. Thiết kế các mô hình CNN thực nghiệm

Để nghiên cứu ảnh hưởng của các cải tiến kiến trúc, bốn mô hình CNN được xây dựng theo hướng tăng dần mức độ cải tiến:

Mô hình	Đặc điểm chính
M1 Basic CNN	Mô hình CNN cơ sở
M2 BatchNorm CNN	Bổ sung Batch Normalization
M3 Residual CNN	Bổ sung Residual Connection
M4 Attention CNN	Bổ sung cơ chế Attention

Bốn mô hình này được sử dụng trên MNIST, Fashion-MNIST và Diabetes.

Ngoài ra, ba kiến trúc CNN kinh điển được điều chỉnh cho ảnh 28 × 28:

Mô hình	**Đặc điểm**
M5 LeNet	Kiến trúc CNN cổ điển, nhẹ
M6 Adapted AlexNet	Phiên bản thích nghi của AlexNet
M7 Adapted VGG	Phiên bản thích nghi của VGG

Ba mô hình M5–M7 được sử dụng cho các dataset ảnh.

Cách thiết kế này cho phép so sánh hai nhóm: **CNN cải tiến từ baseline** và **Các kiến trúc CNN kinh điển được điều chỉnh**

3.23. Thiết lập quá trình huấn luyện

Trong thực nghiệm, các mô hình được huấn luyện với các thiết lập thống nhất để kết quả có thể so sánh.

Một số thông số chính:

Thông số	**Giá trị**
Seed	42
Epoch cho dữ liệu ảnh	5
Epoch cho Diabetes	15
Learning rate	0,001
Optimizer	Adam
Loss cho ảnh	CrossEntropyLoss
Loss cho Diabetes	BCEWithLogitsLoss

Sau mỗi epoch, kết quả trên validation set được theo dõi. Trạng thái tốt nhất của mô hình được lưu lại thay vì mặc định sử dụng trạng thái ở epoch cuối.

Cách làm này giúp giảm khả năng sử dụng một trạng thái mô hình đã bắt đầu overfitting trên validation data.

3.24. Kết quả thực nghiệm trên MNIST

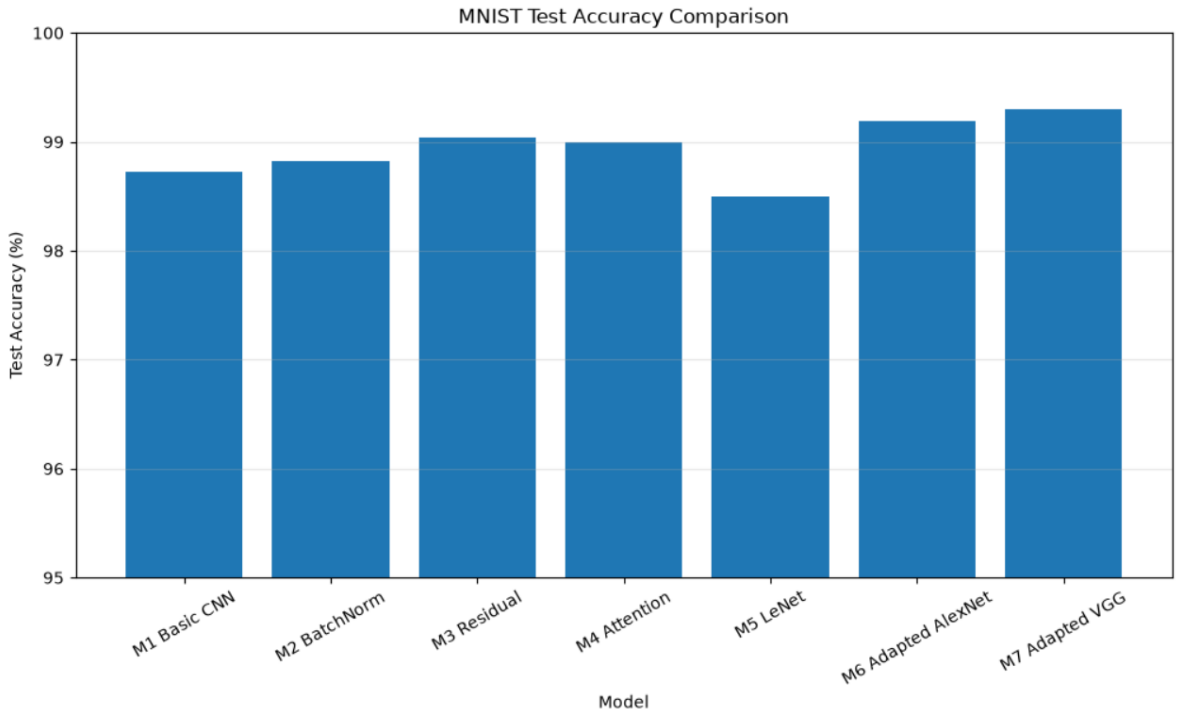
Kết quả trên MNIST:

Model	Accuracy	Parameters
M1 Basic CNN	98,72%	20.490
M2 BatchNorm	98,82%	20.586
M3 Residual	99,04%	25.290
M4 Attention	99,00%	25.438
M5 LeNet	98,50%	44.426
M6 Adapted AlexNet	99,19%	1.701.130
M7 Adapted VGG	**99,30%**	467.818

M7 Adapted VGG đạt accuracy cao nhất với 99,30%. Trong khi đó M1 Basic CNN đã đạt 98,72%, cho thấy MNIST là bài toán tương đối dễ đối với CNN.

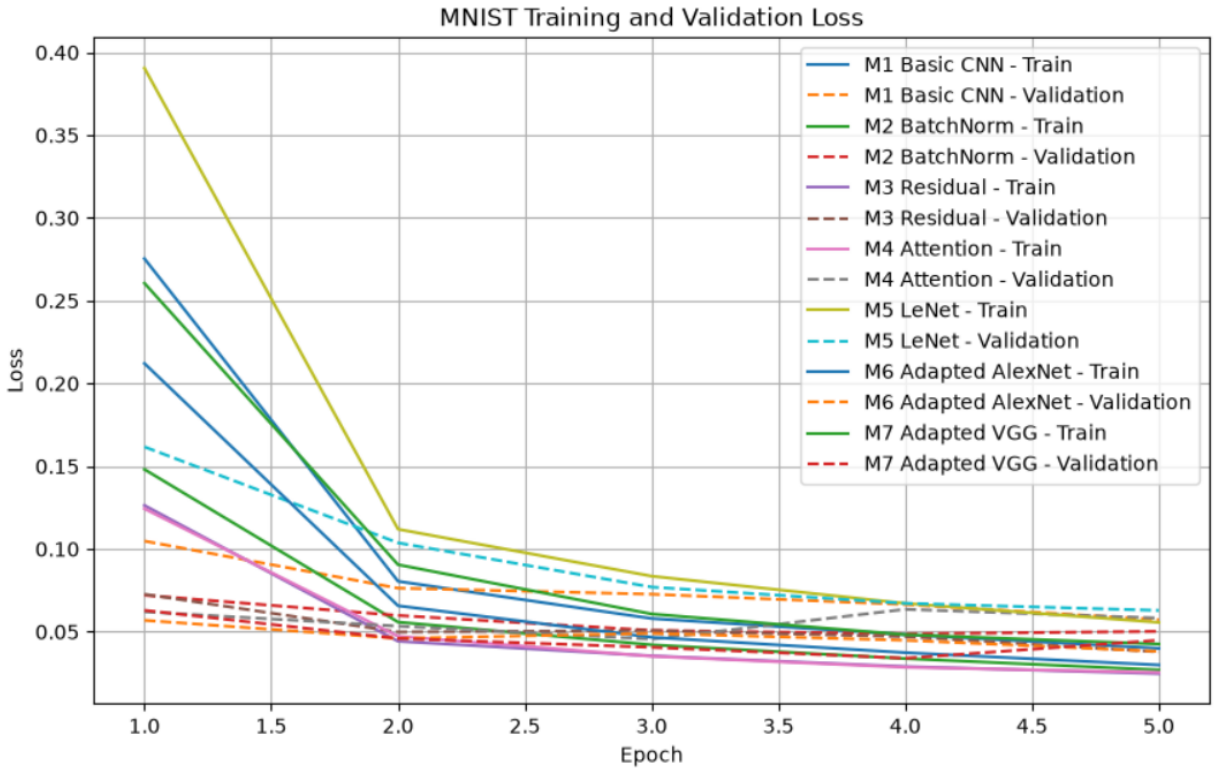
Mức cải thiện của M7 so với M1 là: **99,30% − 98,72% = 0,58 điểm phần trăm**

Mức cải thiện không lớn vì baseline đã đạt performance rất cao.



Hình 3.3 – Biểu đồ so sánh Accuracy của các mô hình trên MNIST

Nhận xét: Các mô hình đều đạt accuracy cao. M7 đạt kết quả cao nhất nhưng cần nhiều tham số hơn đáng kể so với M1.



Hình 3.4 – Training Loss và Validation Loss trên MNIST

Nhận xét: Đường loss cho thấy quá trình mô hình học qua các epoch. Việc so sánh training loss và validation loss giúp nhận biết xu hướng hội tụ và dấu hiệu overfitting.

3.25. Kết quả thực nghiệm trên Fashion-MNIST

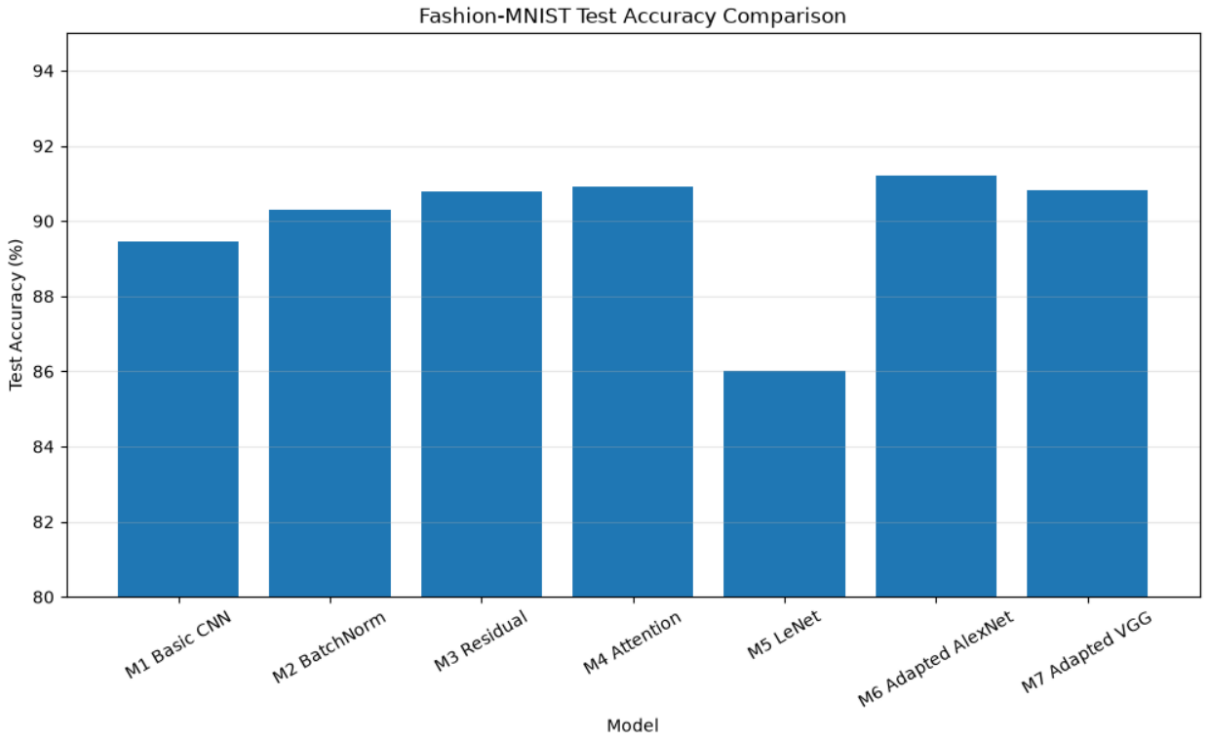
Kết quả:

Model	Accuracy	Parameters	Thời gian (s)
M1 Basic CNN	89,46%	20.490	88,13
M2 BatchNorm	90,29%	20.586	103,22
M3 Residual	90,78%	25.290	190,37
M4 Attention	90,92%	25.438	203,07
M5 LeNet	86,02%	44.426	—
M6 Adapted AlexNet	**91,20%**	1.701.130	246,36
M7 Adapted VGG	90,81%	467.818	346,91

Fashion-MNIST khó hơn MNIST nên accuracy thấp hơn rõ rệt.

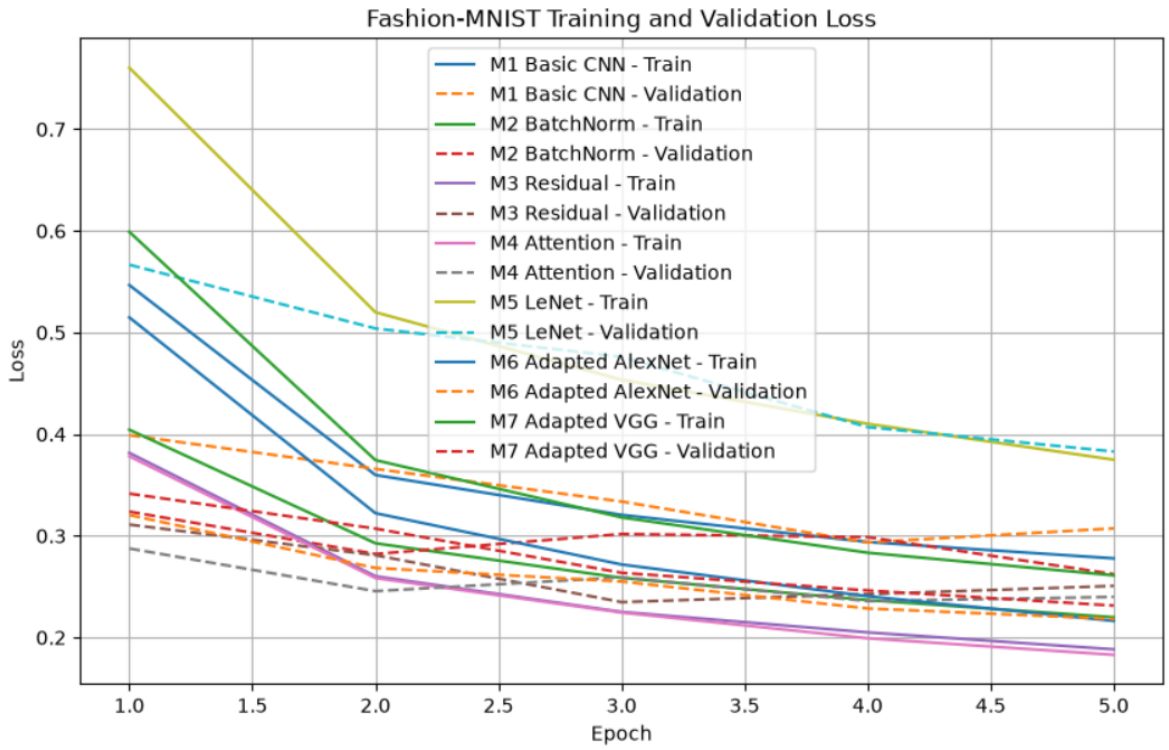
M6 Adapted AlexNet đạt kết quả cao nhất với 91,20%, cao hơn M1: **91,20% − 89,46% = 1,74 điểm phần trăm**

Trong nhóm M1–M4, M4 Attention đạt 90,92%. Một điểm đáng chú ý là M7 có test loss thấp nhưng M6 lại có accuracy cao hơn. Điều này cho thấy không nên đánh giá mô hình chỉ dựa trên một metric duy nhất.



Hình 3.5 – Biểu đồ Accuracy của các mô hình trên Fashion-MNIST

Nhận xét: Khoảng cách giữa các mô hình thể hiện rõ hơn so với MNIST. Điều này cho thấy bài toán khó hơn tạo ra nhiều không gian hơn để các kiến trúc khác nhau thể hiện ưu điểm.



Hình 3.6 – Training/Validation Loss trên Fashion-MNIST

Nhận xét: Biểu đồ giúp quan sát tốc độ hội tụ và sự khác biệt giữa performance trên training và validation.

3.26. Kết quả thực nghiệm trên Diabetes

Kết quả đối với Conv1d trên dữ liệu Diabetes:

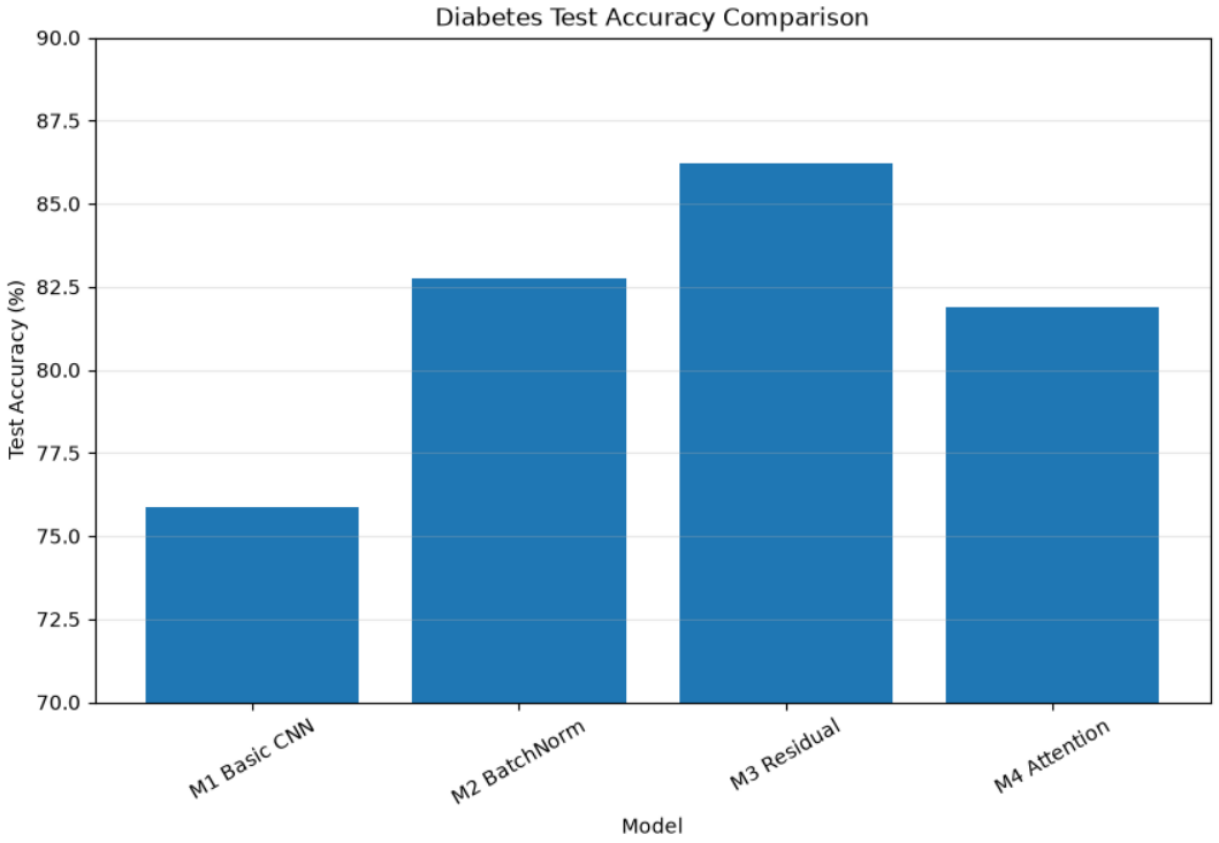
Model	Parameters	Test Loss	Accuracy
M1 Basic CNN	1.761	0,493523	75,86%
M2 BatchNorm	1.857	0,374922	82,76%
M3 Residual	3.489	0,389033	**86,21%**
M4 Attention	3.637	0,397135	81,90%

M3 Residual đạt accuracy cao nhất với 86,21%, tăng: **86,21% − 75,86% = 10,35 điểm phần trăm** so với M1 Basic CNN. Báo cáo thực nghiệm ghi nhận mức cải thiện khoảng 10,34 điểm phần trăm do cách làm tròn.

M2 cũng cải thiện đáng kể so với baseline. Trong khi đó M4 Attention có nhiều tham số và thời gian huấn luyện lớn hơn nhưng không đạt accuracy cao nhất.

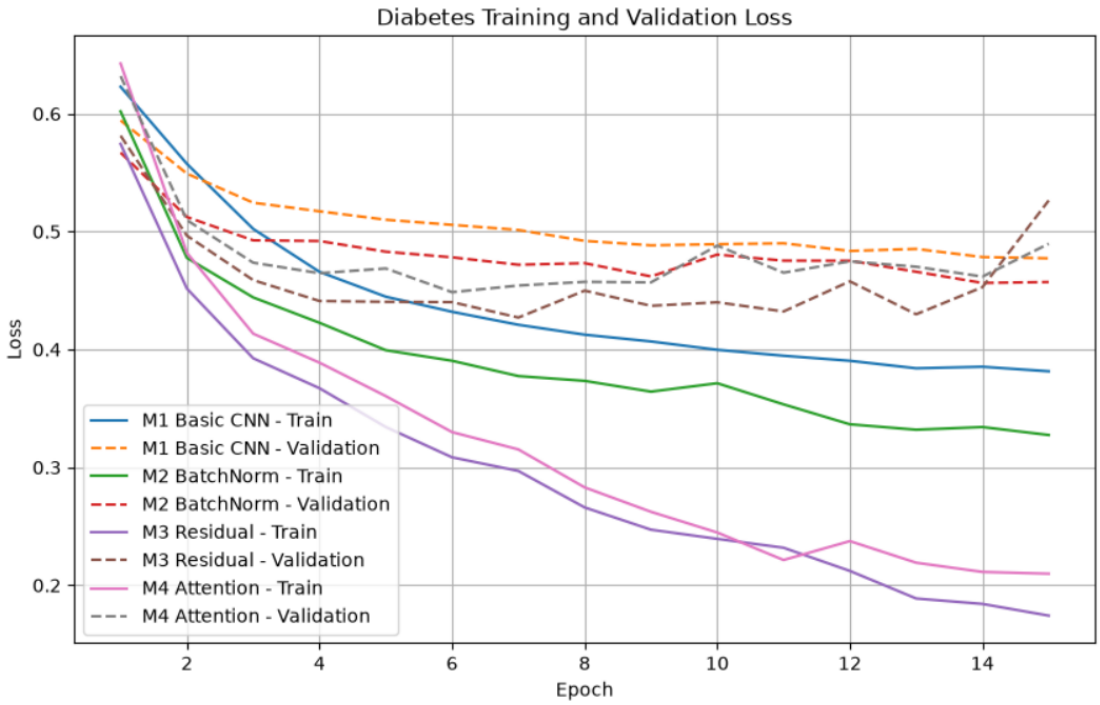
Kết quả này cho thấy **việc tăng độ phức tạp của kiến trúc không đồng nghĩa với việc performance luôn tăng**.

Đặc biệt, dataset Diabetes có quy mô nhỏ hơn nhiều so với MNIST và Fashion-MNIST. Do đó kết quả có thể nhạy với cách chia dữ liệu và không nên được dùng để kết luận rằng CNN luôn là phương pháp tốt nhất cho dữ liệu bảng.



Hình 3.7 – Biểu đồ Accuracy của M1–M4 trên Diabetes

Nhận xét: M3 Residual đạt kết quả cao nhất, trong khi M4 Attention không tạo ra cải thiện tương ứng với mức tăng độ phức tạp.



Hình 3.8 – Training/Validation Loss của Diabetes

3.27. So sánh mức cải thiện của CNN

Có thể tổng hợp mô hình tốt nhất trên từng dataset:

Dataset	Baseline M1	Mô hình tốt nhất	Accuracy tốt nhất	Cải thiện
MNIST	98,72%	M7 Adapted VGG	99,30%	+0,58 điểm %
Fashion-MNIST	89,46%	M6 Adapted AlexNet	91,20%	+1,74 điểm %
Diabetes	75,86%	M3 Residual	86,21%	+10,35 điểm %

Kết quả cho thấy mức cải thiện phụ thuộc mạnh vào dataset.

MNIST có baseline đã rất cao nên mức cải thiện tương đối nhỏ. Fashion-MNIST khó hơn nên sự khác biệt giữa các kiến trúc rõ hơn.

Diabetes có mức cải thiện lớn nhất. Tuy nhiên, đây cũng là dataset nhỏ và việc sử dụng Conv1d trên dữ liệu bảng được thực hiện chủ yếu để minh họa khả năng thích nghi của convolution với dữ liệu một chiều. Vì vậy cần thận trọng khi so sánh trực tiếp kết quả này với hai dataset hình ảnh.

3.28. Đánh giá Precision, Recall và F1-score

Accuracy không phải metric duy nhất được sử dụng.

Kết quả classification tiêu biểu:

Dataset	Model	Accuracy	Precision	Recall	F1-score
MNIST	M7 Adapted VGG	99,30%	99,30%	99,30%	99,30%
Fashion-MNIST	M6 Adapted AlexNet	91,20%	91,16%	91,20%	91,15%
Diabetes	M3 Residual	86,21%	86,69%	86,21%	85,68%

Đối với MNIST, các metric gần như bằng nhau và đều rất cao.

Fashion-MNIST có performance thấp hơn nhưng Precision, Recall và F1 vẫn ở mức tương đối tốt.

Riêng Diabetes, F1-score thấp hơn Accuracy. Điều này cho thấy chỉ nhìn Accuracy có thể chưa phản ánh đầy đủ hành vi phân loại của mô hình.

3.29. Độ phức tạp và thời gian huấn luyện

Ngoài accuracy, một mô hình Deep Learning còn cần được đánh giá về độ phức tạp.

Số lượng tham số của các mô hình trong thực nghiệm dao động từ khoảng 20 nghìn tham số đối với Basic CNN trên ảnh đến hơn 1,7 triệu tham số đối với Adapted AlexNet.

Có thể thấy:

- M1 có kiến trúc đơn giản và số tham số thấp.
- M3 và M4 có nhiều tham số hơn do bổ sung các cơ chế cải tiến.
- M6 Adapted AlexNet có số tham số lớn nhất.
- M7 Adapted VGG có số tham số thấp hơn M6 nhưng vẫn phức tạp hơn đáng kể so với M1.

Thời gian huấn luyện cũng có xu hướng tăng khi kiến trúc phức tạp hơn. Tuy nhiên, thời gian chạy không chỉ phụ thuộc vào số tham số mà còn phụ thuộc vào:

- Số lượng layer.
- Loại phép toán.
- Batch size.
- Phần cứng.
- Framework.

- Cấu hình môi trường chạy.

Vì vậy, khi lựa chọn mô hình cần cân bằng giữa **accuracy, độ phức tạp và chi phí tính toán**.

3.30. Phân tích kết quả

Các thực nghiệm cho thấy CNN có khả năng học đặc trưng hiệu quả trên dữ liệu hình ảnh.

Đối với MNIST, ngay cả Basic CNN cũng đạt 98,72%, chứng tỏ bài toán tương đối đơn giản đối với kiến trúc CNN. Các mô hình phức tạp hơn chỉ cải thiện thêm một phần nhỏ.

Fashion-MNIST khó hơn do các lớp có nhiều đặc điểm tương đồng. Vì vậy sự khác biệt giữa các kiến trúc trở nên rõ hơn. M6 Adapted AlexNet đạt 91,20%, trong khi Basic CNN đạt 89,46%.

Đối với Diabetes, Residual CNN đạt 86,21%, cao hơn đáng kể so với Basic CNN. Tuy nhiên, Attention CNN chỉ đạt 81,90% dù có số tham số cao hơn. Kết quả này cho thấy việc bổ sung một kỹ thuật phức tạp không đảm bảo kết quả tốt hơn trong mọi trường hợp.

Một kết luận quan trọng là kiến trúc tốt nhất phụ thuộc vào dữ liệu. Không có một mô hình CNN duy nhất luôn đứng đầu trên mọi dataset.

Ngoài ra, accuracy cũng không nên được xem xét riêng lẻ. Việc sử dụng confusion matrix, Precision, Recall và F1-score giúp đánh giá chi tiết hơn khả năng phân loại của mô hình.

3.31. So sánh CNN với Machine Learning truyền thống

CNN có một khác biệt quan trọng so với các kỹ thuật Machine Learning được trình bày ở Chương 2. Trong Machine Learning truyền thống, người phát triển thường cần xây dựng representation hoặc feature engineering trước khi đưa dữ liệu vào mô hình.

Ví dụ: **Ảnh → Trích xuất đặc trưng → Vector → ML model**

Trong CNN, mô hình có thể học representation trực tiếp từ dữ liệu: **Ảnh → CNN → Feature representation → Prediction**

Điều này làm giảm nhu cầu thiết kế feature thủ công đối với nhiều bài toán hình ảnh.

Tuy nhiên, CNN cũng có yêu cầu cao hơn về:

- Dữ liệu.
- Tài nguyên tính toán.
- Thời gian huấn luyện.
- Thiết kế kiến trúc.
- Hyperparameter.

Vì vậy CNN không phải là sự thay thế tuyệt đối cho Machine Learning truyền thống. Hai nhóm phương pháp phù hợp với những loại dữ liệu và mục tiêu khác nhau.

3.33. Cài đặt CNN bằng PyTorch

PyTorch cung cấp các thành phần cần thiết để xây dựng CNN.

Một mô hình CNN có thể được định nghĩa thông qua các layer:

- `Conv2d`.
- `BatchNorm2d`.
- `ReLU`.
- `MaxPool2d`.
- `Linear`.

Đối với dữ liệu ảnh, quá trình huấn luyện thường gồm:

Dataset → DataLoader → CNN → Loss → Optimizer → Backpropagation → Update

DataLoader chia dữ liệu thành các batch để mô hình xử lý từng phần thay vì đưa toàn bộ dataset vào bộ nhớ trong một lần.

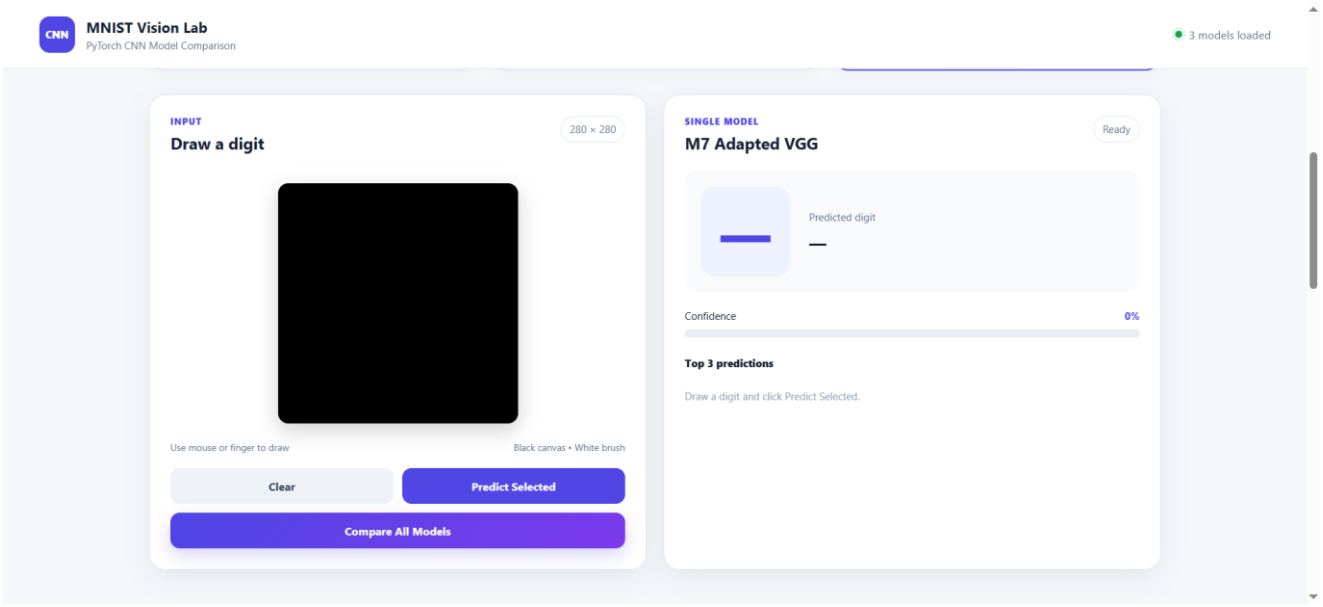
Trong mỗi batch:

- Đưa ảnh lên thiết bị tính toán.
- Forward qua model.
- Tính loss.
- Xóa gradient cũ.
- Backward.
- Optimizer cập nhật trọng số.

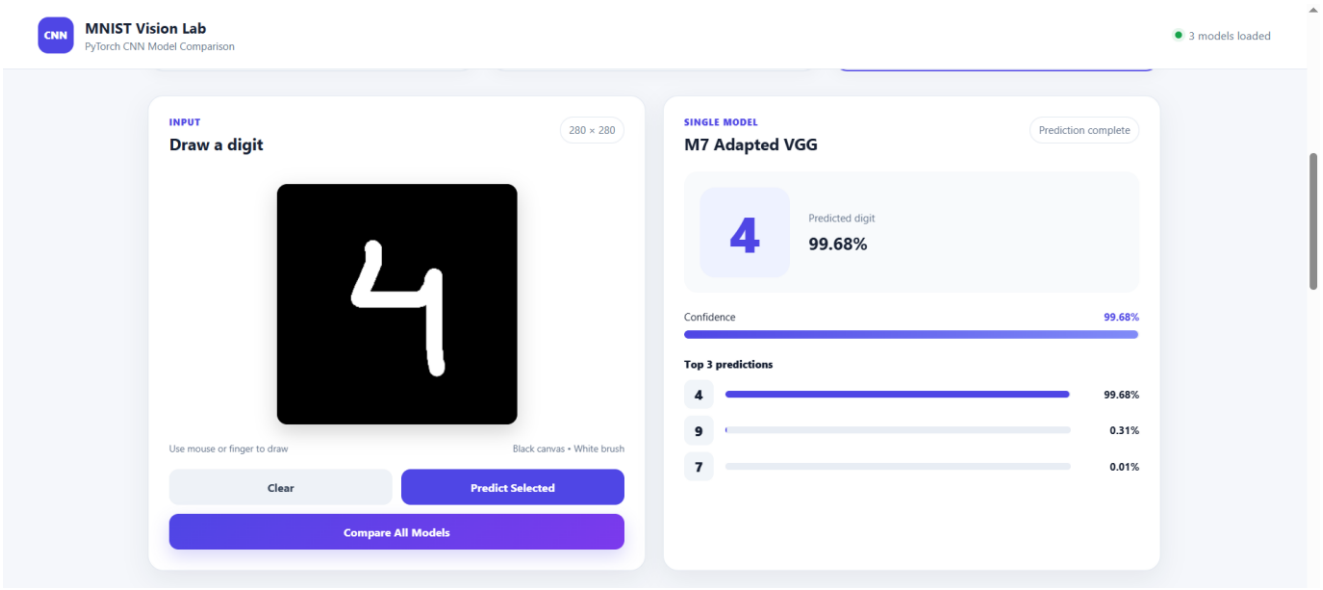
3.34. Demo nhận dạng chữ số bằng CNN

Sau khi huấn luyện, CNN có thể được sử dụng để dự đoán một ảnh chữ số mới.

Quy trình: **Ảnh đầu vào → Resize/Normalize → CNN → Probability → Predicted Class**



Hình 3.9 – Giao diện Web demo nhận dạng chữ số bằng CNN



Hình 3.10 – Kết quả dự đoán chữ số trên Web

3.35. Tiểu kết chương

Chương 3 đã trình bày những kiến thức cơ bản về Convolutional Neural Network, từ cấu trúc dữ liệu đầu vào, phép convolution, kernel, stride, padding đến activation, pooling và fully connected layer. Chương cũng trình bày một số kỹ thuật cải tiến như Batch Normalization, Residual Connection và Attention, cùng với các kiến trúc CNN tiêu biểu như LeNet, AlexNet và VGG.

Ba dataset MNIST, Fashion-MNIST và Diabetes được sử dụng để minh họa khả năng áp dụng CNN trên những dạng dữ liệu khác nhau. Kết quả cho thấy MNIST là bài toán tương đối dễ với CNN, Fashion-MNIST khó hơn và tạo ra sự khác biệt rõ hơn giữa các kiến trúc. Với Diabetes, Residual CNN cho kết quả tốt nhất trong thực nghiệm, nhưng cần thận trọng khi diễn giải do dữ liệu có quy mô nhỏ và việc sử dụng Conv1d chủ yếu mang tính minh họa.

Kết quả thực nghiệm cũng cho thấy việc tăng độ phức tạp của kiến trúc không đảm bảo accuracy luôn tăng. Mô hình cần được lựa chọn dựa trên sự cân bằng giữa performance và chi phí tính toán.

CNN vì vậy có thể được xem là một bước phát triển quan trọng từ Machine Learning truyền thống sang Deep Learning: thay vì phụ thuộc nhiều vào feature engineering thủ công, mô hình có khả năng tự học các representation nhiều tầng từ dữ liệu.

Trong chương tiếp theo, nội dung sẽ chuyển từ dữ liệu có cấu trúc không gian sang dữ liệu tuần tự, trong đó thông tin ở thời điểm hiện tại có thể phụ thuộc vào các thời điểm trước đó. Đây là cơ sở để nghiên cứu Recurrent Neural Network (RNN)**.**

CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY RNN

4.1. Tổng quan về RNN

Mạng nơ-ron hồi quy (Recurrent Neural Network – RNN) là một nhóm mạng nơ-ron được thiết kế để xử lý dữ liệu tuần tự, trong đó thứ tự của các phần tử có ý nghĩa. Khác với mạng nơ-ron thông thường xử lý từng mẫu tương đối độc lập, RNN duy trì một trạng thái ẩn (hidden state) để lưu lại thông tin từ các bước trước đó.

RNN phù hợp với các bài toán như dự báo chuỗi thời gian, xử lý văn bản, nhận dạng tiếng nói và phân tích các dữ liệu có thứ tự theo thời gian. Trong chương này, RNN được áp dụng cho hai bài toán dự báo chuỗi thời gian gồm doanh thu theo ngày và giá đóng cửa cổ phiếu AAPL.

4.2. Nguyên lý hoạt động của RNN

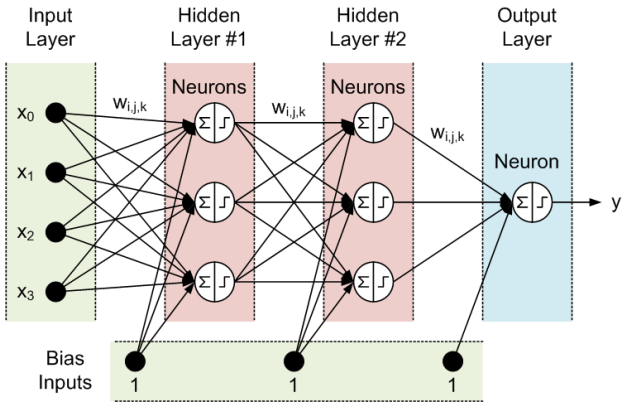
Tại thời điểm t, RNN nhận đầu vào x_t và trạng thái ẩn của bước trước h_(t-1). Trạng thái mới được tính theo:

$$h_t = \tanh(W_x x_t + W_h h_{(t-1)} + b)$$

Trong đó:

- x_t là dữ liệu đầu vào tại timestep t.
- h_(t-1) là hidden state của timestep trước.
- h_t là hidden state mới.
- W_x và W_h là các ma trận trọng số.
- b là bias.
- tanh là hàm kích hoạt.

Nhờ cơ chế này, thông tin từ các timestep trước có thể được truyền sang timestep tiếp theo. Với bài toán many-to-one, toàn bộ chuỗi đầu vào được đưa qua RNN và hidden state cuối cùng được sử dụng để tạo ra một giá trị dự đoán.



Hình 4.1. Sơ đồ hoạt động của RNN

Trong thực nghiệm của chương này, mô hình sử dụng năm lớp RNN xếp chồng. Công thức tại lớp thứ l được biểu diễn như sau:

$$h_t^{(l)} = \tanh\left(W_x^{(l)} h_t^{(l-1)} + W_h^{(l)} h_{t-1}^{(l)} + b^{(l)}\right)$$

Với lớp đầu tiên, đầu vào là đặc trưng dữ liệu x_t; với các lớp tiếp theo, đầu vào là trạng thái đầu ra của lớp trước tại cùng timestep. Sau khi xử lý toàn bộ chuỗi, trạng thái đầu ra tại timestep cuối của lớp RNN thứ năm được đưa vào lớp Linear để tạo giá trị dự báo:

$$\hat{y} = W_o h_T^{(5)} + b_o$$

Trong đó h_T là đầu ra tại timestep cuối của lớp RNN thứ năm, còn W_o và b_o là trọng số và bias của lớp Linear.

4.2.1. Đặc điểm của dữ liệu tuần tự

Dữ liệu tuần tự là dữ liệu trong đó các phần tử có mối quan hệ về thứ tự hoặc thời gian. Khác với dữ liệu dạng bảng thông thường, việc thay đổi thứ tự các phần tử trong chuỗi có thể làm thay đổi ý nghĩa của dữ liệu.

Ví dụ, trong dữ liệu doanh thu theo ngày, doanh thu của ngày hôm nay có thể liên quan đến doanh thu của những ngày trước đó. Tương tự, trong dữ liệu giá cổ phiếu, giá của một phiên giao dịch có thể chịu ảnh hưởng từ xu hướng của các phiên trước.

Một số ví dụ phổ biến của dữ liệu tuần tự gồm:

- Dữ liệu giá cổ phiếu theo thời gian.
- Doanh thu hoặc số lượng giao dịch theo ngày.
- Chuỗi văn bản và câu.
- Dữ liệu cảm biến IoT.
- Tín hiệu âm thanh.
- Dữ liệu thời tiết theo thời gian.

Đặc điểm quan trọng của dữ liệu tuần tự là thông tin trước đó có thể ảnh hưởng đến việc dự đoán hoặc phân tích thông tin hiện tại. Đây là lý do RNN được thiết kế với cơ chế duy trì hidden state để truyền thông tin giữa các timestep.

4.2.2. Timestep và Sequence

Trong RNN, một chuỗi dữ liệu được chia thành nhiều timestep. Mỗi timestep tương ứng với một thời điểm hoặc một phần tử trong chuỗi.

Ví dụ, nếu sử dụng 20 ngày gần nhất để dự đoán ngày tiếp theo thì một mẫu dữ liệu có thể được biểu diễn: $\mathbf{x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow ... \rightarrow x_{20} \rightarrow}$ **dự đoán** $\mathbf{x_{21}}$

Trong đó:

- x_1 đến x_{20} là 20 timestep đầu vào.
- x_{21} là giá trị cần dự đoán.

Toàn bộ 20 timestep tạo thành một sequence.

Trong thực nghiệm của chương này, sequence length được đặt bằng 20. Vì vậy, mỗi lần mô hình nhận vào 20 bước thời gian liên tiếp để dự đoán giá trị ở bước tiếp theo.

4.2.3. Hidden State và bộ nhớ của RNN

Hidden state là thành phần quan trọng giúp RNN lưu lại thông tin từ các timestep trước. Sau khi xử lý một timestep, RNN tạo ra hidden state mới và truyền nó sang timestep tiếp theo.

Có thể hình dung quá trình như sau:

$$\mathbf{x_1 \rightarrow h_1 \rightarrow x_2 \rightarrow h_2 \rightarrow x_3 \rightarrow h_3 \rightarrow ... \rightarrow x_t \rightarrow h_t}$$

Trong đó h_t chứa thông tin được tổng hợp từ dữ liệu hiện tại và một phần thông tin đã được truyền từ các timestep trước.

Nhờ cơ chế này, RNN có khả năng sử dụng ngữ cảnh của chuỗi thay vì chỉ dựa vào một điểm dữ liệu riêng lẻ. Tuy nhiên, khi chuỗi quá dài, thông tin từ những timestep xa có thể khó được duy trì hiệu quả. Đây là một trong những hạn chế quan trọng của RNN truyền thống.

4.2.4. Hiện tượng Vanishing Gradient và Exploding Gradient

Trong quá trình huấn luyện RNN, gradient được truyền ngược qua nhiều timestep. Khi chuỗi dài, gradient có thể gặp hai vấn đề chính là vanishing gradient và exploding gradient.

Vanishing Gradient: Vanishing gradient xảy ra khi gradient ngày càng nhỏ trong quá trình lan truyền ngược qua nhiều timestep. Khi đó, các trọng số ở những timestep xa khó được cập nhật hiệu quả và mô hình khó học được các mối quan hệ dài hạn.

Exploding Gradient: Ngược lại, exploding gradient xảy ra khi gradient tăng quá lớn, khiến quá trình cập nhật trọng số trở nên không ổn định và có thể làm mô hình khó hội tụ.

Hai hiện tượng này là những hạn chế quan trọng của RNN truyền thống khi xử lý chuỗi dài. Đây cũng là lý do các kiến trúc nâng cao như LSTM và GRU được phát triển để cải thiện khả năng ghi nhớ thông tin dài hạn.

4.2.5. Các dạng bài toán RNN

Tùy vào cấu trúc đầu vào và đầu ra, RNN có thể được sử dụng cho nhiều dạng bài toán khác nhau.

Many-to-One: Nhiều timestep đầu vào được sử dụng để tạo ra một đầu ra.

$$x_1, x_2, ..., x_t \rightarrow y$$

Đây là dạng được sử dụng trong thực nghiệm của chương này, trong đó 20 timestep được sử dụng để dự đoán một giá trị ở timestep tiếp theo**.**

Many-to-Many: Mỗi timestep đầu vào có thể tương ứng với một đầu ra:

$$x_1 \rightarrow y_1, x_2 \rightarrow y_2, ..., x_t \rightarrow y_t$$

Dạng này thường phù hợp với các bài toán cần tạo đầu ra tương ứng với từng bước trong chuỗi.

One-to-Many: Một đầu vào được sử dụng để tạo ra một chuỗi đầu ra. Cấu trúc này có thể được sử dụng trong một số bài toán sinh chuỗi.

Trong phạm vi đề tài, dạng Many-to-One được lựa chọn vì phù hợp với bài toán dự báo giá trị tiếp theo của chuỗi thời gian.

4.3. Dữ liệu chuỗi thời gian

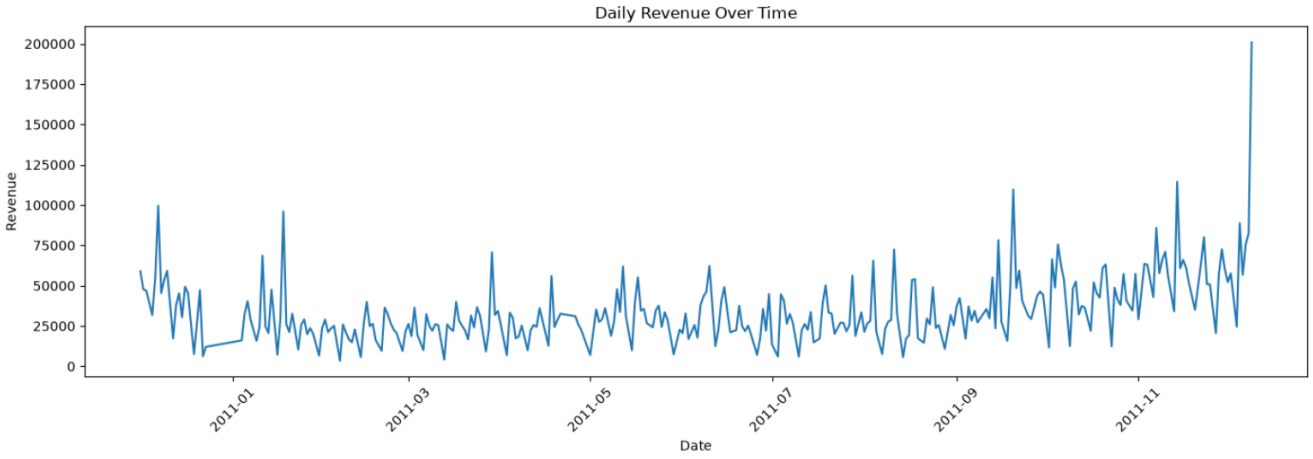
Trong thực nghiệm, hai bộ dữ liệu được sử dụng để đánh giá khả năng dự báo của RNN.

4.3.1. Customer Behavior

Dữ liệu ban đầu là Online Retail, gồm 541.909 dòng và 8 thuộc tính. Sau khi làm sạch dữ liệu và tổng hợp theo ngày, dữ liệu được biểu diễn bằng các đặc trưng:

- Revenue
- Transactions
- Customers
- Quantity
- AvgUnitPrice

Trong đó Revenue được chọn làm giá trị mục tiêu cần dự báo. Sau quá trình tổng hợp, chuỗi có 305 mốc thời gian.



Hình 4.2. Doanh thu hằng ngày của Customer Behavior

4.3.2. AAPL Stock

Dataset thứ hai được lấy từ dữ liệu cổ phiếu S&P 500 và lọc riêng mã AAPL. Dữ liệu gồm 1.259 phiên giao dịch trong giai đoạn 2013–2018.

Các đặc trưng đầu vào gồm:

- Open
- High
- Low
- Close
- Volume

Trong đó Close được chọn làm giá trị mục tiêu dự báo.



Hình 4.3. Giá đóng cửa AAPL theo thời gian

4.4. Chuẩn bị dữ liệu cho RNN

Do RNN yêu cầu dữ liệu có cấu trúc tuần tự, dữ liệu ban đầu được chuyển thành các cửa sổ trượt (sliding window). Mỗi mẫu đầu vào gồm 20 timestep liên tiếp và được sử dụng để dự đoán giá trị tại timestep tiếp theo.

Dữ liệu được chia theo thứ tự thời gian:

- 70% cho tập train.
- 15% cho tập validation.
- 15% cho tập test.

Việc chia theo thời gian giúp tránh sử dụng thông tin tương lai để huấn luyện mô hình. Sau khi chia dữ liệu, scaler chỉ được fit trên tập train rồi sử dụng để biến đổi validation và test.

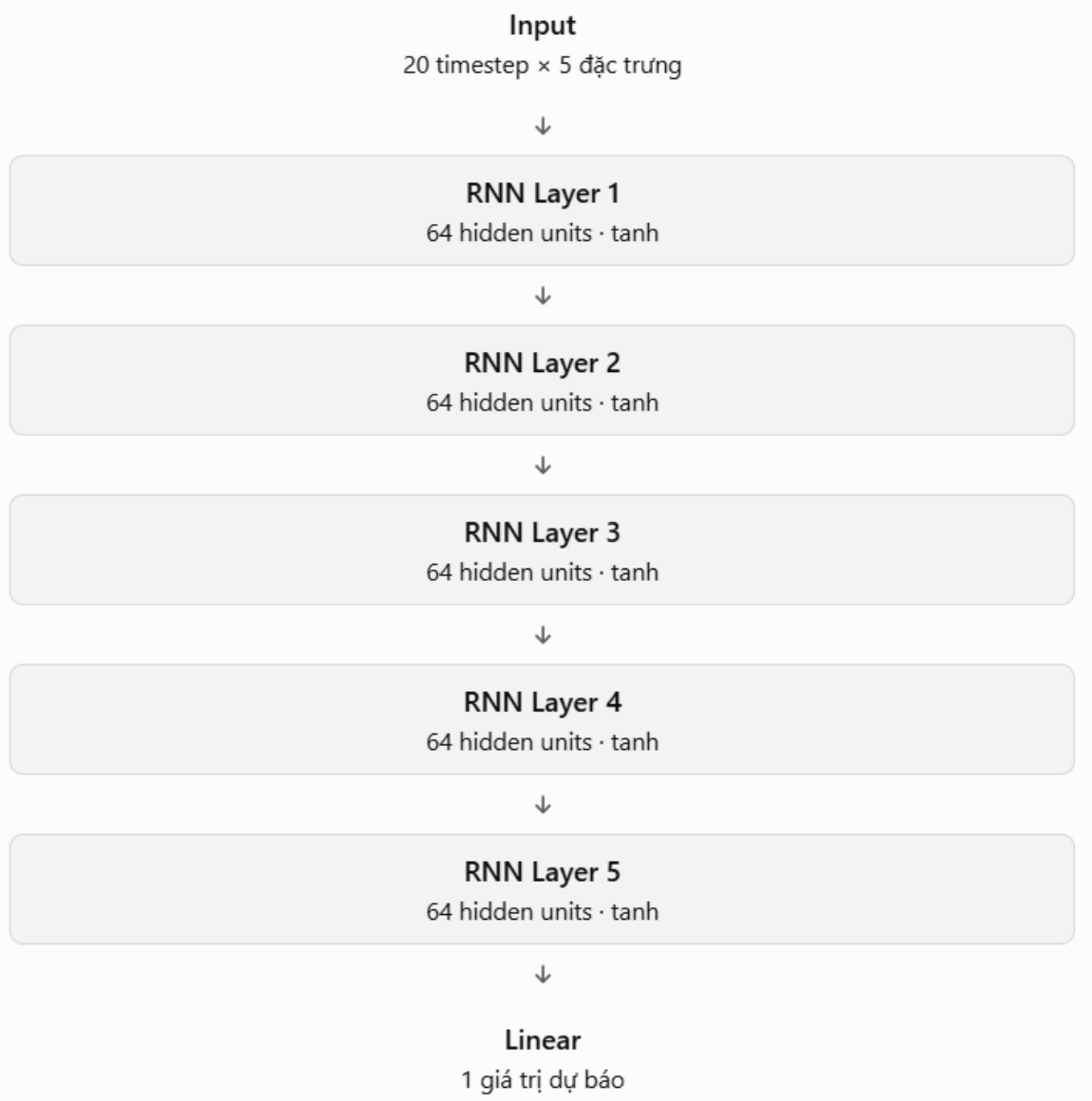
Với Customer Behavior, dữ liệu sau chia có 213 mốc train, 46 mốc validation và 46 mốc test; sau khi tạo sliding window, số sequence tương ứng là 193, 26 và 26. Với AAPL, các tập có 881, 189 và 189 mốc, tạo thành 861, 169 và 169 sequence.

Thành phần	Giá trị
Batch size	16
Sequence length	20
Input size	5
Hidden size	64
Output size	1
Tỷ lệ train/validation/test	70% / 15% / 15%
Số epoch	50

4.5. Thực nghiệm với PyTorch

Mô hình PyTorch sử dụng lớp `nn.RNN` với 64 hidden units, kết hợp với một lớp Linear để tạo ra giá trị dự báo. Mô hình được huấn luyện bằng optimizer Adam và hàm mất mát MSE.

Cấu trúc tổng quát:



Hình 4.4. Cấu trúc tổng quát

Cấu hình huấn luyện:

Các thông số chính của mô hình PyTorch được trình bày trong bảng sau.

Thông số	**Giá trị**
Số timestep đầu vào	20
Số đặc trưng đầu vào	5
Số lớp RNN	5
Số hidden units mỗi lớp	64

Hàm kích hoạt	tanh
Lớp đầu ra	Linear (1 đơn vị)
Hàm mất mát	MSELoss
Bộ tối ưu	Adam
Learning rate	0,001
Số epoch	50
Batch size	16

Trong quá trình huấn luyện, mô hình tạo ra dự đoán từ dữ liệu đầu vào và so sánh với giá trị thực tế thông qua hàm mất mát MSE. Bộ tối ưu Adam sử dụng gradient để cập nhật các tham số của mô hình. Training loss và validation loss được ghi nhận qua các epoch nhằm theo dõi quá trình hội tụ và phát hiện dấu hiệu quá khớp.

Vì kiến trúc đã thay đổi từ một lớp RNN sang năm lớp RNN, mô hình cần được khởi tạo và huấn luyện lại. Các trọng số của mô hình Basic RNN cũ không thể được sử dụng trực tiếp cho kiến trúc mới.

Mã nguồn minh họa kiến trúc PyTorch 5 lớp:

```
import torch
import torch.nn as nn


class DeepRNN(nn.Module):
    def __init__(
        self,
        input_size=5,
        hidden_size=64,
        output_size=1,
        num_layers=5
    ):
        super().__init__()

        self.rnn_layers = nn.ModuleList()
```

```
        for i in range(num_layers):
            layer_input_size = (
                input_size if i == 0 else hidden_size
            )

            self.rnn_layers.append(
                nn.RNN(
                    input_size=layer_input_size,
                    hidden_size=hidden_size,
                    nonlinearity="tanh",
                    batch_first=True
                )
            )

        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        output = x

        for rnn_layer in self.rnn_layers:
            output, _ = rnn_layer(output)

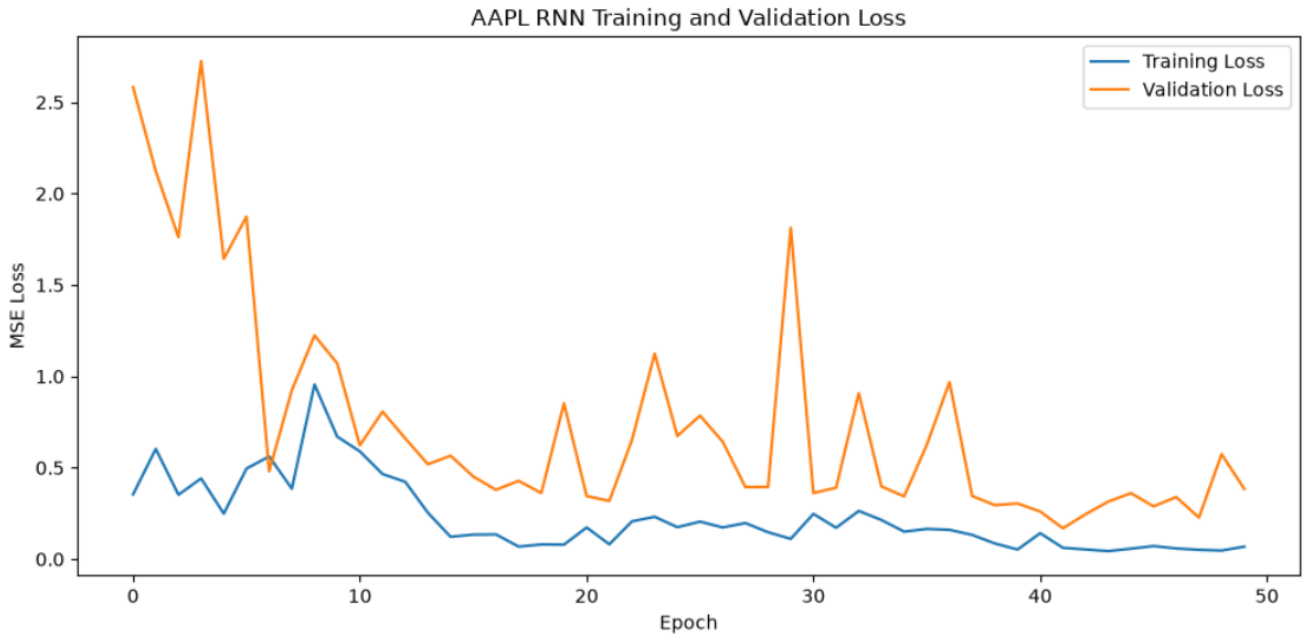
        last_output = output[:, -1, :]
        prediction = self.fc(last_output)

        return prediction
```

Hàm khởi tạo `__init__()` tạo năm lớp RNN và một lớp Linear. Lớp đầu tiên nhận năm đặc trưng đầu vào; các lớp tiếp theo nhận đầu ra có 64 đơn vị ẩn từ lớp trước.

Hàm `forward()` lần lượt truyền dữ liệu qua năm lớp RNN, lấy đầu ra tại timestep cuối rồi đưa qua lớp Linear. Kết quả là một giá trị dự báo cho mỗi mẫu đầu vào.

Đoạn mã trên minh họa cấu trúc mô hình. Trong notebook thực nghiệm, mô hình còn được kết hợp với các hàm chuẩn bị dữ liệu, huấn luyện, đánh giá và tính toán chỉ số.



Hình 4.5. Loss train và validation của RNN PyTorch

4.6. Thực nghiệm với Keras

Để có cơ sở đối chiếu giữa hai framework, mô hình RNN cũng được triển khai bằng Keras. Kiến trúc sử dụng năm lớp `SimpleRNN(64)` xếp chồng, trong đó bốn lớp đầu trả về toàn bộ chuỗi đầu ra để làm đầu vào cho lớp kế tiếp. Lớp thứ năm chỉ cần đầu ra cuối cùng để đưa vào `Dense(1)`.

Cấu trúc tổng quát:

$$
\begin{aligned}
X &\rightarrow \text{SimpleRNN}_1(64) \\
&\rightarrow \text{SimpleRNN}_2(64) \\
&\rightarrow \text{SimpleRNN}_3(64) \\
&\rightarrow \text{SimpleRNN}_4(64) \\
&\rightarrow \text{SimpleRNN}_5(64) \\
&\rightarrow \text{Dense}(1) \rightarrow \hat{y}
\end{aligned}
$$

Mã nguồn minh họa xây dựng mô hình Keras 5 lớp:

```
from tensorflow import keras
from tensorflow.keras import layers
```

```
keras_rnn = keras.Sequential([
    layers.Input(shape=(20, 5)),

    layers.SimpleRNN(
        64,
        activation="tanh",
        return_sequences=True
    ),
    layers.SimpleRNN(
        64,
        activation="tanh",
        return_sequences=True
    ),
    layers.SimpleRNN(
        64,
        activation="tanh",
        return_sequences=True
    ),
    layers.SimpleRNN(
        64,
        activation="tanh",
        return_sequences=True
    ),
    layers.SimpleRNN(
        64,
        activation="tanh"
    ),

    layers.Dense(1)
])

keras_rnn.compile(
    optimizer=keras.optimizers.Adam(
```

```
        learning_rate=0.001
    ),
    loss="mse"
)

keras_rnn.summary()
```

Trong cấu trúc trên, `return_sequences=True` được sử dụng ở bốn lớp đầu nhằm giữ lại đầu ra tại tất cả các timestep. Nhờ đó, lớp RNN kế tiếp tiếp tục nhận được chuỗi đầu ra thay vì chỉ một vector đại diện cho timestep cuối.

Lớp RNN thứ năm không cần trả về toàn bộ chuỗi vì lớp `Dense(1)` chỉ sử dụng biểu diễn cuối cùng để tạo giá trị dự báo. Hàm mất mát MSE và bộ tối ưu Adam được sử dụng để huấn luyện mô hình.

Việc triển khai kiến trúc tương đương trên PyTorch và Keras giúp so sánh kết quả giữa hai framework trong cùng bài toán. Tuy nhiên, kết quả thực nghiệm vẫn có thể khác nhau do cách cài đặt, khởi tạo trọng số và các chi tiết trong quá trình huấn luyện.

4.7. Kết quả dự báo

Các mô hình được đánh giá bằng bốn chỉ số hồi quy gồm:

- **MAE:** sai số tuyệt đối trung bình, càng thấp càng tốt.
- **MSE:** sai số bình phương trung bình, càng thấp càng tốt.
- **RMSE:** căn bậc hai của MSE, càng thấp càng tốt.
- **R^2:** mức độ giải thích biến thiên của dữ liệu; giá trị càng cao càng tốt.

Dataset	Framework	MAE	MSE	RMSE	R^2
Customer Behavior	PyTorch	21787.9082	1.142320e+09	33798.2267	-0.4377
Customer Behavior	Keras	24249.6875	1.279801e+09	35774.3005	-0.6107
AAPL Stock	PyTorch	45.9173	2.214458e+03	47.0580	-19.8783
AAPL Stock	Keras	34.9007	1.324138e+03	36.3887	-11.4842

Kết quả cho thấy **PyTorch đạt RMSE thấp hơn Keras trên dataset Customer Behavior nhưng lại cao hơn trên dataset AAPL Stock**. Với Customer Behavior, RMSE giảm từ 35774.3005 xuống 33798.2267. Với AAPL, RMSE tăng từ 36.3887 lên 47.0580.

4.8. Phân tích kết quả trên Customer Behavior

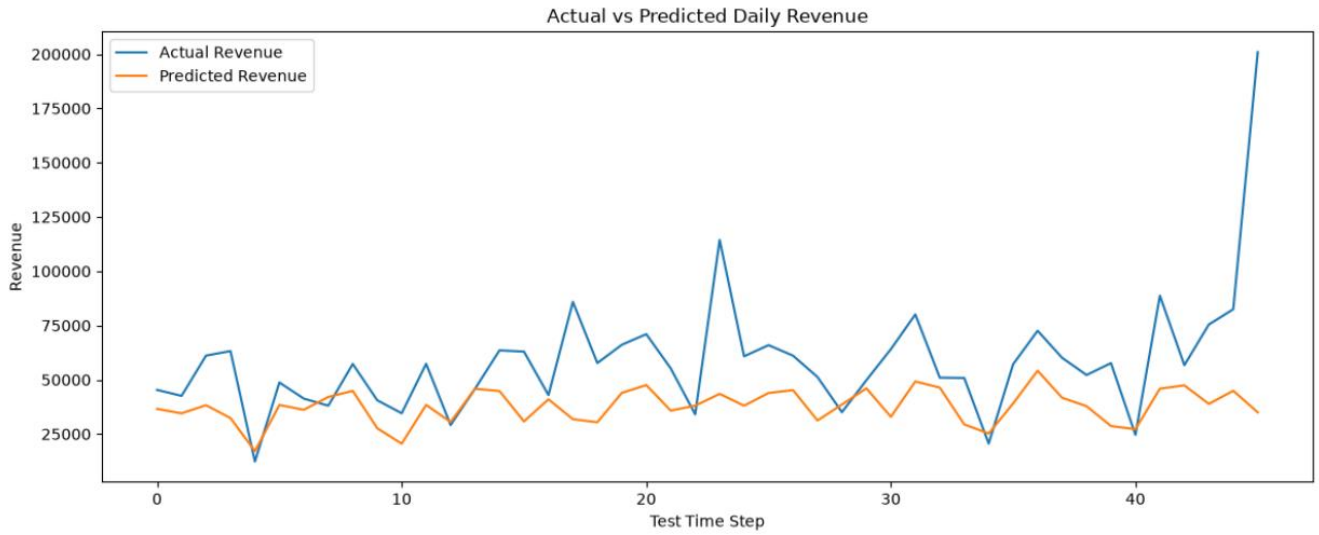
Bộ dữ liệu Customer Behavior được xây dựng từ dữ liệu giao dịch bán lẻ Online Retail. Dữ liệu được tổng hợp theo ngày, bao gồm năm đặc trưng đầu vào: Revenue, Transactions, Customers, Quantity và AvgUnitPrice. Mô hình sử dụng 20 quan sát liên tiếp để dự đoán doanh thu của quan sát tiếp theo.

Trong lần thực nghiệm cập nhật với kiến trúc RNN 5 lớp, mô hình PyTorch đạt MAE là 21.787,91, RMSE là 33.798,23 và R^2 là -0,4377. Mô hình Keras đạt MAE là 24.249,69, RMSE là 35.774,30 và R^2 là -0,6107. Như vậy, PyTorch có sai số MAE, MSE và RMSE thấp hơn Keras trong lần thực nghiệm này.

Khi đối chiếu với kết quả mô hình RNN 1 lớp trước đó, sai số MAE và RMSE của cả hai framework đều giảm trong kết quả cập nhật. Tuy nhiên, R^2 của PyTorch thay đổi từ -0,4048 xuống -0,4377, còn R^2 của Keras thay đổi từ -0,8278 xuống -0,6107. Điều này cho thấy sự thay đổi về sai số không đồng nghĩa với việc tất cả các chỉ số đều cải thiện.

Cả hai mô hình 5 lớp vẫn có R^2 âm, cho thấy khả năng dự đoán chưa tốt so với phương án sử dụng giá trị trung bình của tập kiểm tra làm dự đoán. Nguyên nhân có thể liên quan đến biến động doanh thu giữa các ngày, số lượng chuỗi huấn luyện hạn chế sau khi tổng hợp dữ liệu và việc chưa đưa các yếu tố như mùa vụ, ngày lễ, khuyến mại hoặc đặc điểm sản phẩm vào mô hình.

Nhìn chung, kết quả cho thấy mô hình RNN có thể học mối quan hệ giữa các quan sát theo thời gian, nhưng việc dự đoán doanh thu bán lẻ vẫn còn nhiều hạn chế. Cần tiếp tục kiểm tra dữ liệu đầu vào, cách chia tập dữ liệu và cấu hình huấn luyện để đánh giá chính xác hơn hiệu quả của mô hình.



Hình 4.6. Giá trị doanh thu thực tế và giá trị dự đoán bằng RNN

4.9. Phân tích kết quả trên AAPL

Bộ dữ liệu AAPL Stock được sử dụng để dự đoán giá đóng cửa cổ phiếu Apple. Năm đặc trưng đầu vào gồm Open, High, Low, Close và Volume. Tương tự bài toán Customer Behavior, mô hình sử dụng cửa sổ 20 phiên liên tiếp để dự đoán giá đóng cửa của phiên tiếp theo.

Trong kết quả cập nhật của mô hình RNN 5 lớp, PyTorch đạt MAE là 45,92, RMSE là 47,06 và R^2 là -19,8783. Trong khi đó, Keras đạt MAE là 34,90, RMSE là 36,39 và R^2 là -11,4842. Keras có kết quả tốt hơn PyTorch theo cả bốn chỉ số MAE, MSE, RMSE và R^2 trong lần thực nghiệm này.

So với kết quả RNN 1 lớp trước đó, cả hai mô hình 5 lớp đều có sai số lớn hơn đáng kể. MAE của PyTorch tăng từ 10,3069 lên 45,9173, còn MAE của Keras tăng từ 12,0690 lên 34,9007. R^2 của cả hai mô hình cũng giảm mạnh, cho thấy kết quả dự đoán trong lần cập nhật chưa đạt hiệu quả như lần thực nghiệm trước.

Các giá trị R^2 âm sâu cho thấy mô hình chưa giải thích tốt biến động của giá đóng cửa trên tập kiểm tra. Tuy nhiên, không thể chỉ dựa vào số lớp RNN để xác định nguyên nhân. Kết quả còn có thể chịu ảnh hưởng của cách chia dữ liệu, quá trình chuẩn hóa, số epoch, tham số huấn luyện và tính biến động của chuỗi giá cổ phiếu.

Ngoài năm đặc trưng được sử dụng, giá cổ phiếu còn chịu ảnh hưởng từ thông tin thị trường, tin tức, kết quả kinh doanh và tâm lý nhà đầu tư. Những yếu tố này chưa được thể hiện đầy đủ trong dữ liệu đầu vào của mô hình. Do đó, kết quả thực nghiệm cho thấy việc dự đoán giá cổ

phiếu bằng RNN vẫn còn hạn chế và cần được đánh giá thêm trên các cấu hình dữ liệu nhất quán.



Hình 4.7. Giá AAPL thực tế và giá dự đoán bằng RNN

4.10. So sánh PyTorch và Keras

Để đánh giá sự khác biệt giữa các kiến trúc, kết quả của mô hình RNN 1 lớp ở lần thực nghiệm trước được đối chiếu với kết quả RNN 5 lớp sau khi cập nhật notebook. Các mô hình được triển khai bằng hai framework là PyTorch và Keras.

Dataset	Kiến trúc	Framework	MAE	MSE	RMSE	R^2
Customer Behavior	1 lớp	PyTorch	23.687,0410	1.566.219.043,4881	39.575,4854	-0,4048
Customer Behavior	1 lớp	Keras	30.770,8299	2.037.844.288,3649	45.142,4887	-0,8278
Customer Behavior	5 lớp	PyTorch	21.787,9082	1.142.320.000	33.798,2267	-0,4377
Customer Behavior	5 lớp	Keras	24.249,6875	1.279.801.000	35.774,3005	-0,6107
AAPL Stock	1 lớp	PyTorch	10,3069	120,2314	10,9650	-0,1336
AAPL	1 lớp	Keras	12,0690	172,7596	13,1438	-0,6288

Stock						
AAPL Stock	5 lớp	PyTorch	45,9173	2.214,4580	47,0580	-19,8783
AAPL Stock	5 lớp	Keras	34,9007	1.324,1380	36,3887	-11,4842

Đối với Customer Behavior, mô hình 5 lớp có MAE và RMSE thấp hơn mô hình 1 lớp ở cả PyTorch và Keras. Tuy nhiên, R^2 của PyTorch 5 lớp thấp hơn một chút so với PyTorch 1 lớp. Với Keras, R^2 của mô hình 5 lớp cao hơn mô hình 1 lớp nhưng vẫn âm. Do đó, chưa thể khẳng định kiến trúc 5 lớp cải thiện toàn diện chất lượng dự đoán trên bộ dữ liệu này.

Đối với AAPL Stock, mô hình 1 lớp có kết quả tốt hơn mô hình 5 lớp ở cả hai framework theo các chỉ số được trình bày. Kết quả này cho thấy việc tăng số lớp hồi quy không tự động giúp mô hình dự đoán tốt hơn. Với dữ liệu có số lượng quan sát và đặc điểm biến động nhất định, kiến trúc phức tạp hơn có thể khó tối ưu hoặc không phù hợp với cấu hình huấn luyện hiện tại.

Xét riêng trong lần thực nghiệm RNN 5 lớp, PyTorch có kết quả tốt hơn Keras trên Customer Behavior, trong khi Keras có kết quả tốt hơn PyTorch trên AAPL Stock. Điều này cho thấy hiệu quả của framework phụ thuộc vào bài toán và cấu hình cụ thể, không thể kết luận một framework luôn vượt trội trong mọi trường hợp.

Cần lưu ý rằng các chỉ số trong bảng được lấy từ hai lần thực nghiệm khác nhau. Nếu cách chia dữ liệu hoặc các mẫu kiểm tra giữa hai lần chạy không hoàn toàn giống nhau, sự chênh lệch chỉ có ý nghĩa tham khảo. Để kết luận chính xác về ảnh hưởng của số lớp RNN, cần đánh giá mô hình 1 lớp và 5 lớp trên cùng tập kiểm tra, cùng quy trình tiền xử lý và cùng cách tính chỉ số.

4.11. Hạn chế và hướng phát triển

Qua kết quả thực nghiệm, các mô hình RNN vẫn tồn tại một số hạn chế.

Thứ nhất, khả năng dự đoán chưa ổn định giữa các bộ dữ liệu. Trên Customer Behavior, sai số của mô hình 5 lớp giảm so với kết quả 1 lớp trước đó ở một số chỉ số, nhưng R^2 vẫn âm. Trên AAPL Stock, mô hình 5 lớp có sai số cao hơn đáng kể trong lần thực nghiệm cập nhật.

Thứ hai, số lượng chuỗi dữ liệu huấn luyện của Customer Behavior tương đối hạn chế sau khi tổng hợp giao dịch theo ngày và tạo cửa sổ 20 quan sát. Trong khi đó, dữ liệu AAPL chịu ảnh hưởng từ nhiều yếu tố thị trường chưa được đưa vào mô hình. Những đặc điểm này làm tăng độ khó của bài toán dự đoán.

Thứ ba, kết quả phụ thuộc vào quy trình chuẩn hóa, cách tạo chuỗi, cách chia dữ liệu và các tham số như số lớp, số neuron ẩn, learning rate và số epoch. Nếu các yếu tố này khác nhau giữa những lần thực nghiệm, việc so sánh trực tiếp các chỉ số có thể chưa phản ánh riêng tác động của kiến trúc.

Trong các nghiên cứu tiếp theo, có thể xem xét một số hướng phát triển:

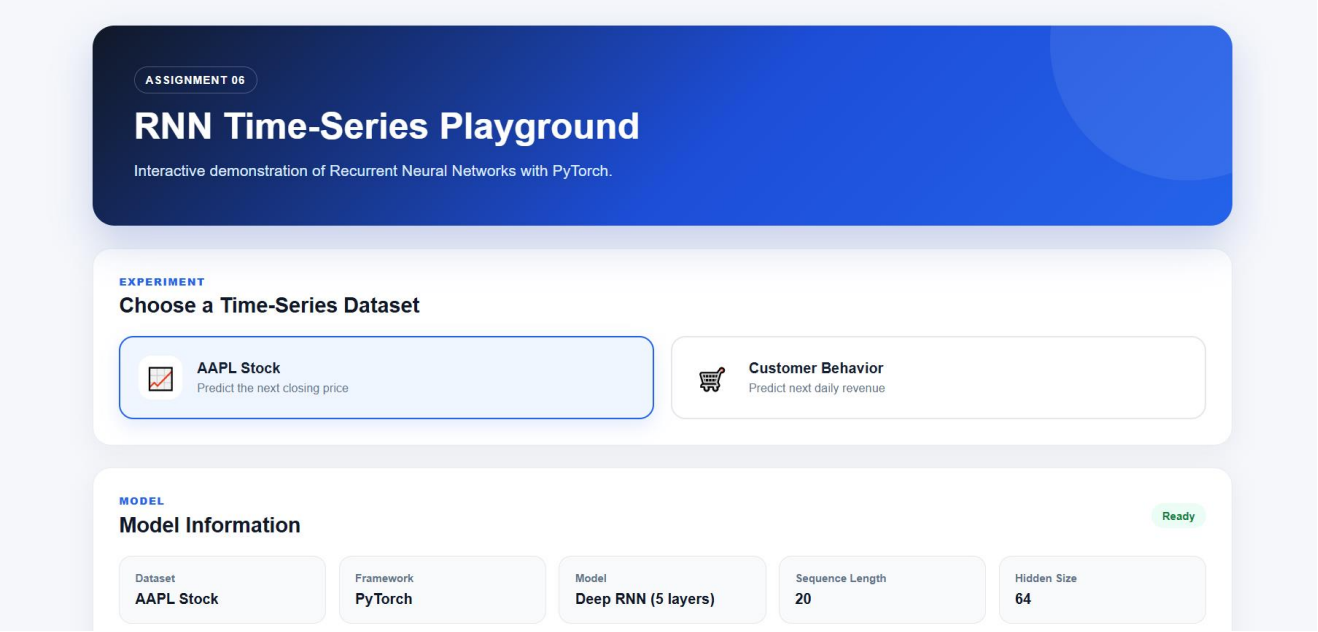
- Chuẩn hóa dữ liệu dựa trên tập huấn luyện khi đánh giá mô hình, đồng thời bảo đảm các mô hình được so sánh trên cùng tập kiểm tra.
- Điều chỉnh số neuron ẩn, learning rate, số epoch và độ dài cửa sổ đầu vào để tìm cấu hình phù hợp với từng bộ dữ liệu.
- Bổ sung đặc trưng thời gian và các yếu tố liên quan đến doanh thu bán lẻ hoặc thị trường chứng khoán nếu có dữ liệu đáng tin cậy.
- Thử nghiệm các kiến trúc LSTM hoặc GRU để đánh giá khả năng ghi nhớ quan hệ dài hạn trong chuỗi.
- Đối chiếu kết quả với các phương pháp dự đoán cơ sở, chẳng hạn dự đoán doanh thu tiếp theo bằng doanh thu quan sát gần nhất.

Các hướng phát triển này cần được kiểm chứng bằng thực nghiệm thay vì giả định trước rằng mô hình phức tạp hơn sẽ cho kết quả tốt hơn.

4.12. Demo web

Để minh họa khả năng ứng dụng của mô hình RNN, hệ thống được triển khai trên nền tảng Flask, cho phép người dùng thực hiện dự đoán thông qua giao diện web thay vì phải thao tác trực tiếp trong Jupyter Notebook.

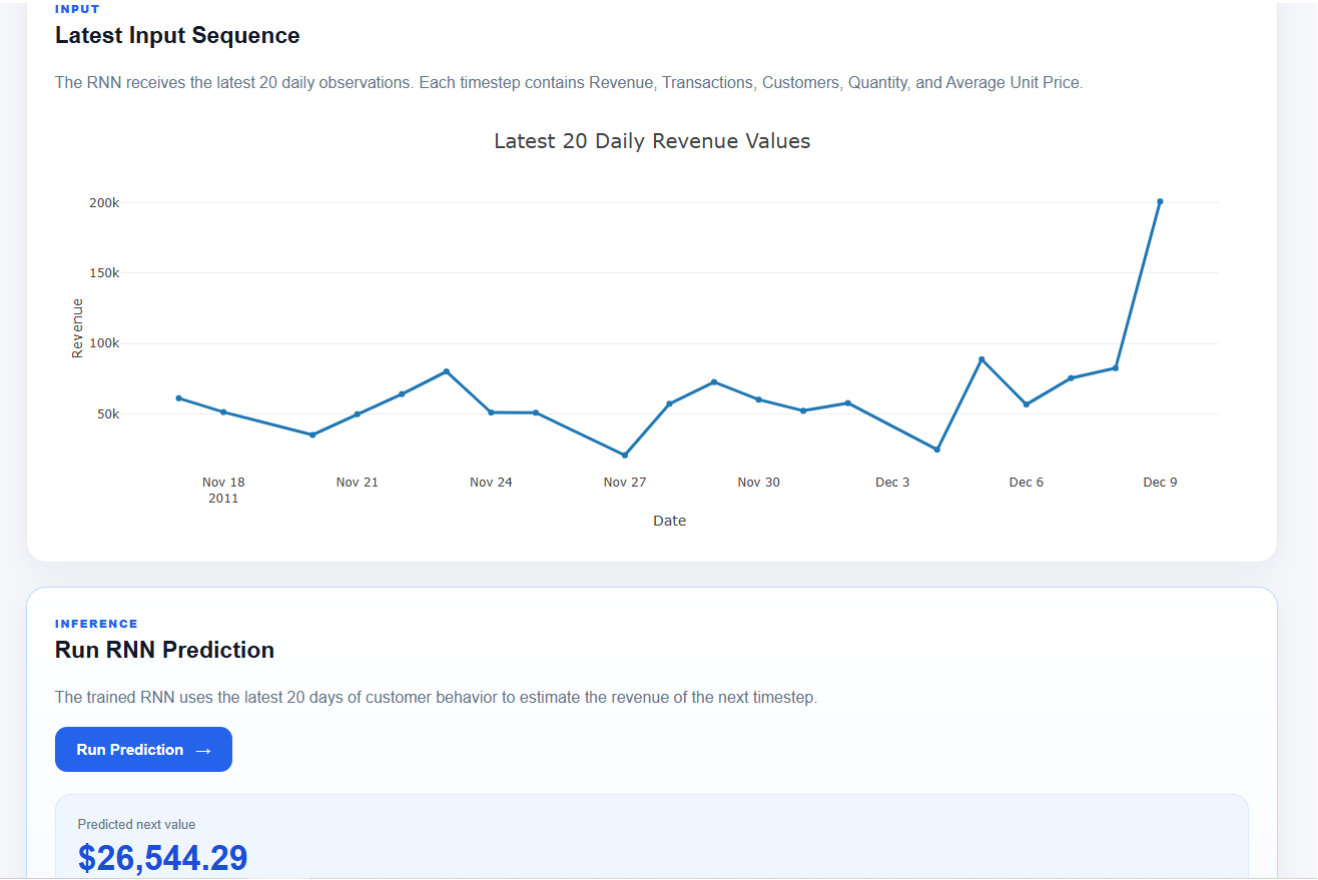
Ứng dụng hỗ trợ hai bài toán gồm dự đoán giá đóng cửa cổ phiếu AAPL và dự đoán doanh thu tiếp theo trên bộ dữ liệu Customer Behavior. Các mô hình PyTorch được lưu thành tệp trọng số; bộ chuẩn hóa và metadata được lưu riêng để phục vụ quá trình nạp mô hình và xử lý dữ liệu đầu vào.



Hình 4.8. Giao diện web dự đoán AAPL Stock và Customer Behavior



Hình 4.9. Kết quả dự đoán với AAPL Stock



Hình 4.10. Kết quả dự đoán Revenue

4.13. Tiểu kết chương

Chương 4 đã trình bày cơ sở hoạt động của mạng nơ-ron hồi quy RNN và quá trình áp dụng mô hình vào hai bài toán chuỗi thời gian: dự đoán doanh thu Customer Behavior và dự đoán giá đóng cửa cổ phiếu AAPL. Các mô hình được triển khai bằng PyTorch và Keras, sử dụng cửa sổ 20 quan sát liên tiếp cùng năm đặc trưng đầu vào cho mỗi bộ dữ liệu.

Kết quả thực nghiệm cho thấy hiệu quả của mô hình phụ thuộc vào bộ dữ liệu, framework và cấu hình kiến trúc. Trên Customer Behavior, các mô hình 5 lớp có sai số MAE và RMSE thấp hơn kết quả 1 lớp trước đó, nhưng R^2 vẫn âm. Trên AAPL Stock, mô hình 1 lớp cho kết quả tốt hơn mô hình 5 lớp trong các kết quả được đối chiếu. Vì vậy, tăng số lớp không phải lúc nào cũng giúp cải thiện chất lượng dự đoán.

Bên cạnh thực nghiệm trong notebook, chương cũng trình bày định hướng triển khai mô hình thành ứng dụng web Flask để người dùng có thể nhập dữ liệu và quan sát kết quả dự đoán. Kết quả kiểm tra ban đầu của web demo cho thấy vẫn cần rà soát quy trình huấn luyện, chuẩn hóa và dự đoán trước khi đánh giá khả năng ứng dụng thực tế.

Nhìn chung, các kết quả là cơ sở để tiếp tục nghiên cứu cách lựa chọn kiến trúc, cải thiện quy trình thực nghiệm và đánh giá mô hình trên các bộ dữ liệu chuỗi thời gian khác nhau.

CHƯƠNG 5. KNOWLEDGE GRAPH VÀ RETRIEVAL-AUGMENTED GENERATION (RAG)

5.1. Mở đầu

Sự phát triển của trí tuệ nhân tạo (Artificial Intelligence – AI), đặc biệt là các mô hình ngôn ngữ lớn (Large Language Models – LLMs), đã tạo ra nhiều phương pháp mới trong việc xử lý ngôn ngữ tự nhiên, tìm kiếm thông tin và xây dựng hệ thống hỏi đáp thông minh. Các mô hình này có khả năng tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên, tổng hợp nội dung và tạo ra câu trả lời tương đối linh hoạt. Tuy nhiên, việc chỉ sử dụng mô hình ngôn ngữ lớn vẫn tồn tại một số hạn chế, chẳng hạn như kiến thức không được cập nhật liên tục, khó kiểm soát nguồn gốc thông tin, khả năng suy diễn sai và khó biểu diễn rõ ràng các mối quan hệ giữa nhiều thực thể.

Để khắc phục những hạn chế trên, hai hướng tiếp cận đáng chú ý là Knowledge Graph (KG) và Retrieval-Augmented Generation (RAG). Knowledge Graph tập trung vào việc biểu diễn tri thức dưới dạng đồ thị gồm các thực thể và mối quan hệ giữa chúng. Trong khi đó, RAG kết hợp cơ chế truy xuất thông tin từ một kho dữ liệu bên ngoài với khả năng tạo ngôn ngữ của mô hình AI. Hai phương pháp giải quyết những khía cạnh khác nhau của bài toán quản lý và khai thác tri thức, đồng thời có thể được kết hợp để xây dựng hệ thống hỏi đáp có khả năng cung cấp câu trả lời gắn với dữ liệu tham chiếu.

Knowledge Graph đặc biệt phù hợp với những lĩnh vực mà thông tin có cấu trúc quan hệ rõ ràng. Chẳng hạn, trong lĩnh vực y tế, một bệnh có thể liên quan đến nhiều triệu chứng, yếu tố nguy cơ, phương pháp xét nghiệm và phương pháp điều trị. Trong lĩnh vực chứng khoán, một doanh nghiệp có thể liên quan đến mã cổ phiếu, ngành nghề, báo cáo tài chính và các sự kiện kinh doanh. Đối với thị trường vàng, dữ liệu có thể được tổ chức theo loại vàng, thời điểm ghi nhận, thị trường và mức giá mua hoặc bán. Việc biểu diễn các mối quan hệ này dưới dạng đồ thị giúp hệ thống truy vấn và kết nối thông tin theo những cấu trúc xác định.

RAG lại có ưu thế trong việc khai thác các nguồn dữ liệu văn bản có nội dung phong phú, chẳng hạn như bài viết chuyên môn, tài liệu hướng dẫn, báo cáo doanh nghiệp, tin tức và các tài liệu giải thích. Khi người dùng đặt câu hỏi, hệ thống có thể tìm kiếm những đoạn tài liệu liên quan rồi cung cấp chúng làm ngữ cảnh cho mô hình ngôn ngữ lớn. Nhờ đó, câu trả lời có cơ sở tham chiếu rõ ràng hơn so với việc chỉ yêu cầu mô hình tạo nội dung từ kiến thức sẵn có.

Trong phạm vi chương này, Knowledge Graph và RAG được nghiên cứu từ cơ sở lý thuyết, cách tổ chức kho tri thức, cơ chế truy xuất và khả năng ứng dụng trong ba lĩnh vực: bệnh tiểu đường, chứng khoán và giá vàng. Mục tiêu là làm rõ cách xây dựng một hệ thống có khả năng

tiếp nhận câu hỏi, tìm kiếm thông tin phù hợp và tạo ra câu trả lời dựa trên dữ liệu bên ngoài. Trên cơ sở đó, các phần thực nghiệm tiếp theo sẽ đánh giá khả năng hoạt động, chất lượng truy xuất và những hạn chế của hệ thống trong từng lĩnh vực cụ thể.

5.1.1. Knowledge Graph và RAG: lịch sử phát triển

a) Quá trình hình thành và phát triển Knowledge Graph

Ý tưởng biểu diễn tri thức bằng các thực thể và quan hệ đã xuất hiện từ lâu trong lĩnh vực trí tuệ nhân tạo. Trong các hệ thống AI truyền thống, tri thức thường được biểu diễn bằng những cấu trúc như luật suy diễn, logic vị từ, mạng ngữ nghĩa và cơ sở tri thức. Những phương pháp này hướng đến việc mô tả các sự kiện, khái niệm và quan hệ để máy tính có thể lưu trữ, truy vấn hoặc suy luận dựa trên tri thức đã được cung cấp.

Mạng ngữ nghĩa (Semantic Network) là một trong những hình thức biểu diễn tri thức có liên quan trực tiếp đến sự phát triển của Knowledge Graph. Trong mạng ngữ nghĩa, các khái niệm được biểu diễn thành những nút, còn các mối quan hệ giữa chúng được biểu diễn bằng các liên kết. Chẳng hạn, một hệ thống có thể mô tả quan hệ "là một loại của", "có triệu chứng" hoặc "được điều trị bằng". Cách biểu diễn này giúp thông tin không còn chỉ tồn tại dưới dạng các mục riêng lẻ mà được tổ chức thành một mạng lưới có ý nghĩa.

Cùng với sự phát triển của Web và các hệ thống dữ liệu phân tán, nhu cầu liên kết thông tin từ nhiều nguồn khác nhau ngày càng tăng. Các công nghệ dữ liệu liên kết (Linked Data), dữ liệu mở liên kết (Linked Open Data) và các mô hình biểu diễn tri thức trên Web đã góp phần thúc đẩy sự phát triển của đồ thị tri thức. Một trong những hình thức biểu diễn phổ biến là bộ ba RDF (Resource Description Framework), trong đó mỗi phát biểu được mô tả bằng ba thành phần: chủ thể (subject), vị từ hoặc quan hệ (predicate) và đối tượng (object).

Ví dụ, phát biểu "Metformin được sử dụng trong điều trị đái tháo đường type 2" có thể được biểu diễn dưới dạng:

- Chủ thể: Metformin.
- Quan hệ: được sử dụng trong điều trị.
- Đối tượng: Đái tháo đường type 2.

Cách biểu diễn theo bộ ba giúp các phát biểu được tổ chức thống nhất và có thể liên kết với những phát biểu khác. Tuy nhiên, để sử dụng trong hệ thống thực tế, dữ liệu còn cần được chuẩn hóa, xác định định danh của thực thể và xử lý các trường hợp nhiều cách viết cùng chỉ một đối tượng.

Thuật ngữ Knowledge Graph trở nên phổ biến rộng rãi hơn khi các công ty công nghệ lớn phát triển những hệ thống liên kết tri thức ở quy mô lớn. Một ví dụ tiêu biểu là Knowledge

Graph do Google giới thiệu năm 2012, nhằm cải thiện khả năng tìm kiếm bằng cách giúp hệ thống hiểu các thực thể và quan hệ thay vì chỉ xử lý từ khóa đơn thuần. Sau đó, Knowledge Graph được ứng dụng trong nhiều lĩnh vực như tìm kiếm thông tin, thương mại điện tử, chăm sóc sức khỏe, quản lý dữ liệu doanh nghiệp và phân tích quan hệ.

Ngày nay, Knowledge Graph không chỉ là một phương pháp biểu diễn dữ liệu mà còn là một thành phần trong các hệ thống AI hiện đại. Đồ thị tri thức có thể kết hợp với cơ sở dữ liệu quan hệ, hệ thống tìm kiếm văn bản, mô hình học máy và mô hình ngôn ngữ lớn. Sự kết hợp này tạo điều kiện để hệ thống vừa khai thác được quan hệ có cấu trúc, vừa xử lý được dữ liệu phi cấu trúc như văn bản.

b) Quá trình hình thành và phát triển RAG

Trước khi RAG trở thành một hướng tiếp cận phổ biến, nhiều hệ thống hỏi đáp dựa trên AI chủ yếu sử dụng luật được thiết kế thủ công, cơ sở dữ liệu tri thức hoặc mô hình học máy được huấn luyện cho một nhiệm vụ cụ thể. Khi các mô hình ngôn ngữ phát triển, hệ thống có thể tạo câu trả lời tự nhiên hơn, nhưng việc cập nhật kiến thức và kiểm soát nguồn thông tin vẫn là những thách thức đáng kể.

Các mô hình ngôn ngữ lớn có thể lưu giữ nhiều thông tin thông qua quá trình huấn luyện trên tập dữ liệu lớn. Tuy nhiên, kiến thức được học trong quá trình huấn luyện không đồng nghĩa với việc mô hình luôn có thông tin mới nhất hoặc có thể xác minh mọi phát biểu. Khi gặp câu hỏi về một sự kiện vừa xảy ra, một tài liệu nội bộ hoặc một dữ liệu không xuất hiện trong tập huấn luyện, mô hình có thể tạo câu trả lời thiếu chính xác nếu không có nguồn thông tin hỗ trợ.

Một hướng giải quyết là đưa thông tin bên ngoài vào quá trình tạo câu trả lời. Thay vì yêu cầu mô hình chỉ dựa vào tham số đã học, hệ thống thực hiện truy xuất dữ liệu phù hợp với câu hỏi và cung cấp phần dữ liệu đó làm ngữ cảnh đầu vào. Phương pháp này tạo nền tảng cho Retrieval-Augmented Generation.

Năm 2020, Lewis và cộng sự công bố công trình Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, trình bày một kiến trúc kết hợp bộ truy xuất tài liệu với mô hình sinh ngôn ngữ cho các nhiệm vụ xử lý ngôn ngữ tự nhiên cần nhiều kiến thức. Công trình này là một mốc quan trọng trong quá trình phát triển RAG hiện đại.

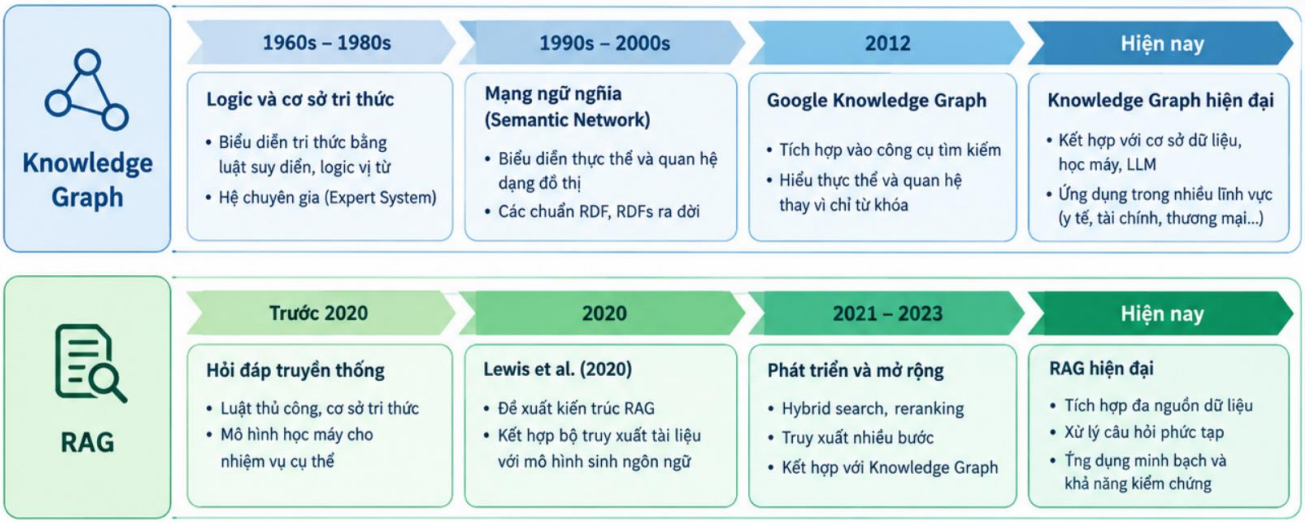
Ở dạng khái quát, một hệ thống RAG gồm hai thành phần chính:

- Retriever: tìm kiếm tài liệu hoặc các đoạn dữ liệu có liên quan đến câu hỏi.
- Generator: sử dụng câu hỏi cùng ngữ cảnh được truy xuất để tạo câu trả lời.

Trong các hệ thống thực tế, quy trình thường có thêm bước chuẩn bị dữ liệu, chia tài liệu thành các đoạn nhỏ, tạo vector embedding và lưu trữ vector trong một cơ sở dữ liệu phù hợp. Khi nhận câu hỏi, hệ thống biến đổi câu hỏi thành dạng biểu diễn phục vụ truy xuất, tìm những đoạn liên quan và đưa chúng vào prompt của mô hình sinh ngôn ngữ.

Cùng với sự phổ biến của LLM, RAG tiếp tục được mở rộng bằng nhiều kỹ thuật như hybrid search, reranking, truy xuất nhiều bước, truy vấn có cấu trúc và kết hợp với Knowledge Graph. Các hướng mở rộng này nhằm cải thiện độ chính xác của tài liệu được truy xuất, khả năng xử lý câu hỏi phức tạp và tính minh bạch của câu trả lời.

Cần lưu ý rằng RAG không tự động bảo đảm câu trả lời đúng. Chất lượng hệ thống vẫn phụ thuộc vào độ tin cậy của nguồn dữ liệu, cách chia đoạn, mô hình embedding, bộ truy xuất, prompt và mô hình sinh. Nếu tài liệu không chứa thông tin cần thiết hoặc bộ truy xuất chọn nhầm nội dung, câu trả lời cuối cùng vẫn có thể sai lệch.



Hình 5.1. Quá trình phát triển Knowledge Graph và RAG.

c) Mối quan hệ giữa Knowledge Graph và RAG

Knowledge Graph và RAG có thể được xem là hai phương pháp bổ trợ nhau trong việc xây dựng hệ thống khai thác tri thức. Knowledge Graph tổ chức thông tin theo các thực thể và quan hệ có cấu trúc, trong khi RAG tập trung vào việc truy xuất ngữ cảnh từ kho dữ liệu để hỗ trợ mô hình sinh câu trả lời.

Ví dụ, với câu hỏi "Một loại thuốc được đề cập trong tài liệu nào và liên quan đến bệnh gì?", Knowledge Graph có thể hỗ trợ xác định quan hệ giữa thuốc và bệnh. RAG có thể truy xuất các đoạn tài liệu giải thích thông tin đó và cung cấp nguồn tham khảo. Khi kết hợp, hệ thống có thể sử dụng đồ thị để xác định các thực thể liên quan, sau đó truy xuất văn bản để lấy thêm ngữ cảnh chi tiết.

Tuy nhiên, không phải mọi ứng dụng RAG đều cần Knowledge Graph. Với kho tài liệu đơn giản, RAG thông thường có thể đáp ứng yêu cầu. Knowledge Graph phát huy lợi thế khi bài toán đòi hỏi truy vấn quan hệ nhiều bước, liên kết thực thể từ nhiều nguồn hoặc phân tích những mối quan hệ khó thể hiện chỉ bằng tìm kiếm văn bản.

5.1.2. Vai trò và ý nghĩa của Knowledge Graph và RAG trong trí tuệ nhân tạo

Knowledge Graph và RAG có vai trò quan trọng trong việc mở rộng khả năng khai thác tri thức của các hệ thống AI. Trong khi mô hình học máy truyền thống thường được thiết kế để dự đoán nhãn hoặc giá trị dựa trên dữ liệu đầu vào, hệ thống KG/RAG hướng đến việc tổ chức, truy xuất và sử dụng thông tin để hỗ trợ trả lời câu hỏi hoặc giải thích một vấn đề.

a) Hỗ trợ quản lý và liên kết tri thức

Một vấn đề thường gặp khi khai thác dữ liệu là thông tin được lưu trữ trong nhiều nguồn khác nhau. Các nguồn có thể sử dụng tên gọi, định dạng hoặc cách mô tả không thống nhất. Knowledge Graph giúp tổ chức dữ liệu thành các thực thể có định danh và các quan hệ được xác định rõ ràng.

Ví dụ, trong lĩnh vực chứng khoán, một doanh nghiệp có thể được nhắc đến bằng tên đầy đủ, tên viết tắt hoặc mã cổ phiếu. Nếu các cách gọi này được chuẩn hóa và liên kết với cùng một thực thể, hệ thống có thể kết nối thông tin về doanh nghiệp, báo cáo tài chính, ngành nghề và sự kiện liên quan. Cách tổ chức này hỗ trợ việc tìm kiếm thông tin theo quan hệ thay vì chỉ dựa vào sự xuất hiện của một từ khóa.

b) Cung cấp ngữ cảnh bên ngoài cho mô hình ngôn ngữ lớn

RAG cho phép hệ thống sử dụng dữ liệu bên ngoài trong thời điểm trả lời câu hỏi. Dữ liệu có thể được cập nhật mà không cần huấn luyện lại toàn bộ mô hình ngôn ngữ. Đây là một ưu điểm quan trọng trong những lĩnh vực có thông tin thay đổi theo thời gian hoặc có nhiều tài liệu chuyên môn.

Trong ứng dụng giá vàng, chẳng hạn, hệ thống có thể truy xuất bản ghi giá theo ngày và tài liệu giải thích thị trường trước khi tạo câu trả lời. Đối với tài liệu y tế, RAG có thể lấy các đoạn nội dung từ nguồn chuyên môn đã lựa chọn để hỗ trợ giải thích. Tuy nhiên, tính cập nhật của câu trả lời phụ thuộc vào thời điểm cập nhật dữ liệu và khả năng truy xuất chính xác của hệ thống.

c) Cải thiện khả năng truy xuất nguồn và kiểm tra thông tin

Một hệ thống hỏi đáp đáng tin cậy không chỉ cần tạo ra câu trả lời dễ hiểu mà còn cần hỗ trợ người dùng kiểm tra căn cứ của câu trả lời đó. RAG có thể lưu lại tên tài liệu, đoạn văn bản, đường dẫn và thời điểm dữ liệu được thu thập để hiển thị nguồn tham khảo.

Knowledge Graph bổ sung một góc nhìn khác: các quan hệ giữa thực thể có thể được lưu trữ rõ ràng, giúp giải thích tại sao hệ thống kết nối hai khái niệm với nhau. Khi thiết kế đúng, hệ thống có thể cung cấp cả đoạn văn bản làm bằng chứng và quan hệ đồ thị liên quan.

Dù vậy, việc hiển thị nguồn không đồng nghĩa với việc câu trả lời chắc chắn đúng. Một nguồn có thể không đáng tin cậy, đã lỗi thời hoặc không thực sự hỗ trợ phát biểu mà mô hình tạo ra. Vì vậy, việc đánh giá cần xem xét cả độ chính xác của truy xuất và mức độ câu trả lời bám sát nguồn.

d) Hỗ trợ giải quyết câu hỏi phức tạp

Một số câu hỏi yêu cầu kết nối nhiều dữ kiện. Ví dụ, người dùng có thể muốn biết một doanh nghiệp thuộc ngành nào, sự kiện nào được ghi nhận trong báo cáo và giá cổ phiếu biến động ra sao trong một khoảng thời gian. Knowledge Graph có thể hỗ trợ truy vấn các quan hệ giữa doanh nghiệp, ngành và sự kiện; RAG có thể truy xuất tài liệu mô tả các thông tin đó.

Trong lĩnh vực y tế, việc kết nối bệnh, triệu chứng, xét nghiệm và phương pháp điều trị có thể hỗ trợ tổ chức kiến thức. Tuy nhiên, câu trả lời trong lĩnh vực sức khỏe phải giới hạn trong phạm vi thông tin có nguồn đáng tin cậy, tránh biến hệ thống hỏi đáp thành công cụ tự chẩn đoán hoặc thay thế tư vấn của nhân viên y tế.

e) Tăng khả năng mở rộng hệ thống AI

Một hệ thống được thiết kế với KB và RAG có thể bổ sung tài liệu hoặc mở rộng sang lĩnh vực mới mà không nhất thiết phải huấn luyện lại mô hình ngôn ngữ từ đầu. Với điều kiện dữ liệu được chuẩn hóa và quy trình indexing được quản lý tốt, hệ thống có thể tiếp nhận thêm tài liệu y tế, thông tin doanh nghiệp hoặc dữ liệu thị trường.

Knowledge Graph cũng có thể mở rộng bằng cách thêm thực thể, quan hệ và thuộc tính mới. Tuy nhiên, mở rộng không chỉ đơn giản là thêm dữ liệu: hệ thống cần xử lý thực thể trùng lặp, quan hệ mâu thuẫn, nguồn dữ liệu có chất lượng khác nhau và các bản ghi không đồng nhất về thời gian.

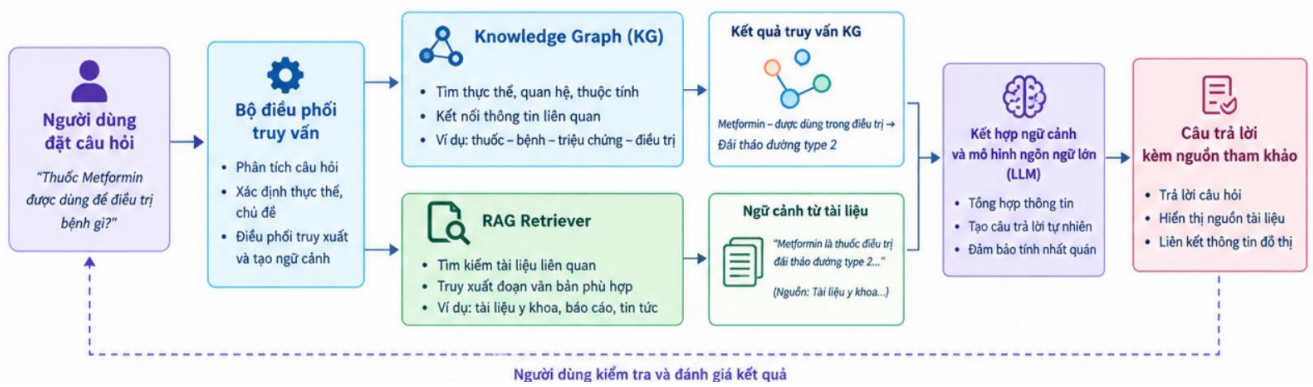
f) Ý nghĩa đối với chương nghiên cứu

Trong chương này, Knowledge Graph và RAG được xem xét như các thành phần có thể hỗ trợ lẫn nhau. Knowledge Graph cung cấp cách biểu diễn tri thức theo cấu trúc quan hệ; RAG cung cấp cơ chế tìm kiếm ngữ cảnh để mô hình ngôn ngữ tạo câu trả lời dựa trên tài liệu.

Ba lĩnh vực bệnh tiểu đường, chứng khoán và giá vàng được lựa chọn để minh họa những loại tri thức khác nhau. Bệnh tiểu đường thiên về kiến thức chuyên môn và quan hệ giữa các khái niệm; chứng khoán kết hợp thông tin doanh nghiệp, sự kiện và chuỗi dữ liệu theo thời gian; giá vàng nhấn mạnh dữ liệu có mốc thời gian, loại giá và nguồn ghi nhận. Sự khác biệt này

cho phép nghiên cứu khả năng áp dụng cùng một kiến trúc vào các bài toán có đặc điểm dữ liệu khác nhau.

Khi bước sang phần thực nghiệm, cần xây dựng bộ câu hỏi phù hợp với từng lĩnh vực, xác định nguồn tham chiếu và sử dụng tiêu chí đánh giá nhất quán. Kết quả thực nghiệm sẽ giúp làm rõ trường hợp nào RAG thông thường đáp ứng tốt, trường hợp nào Knowledge Graph đem lại lợi ích và những hạn chế còn tồn tại.



Hình 5.2. Vai trò của Knowledge Graph và RAG trong hệ thống AI hỏi đáp có truy xuất nguồn.

5.2. Knowledge Graph và Knowledge Base (KB)

5.2.1. Khái niệm và mục tiêu xây dựng Knowledge Graph

Knowledge Base (KB) là cơ sở tri thức dùng để lưu trữ và tổ chức thông tin nhằm phục vụ việc tra cứu, suy luận hoặc hỗ trợ giải quyết một nhiệm vụ cụ thể. Knowledge Graph (KG) là một cách biểu diễn tri thức dưới dạng đồ thị, trong đó các đối tượng được thể hiện bằng nút (node) và mối quan hệ giữa chúng được thể hiện bằng cạnh (relationship). Mỗi nút hoặc quan hệ có thể đi kèm các thuộc tính mô tả, giúp hệ thống lưu trữ và khai thác thông tin có cấu trúc.

Trong đề tài này, Knowledge Graph được xây dựng nhằm tổ chức tri thức từ các bộ dữ liệu thuộc ba lĩnh vực: bệnh tiểu đường, chứng khoán và giá vàng. Thay vì chỉ lưu thông tin dưới dạng các dòng trong tệp CSV hoặc các đoạn văn bản độc lập, KG mô tả các thành phần dữ liệu và mối liên hệ giữa chúng. Cách tổ chức này tạo cơ sở để truy xuất thông tin theo thực thể, nhóm thuộc tính, thống kê hoặc mối quan hệ đã được xác định.

Đối với bộ dữ liệu bệnh tiểu đường, KG hỗ trợ mô tả cấu trúc tập dữ liệu, các nhóm trường dữ liệu, biến mục tiêu và một số kết quả thống kê. Ví dụ, các trường như `bmi`, `glucose_fasting`, `hba1c` và `physical_activity_minutes_per_week` có thể được tổ chức thành các nút để hệ

thống tra cứu theo tên trường hoặc nhóm ý nghĩa. Những kết quả thống kê được lưu riêng giúp phân biệt giữa thông tin mô tả cấu trúc dữ liệu và bằng chứng định lượng.

Đối với dữ liệu chứng khoán và giá vàng, mô hình tri thức hướng tới việc tổ chức thông tin tài chính theo tài liệu, mã cổ phiếu, thời gian và nội dung thống kê tương ứng. Việc lưu thông tin nhận diện như mã tài liệu, mã cổ phiếu và năm cũng hỗ trợ lọc kết quả trước khi xếp hạng bằng chứng.

Mục tiêu của việc xây dựng KG trong đề tài gồm:

- Tổ chức dữ liệu thành các thành phần có cấu trúc và có thể liên kết với nhau.
- Tạo điều kiện truy xuất thông tin dựa trên tên nút, loại nút và mối quan hệ.
- Lưu giữ các thuộc tính nhận diện và thông tin nguồn để kiểm tra bằng chứng.
- Tạo một nhánh truy xuất có thể so sánh với RAG-only và phương pháp kết hợp KG+RAG.
- Làm nền tảng cho việc tích hợp và trình bày kết quả truy xuất trên giao diện web.

Trong phạm vi thực nghiệm, KG không thay thế dữ liệu gốc mà đóng vai trò là một lớp tổ chức tri thức được xây dựng từ dữ liệu và các kết quả phân tích. Điều này cho phép hệ thống vừa giữ được thông tin nguồn, vừa khai thác các liên kết đã được mô hình hóa.

5.2.2. Mô hình hóa tri thức và cấu trúc đồ thị

Trong notebook, quá trình xây dựng KG bắt đầu từ việc xác định cấu trúc dữ liệu và phân nhóm các trường thông tin. Với miền bệnh tiểu đường, mô hình được tạo từ các nhóm trường trong `SCHEMA_GROUPS` và biến mục tiêu `diagnosed_diabetes`. Danh sách nút và quan hệ được xây dựng trước, sau đó lưu thành cấu trúc JSON để kiểm tra và làm đầu vào cho bước nạp dữ liệu vào Neo4j.

Các nhóm nút chính trong mô hình KG của dữ liệu Diabetes gồm:

- Dataset: đại diện cho bộ dữ liệu tổng thể.
- ColumnGroup: đại diện cho nhóm các trường có cùng mục đích hoặc ý nghĩa.
- Column: đại diện cho một trường dữ liệu cụ thể, có thể lưu các thuộc tính như tên trường, kiểu dữ liệu và số lượng giá trị phân biệt.
- Target: đại diện cho biến mục tiêu dùng trong phân tích, cụ thể là `diagnosed_diabetes`.
- StatisticalResult: đại diện cho kết quả thống kê theo trường dữ liệu và nhóm nhãn, gồm các thuộc tính như đặc trưng, giá trị nhãn, số lượng mẫu, giá trị trung bình, trung vị và độ lệch chuẩn khi có dữ liệu tương ứng.

Các nhóm nút này giúp biểu diễn những khía cạnh khác nhau của tri thức. Chẳng hạn, nút `Column` mô tả một trường dữ liệu, trong khi nút `StatisticalResult` lưu một kết quả phân tích

cụ thể. Hai loại thông tin không nên bị xem là tương đương: tên trường cho biết dữ liệu có gì, còn kết quả thống kê cho biết dữ liệu đã được tổng hợp như thế nào.

Các quan hệ trong KG thể hiện sự liên kết giữa các thành phần. Một số loại quan hệ được sử dụng trong mô hình dữ liệu Diabetes gồm `HAS_TARGET`, `CONTAINS_GROUP` và `CONTAINS_COLUMN`. Ví dụ, bộ dữ liệu có thể liên kết với biến mục tiêu thông qua `HAS_TARGET`, liên kết với một nhóm trường thông qua `CONTAINS_GROUP`, và nhóm trường liên kết với từng trường dữ liệu bằng `CONTAINS_COLUMN`.

Đoạn mã dưới đây trích từ notebook, minh họa cách tạo nút cho bộ dữ liệu, biến mục tiêu và các nhóm trường:

```
kg_nodes = [
    {
        "id": "dataset_diabetes",
        "label": "Dataset",
        "name": "Diabetes Dataset",
    },
    {
        "id": f"target_{TARGET}",
        "label": "Target",
        "name": TARGET,
    },
]


kg_relationships = [
    {
        "source": "dataset_diabetes",
        "type": "HAS_TARGET",
        "target": f"target_{TARGET}",
    }
```

```
]


for group_name, columns in SCHEMA_GROUPS.items():
    group_id = (
        "group_"
        + group_name.lower()
        .replace(" ", "_")
        .replace("/", "_")
    )

    kg_nodes.append({
        "id": group_id,
        "label": "ColumnGroup",
        "name": group_name,
    })

    kg_relationships.append({
        "source": "dataset_diabetes",
        "type": "CONTAINS_GROUP",
        "target": group_id,
    })
```

Giải thích: `kg_nodes` lưu danh sách nút, còn `kg_relationships` lưu danh sách quan hệ. Mỗi nút có một mã định danh `id`, loại nút `label` và tên mô tả `name`. Quan hệ xác định nút nguồn, loại quan hệ và nút đích. Việc tách riêng hai danh sách giúp kiểm tra cấu trúc đồ thị trước khi đưa vào cơ sở dữ liệu.

Sau khi tạo mô hình, notebook kiểm tra mã định danh bị trùng và những quan hệ có nút nguồn hoặc nút đích chưa được khai báo. Đây là bước cần thiết để hạn chế lỗi cấu trúc, ví dụ một quan hệ tham chiếu tới nút không tồn tại.

```
node_ids = [node["id"] for node in kg_nodes]
node_id_set = set(node_ids)

duplicate_node_ids = (
    pd.Series(node_ids)
    .value_counts()
    .loc[lambda counts: counts > 1]
)

invalid_relationships = [
    relationship
    for relationship in kg_relationships
    if (
        relationship["source"] not in node_id_set
        or relationship["target"] not in node_id_set
    )
]

if len(duplicate_node_ids) > 0:
    raise ValueError("KG contains duplicate node IDs.")

if invalid_relationships:
    raise ValueError(
        "KG contains relationships pointing to missing nodes."
    )
```

Giải thích: Tập hợp `node_id_set` được dùng để kiểm tra sự tồn tại của nút nguồn và nút đích. Nếu phát hiện mã nút trùng hoặc quan hệ không hợp lệ, chương trình dừng lại để người thực hiện xử lý trước khi nạp dữ liệu. Việc kiểm tra này góp phần bảo đảm tính nhất quán của mô hình KG.

Đối với miền chứng khoán và giá vàng, web demo còn sử dụng mô hình tài liệu tài chính với các trường nhận diện như `doc_id`, `domain`, `kind`, `ticker`, `year` và `text`. Phần nạp KG từ Neo4j trong `app.py` dự kiến đọc các nút `FinancialDataset`, các nút `FinancialFact` và quan hệ

HAS_FACT. Như vậy, cấu trúc tài chính tập trung vào các thông tin hoặc sự kiện có thể truy xuất theo lĩnh vực, mã cổ phiếu và năm.

5.2.3. Lưu trữ KG bằng Neo4j và truy xuất tri thức

Sau khi mô hình đồ thị được tạo và kiểm tra, notebook sử dụng Neo4j để lưu trữ các nút và quan hệ. Neo4j là hệ quản trị cơ sở dữ liệu đồ thị, cho phép biểu diễn trực tiếp các nút, quan hệ và thuộc tính thay vì phải mô phỏng quan hệ bằng nhiều bảng dữ liệu riêng biệt.

Trước khi nạp dữ liệu, notebook thiết lập ràng buộc duy nhất cho thuộc tính id của các nút KGNode. Sau đó, từng nút được ghi vào cơ sở dữ liệu bằng lệnh MERGE:

```
with neo4j_driver.session() as session:
    session.run("""
        CREATE CONSTRAINT kg_node_id_unique IF NOT EXISTS
        FOR (n:KGNode)
        REQUIRE n.id IS UNIQUE
    """).consume()

    for node in kg_nodes:
        session.run("""
            MERGE (n:KGNode {id: $id})
            SET n.kind = $kind,
                n.name = $name,
                n.data_type = $data_type,
                n.unique_values = $unique_values
        """,
            id=node["id"],
            kind=node["label"],
            name=node["name"],
            data_type=node.get("data_type"),
            unique_values=node.get("unique_values"),
        ).consume()
```

Giải thích: Ràng buộc duy nhất giúp tránh tạo nhiều nút có cùng mã định danh. Lệnh MERGE tìm nút có id tương ứng; nếu chưa có thì tạo nút mới, nếu đã có thì sử dụng nút đó và cập nhật các thuộc tính được chỉ định. Cách làm này thuận tiện khi cần chạy lại notebook mà không muốn tạo nút trùng theo cùng mã định danh.

Sau khi nạp nút, các quan hệ được tạo bằng cách tìm hai nút theo mã định danh rồi liên kết chúng:

```
with neo4j_driver.session() as session:

    for relationship in kg_relationships:

        session.run("""

            MATCH (source:KGNode {id: $source_id})

            MATCH (target:KGNode {id: $target_id})
```

```
            MERGE (
                source
            )-[r:KG_RELATION {type: $relationship_type}]->(
                target
            )
        """,
            source_id=relationship["source"],
            target_id=relationship["target"],
            relationship_type=relationship["type"],
        ).consume()
```

Giải thích: `MATCH` tìm các nút nguồn và đích trong Neo4j. `MERGE` tạo quan hệ nếu chưa có quan hệ tương ứng với loại đã chỉ định. Loại quan hệ được lưu trong thuộc tính `type`, nhờ đó hệ thống có thể phân biệt các quan hệ như `HAS_TARGET` và `CONTAINS_COLUMN`.

Đối với truy xuất KG trong notebook Diabetes, hệ thống chuẩn hóa câu hỏi thành các từ khóa, tìm các nút có tên phù hợp và lấy thêm các quan hệ trực tiếp của nút đó để trả về làm bằng chứng. Ở các phiên bản cải tiến, câu hỏi còn được chuẩn hóa dấu tiếng Việt và mở rộng từ khóa theo ý định, chẳng hạn nhóm câu hỏi về lối sống có thể được ánh xạ tới các trường liên quan đến vận động, chế độ ăn, giấc ngủ và hút thuốc.

Đoạn mã minh họa truy xuất các nút khớp từ khóa và các nút lân cận trong notebook:

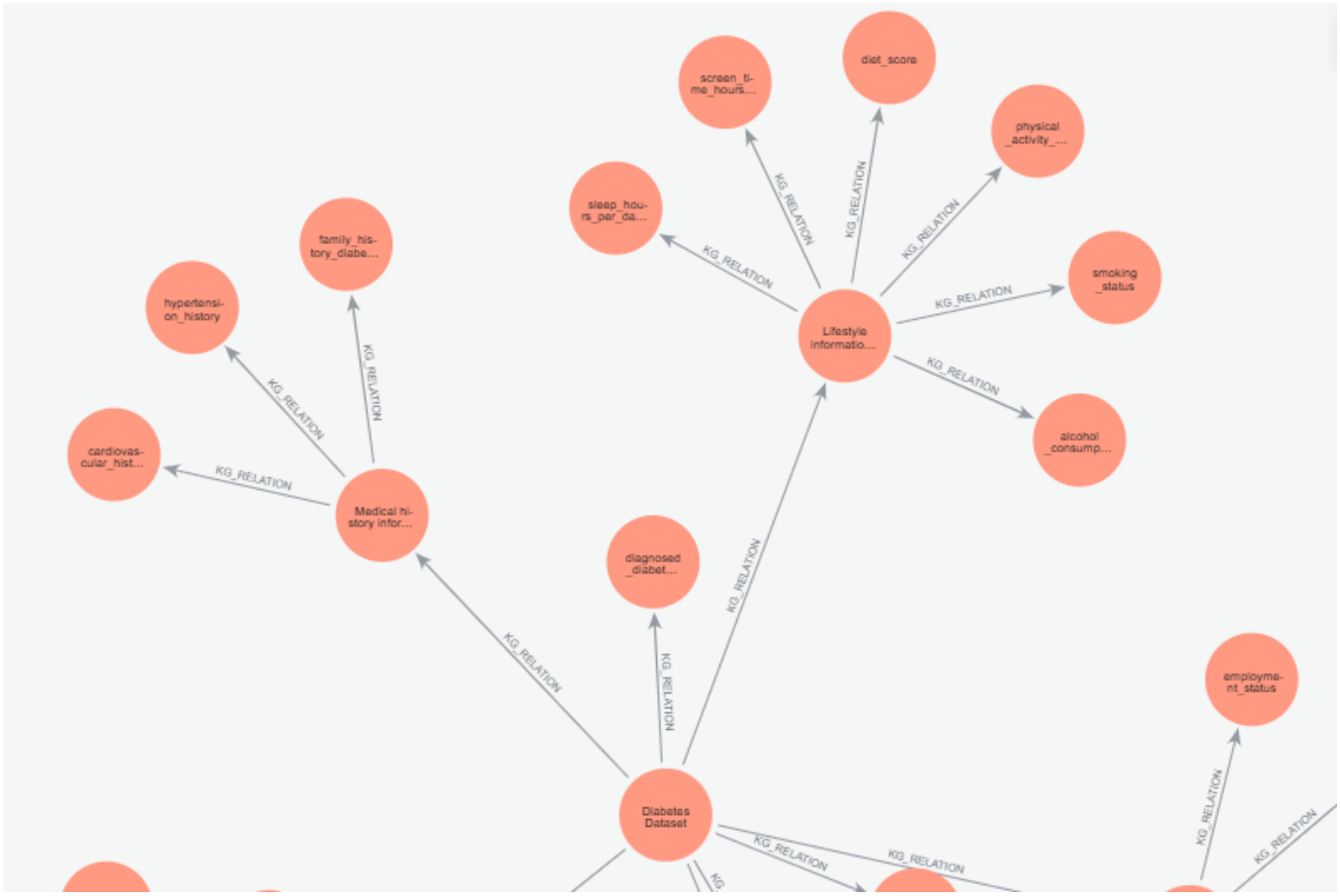
```
with neo4j_driver.session() as session:
    result = session.run("""
        MATCH (source:KGNode)
        WHERE any(
            term IN $terms
            WHERE toLower(source.name) CONTAINS term
        )
        OPTIONAL MATCH
            (source)-[r:KG_RELATION]-(neighbor:KGNode)
        RETURN
            source.id AS source_id,
            source.kind AS source_kind,
            source.name AS source_name,
            r.type AS relationship,
            neighbor.id AS neighbor_id,
            neighbor.kind AS neighbor_kind,
            neighbor.name AS neighbor_name
        LIMIT $limit
    """, terms=query_terms, limit=limit)

    kg_results = result.data()
```

Giải thích: Truy vấn tìm nút có tên chứa ít nhất một từ khóa trong câu hỏi. `OPTIONAL MATCH` tiếp tục lấy quan hệ và nút lân cận nếu có. Kết quả bao gồm thông tin về nút nguồn, loại quan hệ và nút liên quan, giúp người dùng kiểm tra vì sao một kết quả được trả về.

Tuy nhiên, truy xuất theo từ khóa vẫn có giới hạn: cách này phụ thuộc vào mức độ trùng khớp giữa câu hỏi và tên hoặc thuộc tính của nút. Nếu người dùng diễn đạt khác với tên trường đã lưu, hệ thống có thể không tìm thấy bằng chứng phù hợp. Việc chuẩn hóa câu hỏi, mở rộng từ khóa và bổ sung các truy vấn theo nhóm dữ liệu giúp giảm hạn chế này, nhưng chưa tương đương với khả năng hiểu ngữ nghĩa tự do của một mô hình ngôn ngữ lớn.

Trong web demo, có sự khác biệt giữa KG được lưu trong Neo4j và nhánh KG-only của bộ so sánh. Với Diabetes, `app.py` đọc nút `KGNode`, nút `StatisticalResult` và các quan hệ `KG_RELATION` từ Neo4j, sau đó chuyển một phần nội dung thành biểu diễn văn bản để xếp hạng. Với chứng khoán và giá vàng, file CSV tài liệu tài chính được nạp vào cả `KG_DOCS` và `RAG_DOCS`; nếu có dữ liệu tài chính phù hợp trong Neo4j, các fact cũng được đọc bổ sung. Vì vậy, trong phiên bản web demo hiện tại, nhánh KG-only ở miền tài chính chủ yếu là xếp hạng biểu diễn tri thức bằng từ khóa, không phải truy vấn đường đi trên đồ thị cho mọi câu hỏi.



Hình 5.3. Một phần Knowledge Graph của bộ dữ liệu Diabetes trên Neo4j

5.3. Retrieval-Augmented Generation (RAG)

5.3.1. Khái niệm và nguyên lý hoạt động

Retrieval-Augmented Generation (RAG) là phương pháp kết hợp cơ chế truy xuất thông tin với mô hình sinh ngôn ngữ. Thay vì chỉ dựa vào kiến thức có sẵn trong mô hình, hệ thống tìm kiếm các tài liệu liên quan đến câu hỏi từ một kho tri thức bên ngoài, sau đó sử dụng những tài liệu này làm ngữ cảnh để tạo câu trả lời. Nhờ đó, câu trả lời có thể dựa trên thông tin cụ thể từ nguồn dữ liệu được cung cấp và hỗ trợ kiểm tra nguồn tốt hơn.

Một hệ thống RAG đầy đủ thường gồm ba bước chính:

- Chuẩn bị kho tài liệu: thu thập, làm sạch, chia tài liệu thành các đoạn phù hợp và lưu thông tin nguồn.
- Truy xuất (Retrieval): so sánh câu hỏi với các đoạn tài liệu để lựa chọn những bằng chứng liên quan nhất.
- Sinh câu trả lời (Generation): đưa câu hỏi và bằng chứng truy xuất được vào mô hình ngôn ngữ để tạo câu trả lời dựa trên ngữ cảnh.

Trong đề tài này, phần thực nghiệm tập trung vào việc xây dựng kho tài liệu và đánh giá khả năng truy xuất bằng chứng. Notebook sử dụng TF-IDF kết hợp độ tương đồng cosine để xếp hạng các đoạn văn bản. Web demo tiếp tục sử dụng cách truy xuất này, đồng thời bổ sung bộ lọc theo năm, mã cổ phiếu, thông tin metadata và cơ chế kết hợp thứ hạng từ KG và RAG.

5.3.2. Chuẩn bị tài liệu và xây dựng kho RAG

Để truy xuất được thông tin, dữ liệu đầu vào cần được chuyển thành các đơn vị văn bản có thể tìm kiếm. Trong notebook, hệ thống xây dựng các đoạn tài liệu từ những nguồn đã chuẩn bị, đồng thời giữ lại thông tin nhận diện như mã đoạn, đường dẫn hoặc tên nguồn. Việc giữ thông tin nguồn giúp xác định đoạn văn bản nào đã được sử dụng làm bằng chứng cho một truy vấn.

Đối với Diabetes, kho tài liệu gồm các nội dung liên quan đến cấu trúc bộ dữ liệu, kết quả thống kê và tài liệu tham khảo về kiến thức bệnh tiểu đường. Các nội dung có nguồn khác nhau được phân biệt để hệ thống có thể trả về bằng chứng thích hợp với từng loại câu hỏi. Ví dụ, câu hỏi về số dòng hoặc số cột cần bằng chứng từ thông tin mô tả bộ dữ liệu; câu hỏi về ý nghĩa của HbA1c cần ưu tiên tài liệu tham khảo giải thích chỉ số này, thay vì chỉ trả về tên một trường dữ liệu trong KG.

Đối với chứng khoán và giá vàng, web demo đọc tài liệu từ tệp `financial_knowledge_documents.csv` trong thư mục `results/comparison`. Mỗi bản ghi được

chuẩn hóa về cấu trúc gồm mã tài liệu, lĩnh vực, nội dung, loại tài liệu và metadata như mã cổ phiếu hoặc năm. Việc thống nhất cấu trúc giúp ba lĩnh vực sử dụng chung các hàm truy xuất và cách hiển thị kết quả.

Đoạn mã dưới đây trích từ `app.py`, minh họa việc chuẩn hóa nội dung tài liệu:

```
def make_doc(doc_id, domain, text, **metadata):
    clean_text = str(text or "").strip()

    year = safe_int(metadata.get("year", 0), 0)

    ticker = str(
        metadata.get("ticker", "") or ""
    ).strip().upper()

    kind = str(
        metadata.get("kind", "knowledge") or "knowledge"
    ).strip()

    return {
        "doc_id": str(doc_id or "").strip(),
        "domain": str(domain or "").strip().lower(),
        "text": clean_text,
        "kind": kind,
        "ticker": ticker,
        "year": year,
    }
```

Giải thích: Hàm `make_doc()` đưa các tài liệu về cùng một cấu trúc, bất kể tài liệu thuộc lĩnh vực nào. Nội dung văn bản được loại bỏ khoảng trắng thừa; mã cổ phiếu được chuẩn hóa thành chữ in hoa; năm được chuyển về dạng số nguyên khi có thể. Các trường metadata này không chỉ phục vụ hiển thị mà còn được sử dụng để lọc tài liệu trước khi xếp hạng.

Với tài liệu Diabetes ở dạng `.txt` hoặc `.md`, ứng dụng đọc nội dung từ thư mục `documents/diabetes`. Nếu không tìm thấy tài liệu văn bản nhưng đã nạp được biểu diễn tri thức từ Neo4j, ứng dụng có thể dùng biểu diễn văn bản từ KG làm nguồn dự phòng cho nhánh RAG. Trường hợp này được ghi chú trên giao diện để người sử dụng biết nguồn dữ liệu đang được dùng.

Sau khi chuẩn hóa, các tài liệu được loại trùng dựa trên `doc_id`. Nếu có nhiều bản ghi cùng mã, ứng dụng ưu tiên bản có metadata đầy đủ hơn hoặc nội dung dài hơn. Bước này giúp hạn chế việc cùng một tài liệu xuất hiện nhiều lần trong danh sách kết quả.

5.3.3. Truy xuất tài liệu bằng TF-IDF và cosine similarity

Trong notebook, TF-IDF được sử dụng để biểu diễn tài liệu và câu hỏi dưới dạng vector số. TF-IDF là viết tắt của Term Frequency–Inverse Document Frequency, kết hợp tần suất xuất hiện của từ trong tài liệu với mức độ phổ biến của từ trong toàn bộ tập tài liệu. Những từ xuất

hiện thường xuyên trong một tài liệu nhưng không quá phổ biến trong toàn bộ tập có thể nhận trọng số cao hơn.

Sau khi biểu diễn câu hỏi và tài liệu thành vector, hệ thống sử dụng cosine similarity để đo mức độ tương đồng giữa chúng. Điểm tương đồng càng cao thì tài liệu càng được ưu tiên trong danh sách kết quả. Phương pháp này phù hợp để xây dựng một bộ truy xuất cơ sở vì có thể triển khai trực tiếp bằng thư viện `scikit-learn`, không yêu cầu huấn luyện một mô hình ngôn ngữ riêng.

Đoạn mã sau được trích từ notebook, minh họa cách xây dựng ma trận TF-IDF và truy xuất các đoạn văn bản phù hợp:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

if not rag_chunks:
    raise ValueError("Không có tài liệu để xây dựng bộ truy xuất.")

chunk_texts = [chunk["text"] for chunk in rag_chunks]

tfidf_vectorizer = TfidfVectorizer(
    lowercase=True,
    ngram_range=(1, 2),
)

tfidf_matrix = tfidf_vectorizer.fit_transform(chunk_texts)


def retrieve_tfidf(query, top_k=3):
    query_vector = tfidf_vectorizer.transform([query])

    scores = cosine_similarity(
        query_vector,
        tfidf_matrix
    ).ravel()

    ranked_indices = np.argsort(scores)[::-1][:top_k]

    results = []

    for index in ranked_indices:
        results.append({
            "chunk_id": rag_chunks[index]["chunk_id"],
            "source": rag_chunks[index]["source"],
            "score": float(scores[index]),
            "text": rag_chunks[index]["text"],
        })

    return results
```

Giải thích quy trình:

- `chunk_texts` lấy phần văn bản của từng đoạn tài liệu trong kho RAG.

- `TfidfVectorizer` xây dựng bộ từ vựng và chuyển các đoạn văn bản thành ma trận vector TF-IDF. Tham số `lowercase=True` chuẩn hóa chữ thường; `ngram_range=(1, 2)` cho phép biểu diễn cả từ đơn và cụm gồm hai từ liên tiếp.
- `transform()` chuyển câu hỏi của người dùng thành vector theo cùng bộ từ vựng đã được xây dựng.
- `cosine_similarity()` tính điểm tương đồng giữa câu hỏi và các đoạn tài liệu.
- `np.argsort(scores)[::-1][:top_k]` chọn tối đa `top_k` đoạn có điểm cao nhất theo thứ tự giảm dần.

Kết quả trả về gồm mã đoạn, nguồn, điểm tương đồng và nội dung, giúp kiểm tra trực tiếp bằng chứng thay vì chỉ nhận một danh sách từ khóa.

Trong web demo, cơ chế này được triển khai trong hàm `retrieve_rag()`. Ứng dụng xây dựng chỉ mục TF-IDF riêng cho từng lĩnh vực và lưu ma trận vector để tái sử dụng khi người dùng đặt câu hỏi. Phiên bản web sử dụng thêm `sublinear_tf=True` trong `TfidfVectorizer` để điều chỉnh cách tính trọng số tần suất từ.

Ngoài việc xếp hạng theo độ tương đồng, web demo còn lọc tài liệu trước khi trả kết quả. Khi câu hỏi hoặc giao diện có chỉ định năm, hệ thống chỉ giữ các tài liệu có năm phù hợp trong metadata, nội dung hoặc mã tài liệu. Đối với chứng khoán, mã cổ phiếu cũng được đối chiếu với metadata, nội dung hoặc mã tài liệu. Nhờ đó, truy vấn có điều kiện cụ thể không chỉ dựa vào điểm tương đồng văn bản mà còn phải thỏa mãn điều kiện lọc.

Ví dụ, với câu hỏi “Giá vàng năm 2012 có những dữ liệu nào?”, ứng dụng cố gắng xác định năm 2012 rồi lọc tài liệu trước khi xếp hạng. Nếu không có tài liệu phù hợp, hệ thống trả về trạng thái không tìm thấy bằng chứng thay vì tự thay thế bằng dữ liệu của một năm khác. Tuy nhiên, hiệu quả của bộ lọc vẫn phụ thuộc vào việc metadata và nội dung tài liệu có lưu thông tin thời gian chính xác hay không.

5.3.4. Kết hợp KG và RAG bằng Reciprocal Rank Fusion

Bên cạnh việc đánh giá KG-only và RAG-only riêng biệt, đề tài xây dựng phương pháp KG+RAG nhằm kết hợp bằng chứng từ hai nhánh truy xuất. Ý tưởng của phương pháp kết hợp là tận dụng hai cách xếp hạng khác nhau: nhánh KG ưu tiên mức độ phù hợp giữa câu hỏi và các biểu diễn tri thức, còn nhánh RAG ưu tiên độ tương đồng giữa câu hỏi với nội dung tài liệu.

Trong web demo, hai danh sách kết quả được kết hợp bằng Reciprocal Rank Fusion (RRF). Thay vì cộng trực tiếp điểm của hai phương pháp — vốn có thể có thang điểm khác nhau —

RRF sử dụng vị trí xếp hạng của tài liệu trong từng danh sách. Tài liệu xuất hiện ở vị trí cao trong một hoặc cả hai danh sách sẽ nhận điểm kết hợp cao hơn.

Công thức tổng quát của RRF được biểu diễn như sau:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}$$

Trong đó:

- **d** là tài liệu cần tính điểm kết hợp.
- **R** là tập hợp các danh sách kết quả từ những phương pháp truy xuất.
- **$\mathrm{rank}_r(d)$** là vị trí của tài liệu \(d\) trong danh sách \(r\).
- **k** là hằng số điều chỉnh ảnh hưởng của thứ hạng.

Trong `app.py`, giá trị `rrf_k` được đặt bằng 60. Đoạn mã minh họa cách tính điểm kết hợp từ hai danh sách KG và RAG như sau:

```
fused_scores = Counter()
doc_lookup = {}
rrf_k = 60

for results in (kg_results, rag_results):
    for rank, doc in enumerate(results, start=1):
        doc_id = doc["doc_id"]

        fused_scores[doc_id] += (
            1.0 / (rrf_k + rank)
        )

        doc_lookup[doc_id] = doc

ordered_ids = sorted(
    fused_scores,
    key=lambda doc_id: fused_scores[doc_id],
    reverse=True,
)[:top_k]
```

Giải thích: `kg_results` và `rag_results` lần lượt chứa các kết quả từ hai nhánh. Hàm `enumerate()` xác định vị trí xếp hạng của từng tài liệu, bắt đầu từ 1. Mỗi tài liệu được cộng thêm một giá trị 1/(60+rank) vào `fused_scores`. Sau khi duyệt cả hai danh sách, hệ thống sắp xếp tài liệu theo điểm RRF giảm dần và lấy số kết quả cần hiển thị.

Cách kết hợp này không bảo đảm KG+RAG luôn tốt hơn từng phương pháp riêng lẻ. Hiệu quả phụ thuộc vào chất lượng của hai danh sách đầu vào, cách tổ chức tài liệu và mức độ phù hợp

của bằng chứng với câu hỏi. Nếu một nhánh trả về nhiều kết quả không liên quan, thứ hạng kết hợp vẫn có thể bị ảnh hưởng. Vì vậy, KG-only, RAG-only và KG+RAG cần được đánh giá trên cùng bộ câu hỏi và theo cùng tiêu chí.

5.3.5. Vai trò của RAG trong hệ thống và giới hạn hiện tại

Trong đề tài, RAG bổ sung cho KG ở khả năng tìm kiếm bằng chứng từ văn bản. KG phù hợp với thông tin đã được mô hình hóa thành nút, quan hệ và thuộc tính; RAG cho phép truy xuất các đoạn tài liệu chứa nội dung giải thích hoặc thông tin nguồn mà đồ thị chưa biểu diễn đầy đủ. Khi kết hợp, hệ thống có thể tận dụng cả thông tin có cấu trúc lẫn nội dung văn bản.

Tuy nhiên, phiên bản hiện tại có một số giới hạn cần lưu ý:

- Chưa có bước sinh ngôn ngữ: hệ thống trả về các bằng chứng được xếp hạng, chưa tổng hợp chúng thành câu trả lời tự nhiên bằng LLM.
- TF-IDF dựa trên đặc trưng từ vựng: câu hỏi và tài liệu có ý nghĩa tương tự nhưng sử dụng cách diễn đạt khác nhau vẫn có thể nhận điểm thấp.
- Chất lượng kết quả phụ thuộc vào kho tài liệu: nếu thông tin không có trong nguồn đã nạp, hệ thống không thể truy xuất bằng chứng đáng tin cậy cho thông tin đó.
- RRF chỉ kết hợp thứ hạng: phương pháp này không tự xác minh tính đúng đắn của nội dung trong từng tài liệu.

Kết quả truy xuất không đồng nghĩa với câu trả lời đúng: việc tìm thấy một tài liệu có điểm cao không đủ để khẳng định mọi thông tin trong tài liệu đều đáp ứng yêu cầu của câu hỏi.

Vì vậy, hệ thống sử dụng kết quả truy xuất làm bằng chứng để người dùng kiểm tra, đồng thời đánh giá các phương pháp bằng các chỉ số như tỷ lệ câu hỏi tìm được bằng chứng phù hợp và thời gian truy xuất. Các chỉ số này phản ánh hiệu quả tìm kiếm trong phạm vi bộ câu hỏi và nguồn dữ liệu đã thử nghiệm, không phải độ chính xác của một mô hình sinh câu trả lời.

5.3.6. Thử nghiệm so sánh phương pháp truy xuất TF-IDF và Embedding

Bên cạnh phương pháp truy xuất dựa trên TF-IDF và độ tương đồng cosine đã trình bày ở các mục trước, nghiên cứu thực hiện thêm một thí nghiệm sử dụng Embedding để so sánh khả năng tìm kiếm tài liệu liên quan. Mục tiêu của thí nghiệm là đánh giá sự khác biệt giữa phương pháp biểu diễn văn bản dựa trên tần suất từ và phương pháp biểu diễn ngữ nghĩa bằng vector.

Phương pháp thực hiện

Thí nghiệm sử dụng mô hình `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` để chuyển câu hỏi và tài liệu thành các vector embedding. Độ tương đồng giữa câu hỏi

và tài liệu được sử dụng để xếp hạng các tài liệu ứng viên. Kết quả được so sánh với phương pháp TF-IDF trên cùng một bộ gồm 20 câu hỏi đánh giá, trong đó có 8 câu hỏi thuộc lĩnh vực Diabetes, 6 câu hỏi thuộc lĩnh vực chứng khoán và 6 câu hỏi thuộc lĩnh vực giá vàng.

Các chỉ số được sử dụng gồm:

- **Hit@5:** Tỷ lệ câu hỏi có ít nhất một tài liệu phù hợp xuất hiện trong năm kết quả đầu tiên. Giá trị càng cao thì khả năng tìm thấy tài liệu phù hợp trong nhóm kết quả đầu càng tốt.
- **MRR (Mean Reciprocal Rank):** Đánh giá vị trí xuất hiện của tài liệu phù hợp đầu tiên. Chỉ số càng cao cho thấy tài liệu phù hợp thường xuất hiện ở vị trí càng gần đầu danh sách.
- **Thời gian truy xuất trung bình:** Thời gian trung bình để thực hiện bước tìm kiếm và xếp hạng tài liệu, tính bằng mili giây (ms). Giá trị thấp hơn thể hiện tốc độ truy xuất nhanh hơn trong điều kiện thí nghiệm.

Kết quả so sánh trên toàn bộ bộ câu hỏi

Phương pháp	Số câu hỏi	Hit@5 (%)	MRR (%)	Thời gian trung bình (ms)
Embedding	20	100,00	77,67	33,962
TF-IDF	20	95,00	75,00	22,668

Kết quả cho thấy Embedding đạt Hit@5 bằng 100%, cao hơn mức 95% của TF-IDF. MRR của Embedding cũng cao hơn nhẹ, đạt 77,67% so với 75,00%. Điều này cho thấy trong bộ câu hỏi đánh giá, Embedding có khả năng đưa tài liệu phù hợp vào nhóm năm kết quả đầu ổn định hơn. Tuy nhiên, thời gian truy xuất trung bình của Embedding là 33,962 ms, cao hơn TF-IDF với 22,668 ms.

Kết quả theo từng lĩnh vực

Lĩnh vực	Phương pháp	Hit@5 (%)	MRR (%)	Thời gian trung bình (ms)
Diabetes	Embedding	100,00	63,54	34,941
Diabetes	TF-IDF	87,50	68,75	42,481
Chứng khoán	Embedding	100,00	100,00	36,487

Chứng khoán	TF-IDF	100,00	58,33	14,790
Giá vàng	Embedding	100,00	74,17	30,130
Giá vàng	TF-IDF	100,00	100,00	4,128

Kết quả theo lĩnh vực cho thấy không có phương pháp nào tốt hơn trên tất cả các chỉ số. Với Diabetes, Embedding đạt Hit@5 cao hơn TF-IDF, trong khi TF-IDF có MRR cao hơn. Với dữ liệu chứng khoán, cả hai phương pháp đều đạt Hit@5 bằng 100%, nhưng Embedding có MRR cao hơn và TF-IDF có thời gian truy xuất thấp hơn. Với dữ liệu giá vàng, cả hai phương pháp đều đạt Hit@5 bằng 100%; TF-IDF có MRR cao hơn và tốc độ truy xuất nhanh hơn trong thí nghiệm này.

Một trường hợp minh họa là câu hỏi D21: "What is (are) Diabetic Kidney Disease?". Tài liệu mục tiêu có mã `medquad:5_NIDDK_QA/0000033.xml:11` được Embedding xếp ở vị trí thứ hai, trong khi TF-IDF không đưa tài liệu này vào năm kết quả đầu tiên. Trường hợp này cho thấy Embedding có thể hỗ trợ tìm kiếm khi cách diễn đạt của câu hỏi và tài liệu không trùng khớp hoàn toàn về từ ngữ.

Số câu có kết quả Hit@5 khác nhau giữa hai phương pháp: 1

	question_id	domain	question	target_doc_id	Embedding	TF-IDF
4	D21	diabetes	What is (are) Diabetic Kidney Disease ?	medquad:5_NIDDK_QA/0000033.xml:11	1	0

	question_id	domain	method	target_doc_id	retrieved_doc_ids	target_rank	hit_at_5	retrieved_evidence_preview
9	D21	diabetes	Embedding	medquad:5_NIDDK_QA/0000033.xml:11	diabetes_chunk_1077 | diabetes_chunk_1073 | di...	2.0	1	What is (are) Diabetic Kidney Disease ?\n\n## ...
8	D21	diabetes	TF-IDF	medquad:5_NIDDK_QA/0000033.xml:11	diabetes_chunk_1076 | diabetes_chunk_1078 | di...	NaN	0	What is (are) Diabetic Kidney Disease ?\n\nWha...

Hình 5.4. Tìm số câu hỏi có kết quả khác nhau giữa hai phương pháp

Nhận xét và giới hạn

Từ kết quả thực nghiệm, Embedding có ưu thế nhẹ về Hit@5 và MRR khi xét toàn bộ 20 câu hỏi, nhưng phải đánh đổi bằng thời gian truy xuất trung bình cao hơn. Kết quả theo từng lĩnh vực cũng cho thấy hiệu quả phụ thuộc vào đặc điểm câu hỏi và tài liệu; do đó, chưa thể kết luận Embedding luôn tốt hơn TF-IDF.

Thí nghiệm hiện chỉ sử dụng 20 câu hỏi đánh giá, nên kết quả có tính chất minh họa và chưa đủ để khẳng định phương pháp nào tối ưu cho mọi truy vấn. Ngoài ra, các chỉ số phản ánh khả năng truy xuất tài liệu, không trực tiếp đánh giá độ chính xác của câu trả lời được mô hình sinh tạo ra.

Trong phạm vi đề tài, Embedding được sử dụng như một phương pháp thực nghiệm bổ sung trong notebook để so sánh với TF-IDF. Ứng dụng web Flask hiện tại vẫn giữ nguyên cơ chế truy xuất đã triển khai, trong đó TF-IDF được sử dụng ở nhánh RAG. Vì vậy, kết quả của thí nghiệm Embedding chưa được xem là kết quả đánh giá của ứng dụng web và không làm thay đổi các kết quả KG-only, RAG-only và KG+RAG đã trình bày ở các mục trước.

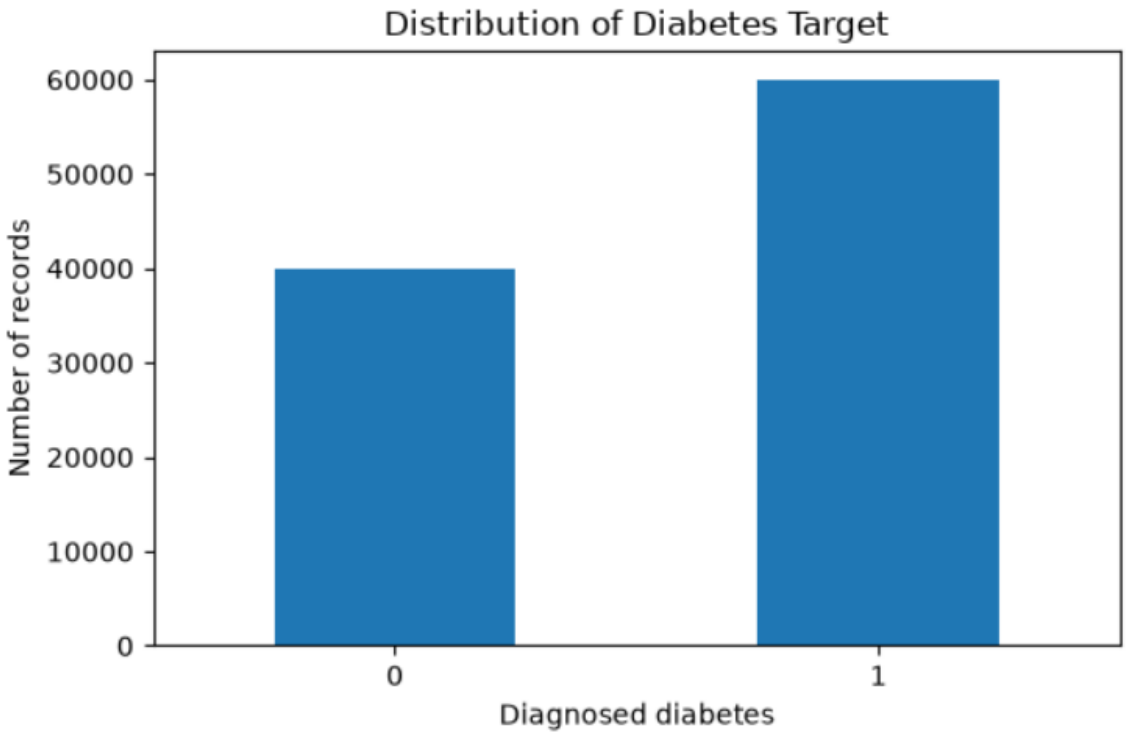
5.4. Knowledge Graph và RAG cho bộ dữ liệu bệnh tiểu đường (Diabetes)

5.4.1. Mục tiêu và đặc điểm dữ liệu

Trong bài toán bệnh tiểu đường, Knowledge Graph (KG) và Retrieval-Augmented Generation (RAG) được sử dụng nhằm tổ chức và truy xuất thông tin liên quan đến bộ dữ liệu sức khỏe. Thay vì chỉ lưu trữ dữ liệu dưới dạng bảng, hệ thống biểu diễn các thành phần quan trọng như tập dữ liệu, nhóm thuộc tính, từng thuộc tính, biến mục tiêu và các kết quả thống kê thành những đối tượng tri thức có liên kết với nhau.

Bộ dữ liệu được sử dụng là `diabetes_health_indicators.csv`. Theo kết quả kiểm tra trong notebook, dữ liệu có 100.000 bản ghi và 29 cột. Biến mục tiêu phân chia các bản ghi thành hai nhóm: lớp 0 có 40.002 bản ghi và lớp 1 có 59.998 bản ghi. Trong lần kiểm tra này, notebook không phát hiện giá trị thiếu hoặc bản ghi trùng lặp.

Dữ liệu có thể được sử dụng để tra cứu cấu trúc bộ dữ liệu, ý nghĩa của các thuộc tính, phân bố biến mục tiêu và những kết quả thống kê đã được tính toán. Tuy nhiên, việc một thuộc tính có liên hệ thống kê với biến mục tiêu không đồng nghĩa với việc thuộc tính đó trực tiếp gây ra bệnh. Hệ thống trong phạm vi đề tài chủ yếu phục vụ tra cứu và cung cấp bằng chứng, không thay thế cho công cụ chẩn đoán y khoa.



Hình 5.5. Phân bố biến mục tiêu Diabetes trong bộ dữ liệu.

Biểu đồ trên giúp minh họa số lượng bản ghi của hai lớp và cho thấy phân bố dữ liệu trước khi xây dựng hệ thống truy xuất tri thức.

5.4.2. Xây dựng Knowledge Graph cho Diabetes

Knowledge Graph được xây dựng nhằm thể hiện cấu trúc và mối liên hệ giữa các thành phần của bộ dữ liệu. Trong mô hình này, các nút chính gồm:

- `Dataset`: đại diện cho bộ dữ liệu Diabetes.
- `ColumnGroup`: đại diện cho nhóm thuộc tính.
- `Column`: đại diện cho từng cột hoặc thuộc tính trong dữ liệu.
- `Target`: đại diện cho biến mục tiêu cần tra cứu.
- `StatisticalResult`: lưu trữ các kết quả thống kê liên quan đến thuộc tính hoặc biến mục tiêu.

Các quan hệ như `HAS_TARGET`, `CONTAINS_GROUP` và `CONTAINS_COLUMN` thể hiện mối liên kết giữa bộ dữ liệu, nhóm thuộc tính, từng cột và biến mục tiêu. Bên cạnh cấu trúc này, các nút `StatisticalResult` được sử dụng để lưu lại những thông tin thống kê đã được tính toán trong notebook.

Ví dụ, khi cần tra cứu biến mục tiêu của bộ dữ liệu, hệ thống có thể dựa vào liên kết từ nút `Dataset` đến nút `Target`. Khi cần tìm hiểu một thuộc tính cụ thể, hệ thống có thể sử dụng các nút `Column` và những thông tin thống kê liên quan để xác định bằng chứng phù hợp.

Việc lưu trữ các nút và quan hệ trong Neo4j cho phép hệ thống biểu diễn tri thức theo cấu trúc đồ thị, thay vì chỉ tìm kiếm trên các dòng dữ liệu độc lập. Đây là cơ sở để thực hiện truy vấn dựa trên tên nút, thuộc tính và các quan hệ đã được xây dựng.

5.4.3. Truy xuất tri thức bằng KG và RAG

Trong hệ thống, phương pháp KG thực hiện tra cứu dựa trên cấu trúc tri thức đã lưu trong Neo4j. Câu hỏi của người dùng được phân tích để tìm những nút, thuộc tính hoặc kết quả thống kê có liên quan. Cách tiếp cận này phù hợp với các câu hỏi yêu cầu thông tin có cấu trúc, chẳng hạn như tên biến mục tiêu, nhóm thuộc tính hoặc thông tin thống kê đã được lưu trong cơ sở tri thức.

Trong khi đó, phương pháp RAG sử dụng các tài liệu văn bản đã chuẩn bị từ bộ dữ liệu và kết quả phân tích. Mỗi tài liệu được biểu diễn thành một đoạn văn bản để hệ thống so sánh với câu hỏi đầu vào. Notebook sử dụng TF-IDF và độ tương đồng cosine để xếp hạng các tài liệu theo mức độ liên quan.

Đoạn code minh họa cách xây dựng bộ truy xuất văn bản bằng TF-IDF:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vectorizer = TfidfVectorizer(
    lowercase=True,
    ngram_range=(1, 2)
)

document_vectors = vectorizer.fit_transform(
    [doc["text"] for doc in documents]
)

def retrieve_rag(query, top_k=5):
    query_vector = vectorizer.transform([query])

    scores = cosine_similarity(
        query_vector,
        document_vectors
    ).flatten()

    top_indices = scores.argsort()[::-1][:top_k]

    results = []
    for idx in top_indices:
        results.append({
            "source": documents[idx]["source"],
            "score": float(scores[idx]),
            "text": documents[idx]["text"]
```

```
        })
    return results
```

Trong đoạn code, `TfidfVectorizer` chuyển đổi tài liệu thành các vector số dựa trên tần suất từ và mức độ đặc trưng của từ trong tập tài liệu. Tham số `ngram_range=(1, 2)` cho phép xem xét cả từ đơn và cụm hai từ liên tiếp. Sau đó, câu hỏi của người dùng được chuyển đổi bằng cùng bộ vector hóa và so sánh với các tài liệu bằng `cosine_similarity`.

Các tài liệu có điểm tương đồng cao được ưu tiên trả về. Mỗi kết quả bao gồm nguồn, điểm liên quan và nội dung văn bản để người dùng có thể kiểm tra bằng chứng. Điểm tương đồng chỉ phản ánh mức độ gần nhau theo cách biểu diễn TF-IDF, không phải xác suất đúng của thông tin.

Phương pháp kết hợp KG và RAG sử dụng kết quả xếp hạng từ cả hai nhánh truy xuất. KG cung cấp thông tin theo cấu trúc tri thức; RAG bổ sung khả năng tìm kiếm trên các đoạn văn bản. Việc kết hợp giúp hệ thống tận dụng hai dạng biểu diễn khác nhau thay vì chỉ phụ thuộc vào một phương pháp.

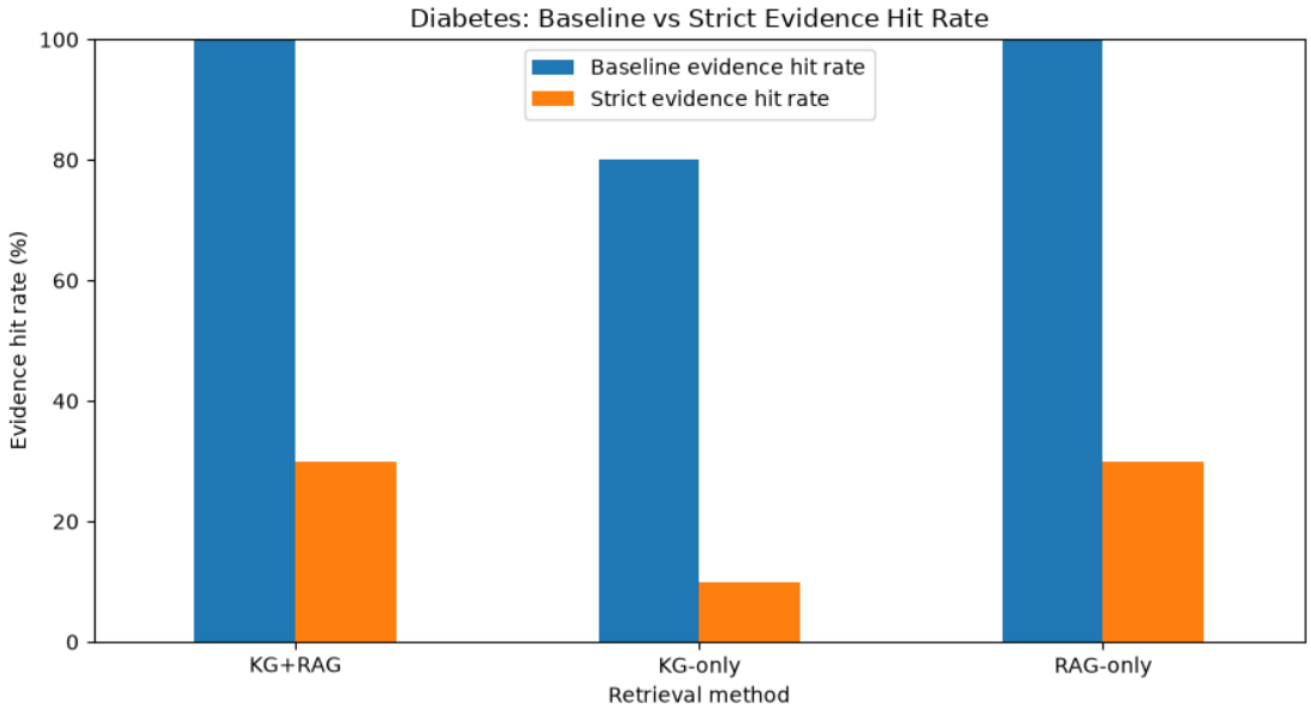
5.4.4. Đánh giá khả năng truy xuất trên Diabetes

Notebook đánh giá ba phương pháp: KG-only, RAG-only và KG+RAG. Hai chỉ số được sử dụng là tỷ lệ truy xuất bằng chứng đạt yêu cầu và thời gian truy xuất trung bình.

Phương pháp	Tỷ lệ truy xuất bằng chứng đạt yêu cầu	Thời gian trung bình
KG-only	10%	277,259 ms
RAG-only	30%	5,944 ms
KG+RAG	30%	185,211 ms

Kết quả cho thấy KG-only đạt tỷ lệ truy xuất bằng chứng đạt yêu cầu là 10%, trong khi RAG-only và KG+RAG cùng đạt 30%. Trong thí nghiệm này, RAG-only có thời gian truy xuất trung bình thấp nhất, khoảng 5,944 ms. Phương pháp KG+RAG có thời gian trung bình cao hơn RAG-only do phải kết hợp kết quả từ hai nhánh truy xuất.

Các kết quả trên phản ánh hiệu quả truy xuất theo bộ câu hỏi đánh giá của notebook. Tỷ lệ 30% không có nghĩa là mô hình chẩn đoán bệnh đạt độ chính xác 30%; đây là chỉ số đánh giá việc hệ thống có tìm được bằng chứng được xem là phù hợp theo tiêu chí của thí nghiệm hay không. Kết quả cũng cho thấy việc kết hợp KG và RAG chưa cải thiện tỷ lệ truy xuất so với RAG-only trong bộ kiểm thử này.



Hình 5.6. So sánh tỷ lệ truy xuất bằng chứng của các phương pháp trên Diabetes.

5.4.5. Nhận xét

Thí nghiệm trên Diabetes cho thấy Knowledge Graph có ưu điểm trong việc biểu diễn rõ cấu trúc và mối quan hệ giữa các thành phần của bộ dữ liệu. RAG có ưu điểm về tốc độ trong cấu hình thực nghiệm này và có thể tìm kiếm trên các đoạn văn bản chứa thông tin liên quan. Phương pháp KG+RAG cung cấp hướng tiếp cận kết hợp hai nguồn bằng chứng, tuy nhiên kết quả hiện tại chưa cho thấy tỷ lệ truy xuất tốt hơn RAG-only.

Do đó, hiệu quả của từng phương pháp phụ thuộc vào cách xây dựng đồ thị, nội dung tài liệu, cách xử lý câu hỏi và tiêu chí đánh giá. Trong phạm vi đề tài, hệ thống có thể hỗ trợ người dùng tra cứu thông tin về dữ liệu Diabetes, nhưng không được xem là hệ thống dự đoán hoặc chẩn đoán bệnh.

5.5. Knowledge Graph và RAG cho dữ liệu chứng khoán

5.5.1. Mục tiêu và đặc điểm dữ liệu

Đối với lĩnh vực chứng khoán, KG và RAG được áp dụng nhằm tổ chức, lưu trữ và truy xuất thông tin từ dữ liệu giá cổ phiếu lịch sử. Khác với bộ dữ liệu Diabetes, dữ liệu chứng khoán có yếu tố thời gian rõ rệt: mỗi bản ghi thường gắn với một mã cổ phiếu và một ngày giao dịch, kèm theo các thông tin về giá và khối lượng giao dịch.

Bộ dữ liệu được sử dụng là `all_stocks_5yr.csv`, gồm 619.040 bản ghi và 7 cột: `date`, `open`, `high`, `low`, `close`, `volume` và `Name`. Dữ liệu thể hiện thông tin giao dịch lịch sử trong giai đoạn 2013–2018. Trong quá trình kiểm tra dữ liệu, notebook cũng ghi nhận một số giá trị thiếu ở các cột giá mở cửa, giá cao nhất, giá thấp nhất hoặc giá đóng cửa.

Đặc điểm này đặt ra yêu cầu hệ thống cần giữ được thông tin về mã cổ phiếu và năm khi chuẩn bị tài liệu, đồng thời tránh trả về bằng chứng thuộc mã cổ phiếu hoặc khoảng thời gian khác với nội dung người dùng yêu cầu. Phạm vi thí nghiệm sử dụng dữ liệu lịch sử, không phải dữ liệu giá chứng khoán trực tiếp theo thời gian thực.

5.5.2. Chuẩn bị tài liệu tri thức chứng khoán

Để hỗ trợ truy xuất, dữ liệu bảng được chuyển thành các tài liệu văn bản có cấu trúc. Mỗi tài liệu có thể chứa nội dung mô tả hoặc thông tin tổng hợp liên quan đến một mã cổ phiếu, năm hoặc một nhóm dữ liệu. Bên cạnh phần nội dung dùng để tìm kiếm, tài liệu còn lưu các trường metadata để hỗ trợ lọc kết quả.

Các trường quan trọng gồm:

- `doc_id`: mã định danh của tài liệu.
- `domain`: lĩnh vực dữ liệu, trong trường hợp này là chứng khoán.
- `kind`: loại tài liệu hoặc loại thông tin.
- `ticker`: mã cổ phiếu liên quan.
- `year`: năm được đề cập.
- `text`: nội dung văn bản dùng để truy xuất.

Trong notebook, dữ liệu tài chính được chuẩn bị thành các tài liệu dùng chung cho thí nghiệm chứng khoán và giá vàng. Kết quả chuẩn bị tài liệu được lưu tại `results/comparison/financial_knowledge_documents.csv`. Theo kết quả thực nghiệm, phần dữ liệu chứng khoán có 3.481 tài liệu được sử dụng cho truy xuất.

Cách tổ chức này cho phép hệ thống sử dụng nội dung văn bản để tìm kiếm, đồng thời dùng metadata để giới hạn phạm vi kết quả. Chẳng hạn, nếu câu hỏi đề cập đến một mã cổ phiếu và một năm cụ thể, hệ thống có thể ưu tiên hoặc lọc các tài liệu phù hợp với mã và năm đó trước khi xếp hạng kết quả.

	date	open	high	low	close	volume	Name
0	2013-02-08	15.07	15.12	14.63	14.75	8407500	AAL
1	2013-02-11	14.89	15.01	14.26	14.46	8882000	AAL
2	2013-02-12	14.45	14.51	14.10	14.27	8126000	AAL
3	2013-02-13	14.30	14.94	14.25	14.66	10259500	AAL
4	2013-02-14	14.94	14.96	13.16	13.99	31879900	AAL

Hình 5.7. Một phần dữ liệu chứng khoán sau khi kiểm tra trong notebook.

5.5.3. Truy xuất bằng chứng bằng KG, RAG và KG+RAG

Trong thí nghiệm chứng khoán, ba phương pháp truy xuất được so sánh.

KG-only: Phương pháp này sử dụng các thông tin tri thức đã được chuẩn bị để tìm kiếm những tài liệu có liên quan đến câu hỏi. Việc đối chiếu từ khóa và metadata như mã cổ phiếu, năm và nội dung tài liệu giúp thu hẹp phạm vi tìm kiếm. Trong phiên bản triển khai của đề tài, nhánh KG đối với dữ liệu tài chính có sử dụng các bản ghi tài liệu đã được chuẩn bị; vì vậy, không nên diễn giải rằng mọi truy vấn chứng khoán đều thực hiện suy luận trên một đồ thị quan hệ tài chính đầy đủ.

RAG-only: Phương pháp này sử dụng TF-IDF và độ tương đồng cosine để xếp hạng tài liệu theo mức độ liên quan với câu hỏi. Các trường metadata như mã cổ phiếu và năm có thể được sử dụng để lọc dữ liệu trước khi xếp hạng. Nhờ vậy, kết quả trả về không chỉ dựa vào sự trùng khớp từ ngữ mà còn có thể được giới hạn theo điều kiện của câu hỏi.

KG+RAG: Phương pháp kết hợp kết quả từ hai nhánh truy xuất bằng thuật toán Reciprocal Rank Fusion (RRF). Thay vì cộng trực tiếp các điểm số vốn có thể được tính theo những cách khác nhau, RRF kết hợp dựa trên vị trí xếp hạng của tài liệu trong từng danh sách kết quả.

Đoạn code minh họa nguyên tắc kết hợp thứ hạng được sử dụng trong phần triển khai:

```
from collections import Counter

fused_scores = Counter()
doc_lookup = {}
rrf_k = 60

for results in (kg_results, rag_results):
    for rank, doc in enumerate(results, start=1):
        doc_id = doc["doc_id"]

        fused_scores[doc_id] += (
            1.0 / (rrf_k + rank)
        )
```

```
        doc_lookup[doc_id] = doc

ordered_ids = sorted(
    fused_scores,
    key=lambda doc_id: fused_scores[doc_id],
    reverse=True
)[:top_k]
```

Trong đoạn code, `kg_results` và `rag_results` là hai danh sách tài liệu đã được xếp hạng từ KG và RAG. Với mỗi tài liệu, thuật toán cộng thêm một giá trị dựa trên thứ hạng của tài liệu trong danh sách tương ứng. Tham số `rrf_k = 60` giúp điều chỉnh ảnh hưởng của thứ hạng. Tài liệu được xếp hạng cao trong một hoặc cả hai danh sách sẽ có cơ hội nhận điểm kết hợp cao hơn.

Sau khi tính điểm, hệ thống sắp xếp tài liệu theo điểm RRF giảm dần và lấy ra số lượng kết quả cần thiết. Đây là bước kết hợp thứ hạng, không phải bước sinh câu trả lời bằng mô hình ngôn ngữ lớn.

5.5.4. Đánh giá khả năng truy xuất trên dữ liệu chứng khoán

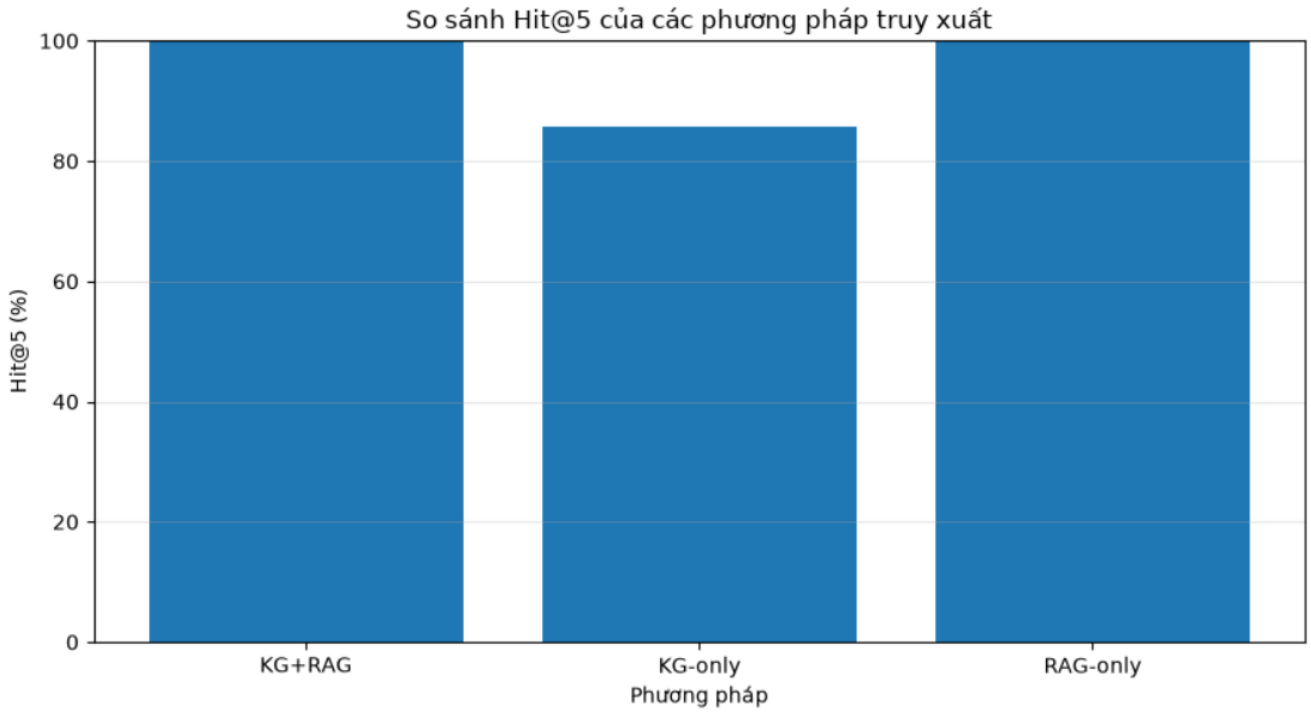
Notebook sử dụng chỉ số Hit@5 để đánh giá khả năng truy xuất. Chỉ số này phản ánh tỷ lệ câu hỏi mà trong năm kết quả đứng đầu có ít nhất một bằng chứng được xem là phù hợp theo tiêu chí đánh giá của thí nghiệm.

Kết quả thực nghiệm được tổng hợp như sau:

Phương pháp	Hit@5	Thời gian truy xuất trung bình
KG-only	85,71%	50,814 ms
RAG-only	100%	7,151 ms
KG+RAG	100%	64,886 ms

Kết quả cho thấy KG-only đạt Hit@5 là 85,71%, trong khi RAG-only và KG+RAG cùng đạt 100% trên bộ câu hỏi đánh giá. Trong thí nghiệm này, RAG-only có thời gian truy xuất trung bình thấp nhất, khoảng 7,151 ms. Phương pháp KG+RAG có thời gian trung bình cao nhất trong ba phương pháp do phải thực hiện cả hai nhánh truy xuất và kết hợp kết quả.

Cần lưu ý rằng bộ đánh giá chỉ gồm 7 câu hỏi. Vì vậy, kết quả Hit@5 bằng 100% chỉ phản ánh các câu hỏi đã được kiểm tra, chưa đủ cơ sở để khẳng định hệ thống luôn tìm đúng bằng chứng trong mọi tình huống thực tế. Kết quả này cũng không thể hiện khả năng dự đoán giá cổ phiếu hoặc hiệu quả đầu tư.



Hình 5.8. So sánh Hit@5 của các phương pháp trên dữ liệu chứng khoán.

5.5.5. Nhận xét

Thí nghiệm trên dữ liệu chứng khoán cho thấy RAG-only có kết quả tốt trong bộ câu hỏi đánh giá và có thời gian truy xuất trung bình thấp nhất. KG+RAG cũng đạt Hit@5 bằng 100%, nhưng thời gian xử lý cao hơn do phải kết hợp hai nhánh truy xuất. KG-only đạt kết quả thấp hơn hai phương pháp còn lại trong thí nghiệm này.

Kết quả cho thấy việc sử dụng metadata như mã cổ phiếu và năm, kết hợp với truy xuất văn bản, có vai trò quan trọng trong việc tìm đúng bằng chứng. Tuy nhiên, do số lượng câu hỏi đánh giá còn nhỏ và dữ liệu chỉ bao phủ giai đoạn lịch sử 2013–2018, cần thận trọng khi mở rộng kết luận sang các mã cổ phiếu, thời kỳ hoặc tình huống khác. Hệ thống trong đề tài được sử dụng để tra cứu bằng chứng từ dữ liệu lịch sử, không cung cấp giá thị trường thời gian thực và không đưa ra khuyến nghị mua bán cổ phiếu.

5.6. Knowledge Graph và RAG cho dữ liệu giá vàng

5.6.1. Mục tiêu và đặc điểm dữ liệu

Bên cạnh Diabetes và chứng khoán, đề tài sử dụng dữ liệu giá vàng để kiểm tra khả năng áp dụng cùng quy trình KG/RAG cho một lĩnh vực dữ liệu tài chính khác. Mục tiêu là tổ chức thông tin lịch sử thành các tài liệu có thể truy xuất, hỗ trợ tìm kiếm bằng chứng theo nội dung câu hỏi và so sánh hiệu quả của ba phương pháp KG-only, RAG-only và KG+RAG.

Bộ dữ liệu được sử dụng là `gold_price.csv`, gồm 13.461 bản ghi. Trong quá trình kiểm tra, notebook ghi nhận 141 giá trị thiếu ở trường giá. Do đó, khi sử dụng kết quả truy xuất, cần phân biệt giữa thông tin thực sự có trong dữ liệu và thông tin không thể xác định từ những bản ghi thiếu dữ liệu. Không nên tự suy diễn hoặc bổ sung giá vàng khi nguồn dữ liệu không cung cấp giá trị tương ứng.

Dữ liệu giá vàng trong thí nghiệm là dữ liệu lịch sử. Hệ thống không tự động lấy giá vàng mới nhất từ thị trường và không được xem là công cụ cung cấp báo giá trực tiếp.

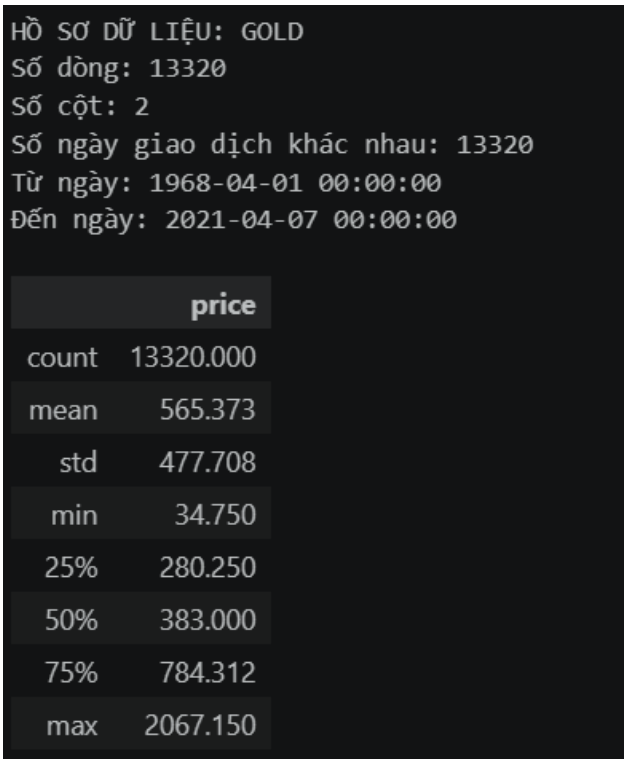
5.6.2. Chuẩn bị tài liệu tri thức giá vàng

Tương tự dữ liệu chứng khoán, dữ liệu giá vàng được chuyển thành các tài liệu văn bản phục vụ truy xuất. Những tài liệu này được lưu cùng bộ tài liệu tài chính tại `results/comparison/financial_knowledge_documents.csv`, nhưng được phân biệt bằng trường metadata `domain` để hệ thống có thể tách riêng nội dung thuộc lĩnh vực giá vàng.

Các trường như `doc_id`, `domain`, `kind`, `year` và `text` giúp xác định nguồn, loại thông tin, thời gian và nội dung cần tìm kiếm. Việc sử dụng metadata theo lĩnh vực giúp hạn chế nguy cơ trả về tài liệu chứng khoán khi người dùng đang tra cứu giá vàng.

Theo kết quả thực nghiệm, phần dữ liệu giá vàng có 55 tài liệu được chuẩn bị cho quá trình truy xuất. Số lượng này là số tài liệu phục vụ thí nghiệm, không phải số bản ghi ban đầu của tệp CSV.

Quy trình chuẩn bị dữ liệu và truy xuất được giữ thống nhất với phần chứng khoán để thuận tiện cho việc so sánh. Tuy nhiên, nội dung tài liệu và điều kiện lọc cần phù hợp với dữ liệu giá vàng, thay vì sử dụng trực tiếp các điều kiện dành cho mã cổ phiếu.



Hình 5.9. Thông tin dữ liệu giá vàng sau khi kiểm tra trong notebook.

5.6.3. Truy xuất tri thức trên dữ liệu giá vàng

Quy trình truy xuất trên dữ liệu giá vàng sử dụng ba phương pháp tương tự phần chứng khoán.

Với KG-only, hệ thống tìm kiếm trên các thông tin tri thức và tài liệu đã chuẩn bị cho lĩnh vực giá vàng. Phạm vi tài liệu được giới hạn theo lĩnh vực và các điều kiện phù hợp với câu hỏi. Cách làm này giúp tránh trộn lẫn thông tin giữa giá vàng và giá cổ phiếu.

Với RAG-only, hệ thống chuyển câu hỏi thành vector TF-IDF và tính độ tương đồng cosine với các tài liệu giá vàng. Những tài liệu có mức độ liên quan cao được đưa lên đầu danh sách kết quả. Metadata được sử dụng để hỗ trợ giới hạn phạm vi trước khi hoặc trong quá trình xếp hạng.

Với KG+RAG, hệ thống kết hợp thứ hạng từ KG-only và RAG-only bằng RRF. Cách kết hợp này giữ nguyên nguyên tắc đã trình bày tại mục 4.5.3, nhờ đó có thể so sánh các phương pháp trên cùng một cách tiếp cận.

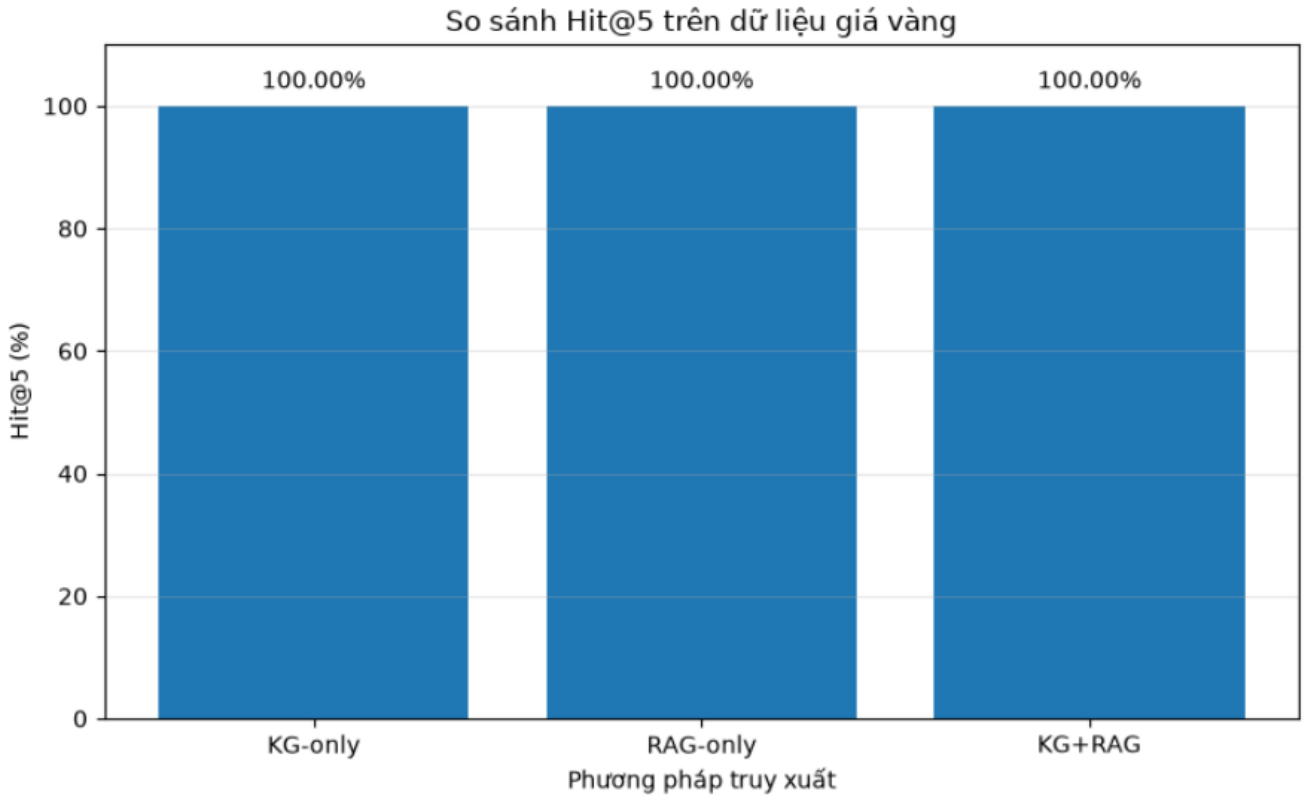
Việc tái sử dụng quy trình truy xuất giúp đánh giá xem sự khác biệt về kết quả có liên quan đến phương pháp truy xuất hay đặc điểm của từng tập tài liệu. Tuy vậy, vì số lượng tài liệu, nội dung tài liệu và bộ câu hỏi của mỗi lĩnh vực khác nhau, không thể chỉ dựa vào kết quả của một lĩnh vực để kết luận phương pháp đó luôn tốt hơn ở lĩnh vực còn lại.

5.6.4. Đánh giá khả năng truy xuất trên dữ liệu giá vàng

Đối với thí nghiệm giá vàng, kết quả cần được xem xét theo hai khía cạnh: khả năng tìm thấy bằng chứng phù hợp trong nhóm kết quả đứng đầu và thời gian truy xuất trung bình. Chỉ số Hit@5 được sử dụng để đánh giá khả năng có ít nhất một bằng chứng phù hợp trong năm kết quả đầu tiên.

Bảng dưới đây trình bày cấu trúc tổng hợp kết quả cần sử dụng từ phần đánh giá giá vàng trong notebook:

Phương pháp	Hit@5	Thời gian truy xuất trung bình
KG-only	100	14.830
RAG-only	100	7.874
KG+RAG	100	33.516



Hình 5.10. So sánh Hit@5 của các phương pháp trên dữ liệu giá vàng.

5.6.5. Nhận xét

Thí nghiệm trên dữ liệu giá vàng mở rộng quy trình truy xuất tri thức sang một loại dữ liệu tài chính khác. Việc phân biệt lĩnh vực thông qua metadata, chuẩn bị tài liệu văn bản và áp dụng TF-IDF giúp hệ thống tìm kiếm thông tin trong phạm vi tài liệu giá vàng. Phương pháp KG+RAG cung cấp cơ chế kết hợp thứ hạng từ hai nhánh truy xuất, tạo cơ sở để so sánh với từng phương pháp riêng lẻ.

Tuy nhiên, hiệu quả thực tế cần được kết luận dựa trên các chỉ số đánh giá đã thu được từ notebook. Số lượng tài liệu không tự quyết định phương pháp nào có kết quả tốt hơn; chất lượng nội dung, mức độ phù hợp giữa câu hỏi và tài liệu, cách lọc metadata và thành phần bộ câu hỏi đánh giá đều có thể ảnh hưởng đến kết quả.

Ngoài ra, dữ liệu có 141 giá trị giá bị thiếu nên hệ thống cần giữ nguyên giới hạn của nguồn dữ liệu, không tự tạo ra giá trị không được ghi nhận. Trong phạm vi đề tài, chức năng giá vàng được thiết kế để tra cứu bằng chứng lịch sử, không thay thế nguồn báo giá thị trường theo thời gian thực và không đưa ra khuyến nghị đầu tư.

5.7. Deploy

5.7.1. Mục tiêu triển khai

Sau khi hoàn thành việc xây dựng Knowledge Graph, chuẩn bị tài liệu và thử nghiệm các phương pháp truy xuất trên ba lĩnh vực Diabetes, chứng khoán và giá vàng, đề tài xây dựng một ứng dụng web nhằm tích hợp các chức năng truy xuất tri thức và tổng hợp câu trả lời vào một giao diện thống nhất.

Ứng dụng được phát triển bằng Flask ở phía máy chủ, kết hợp HTML để xây dựng cấu trúc giao diện và CSS để định dạng cách hiển thị. Backend tiếp nhận câu hỏi, xác định lĩnh vực và phương pháp truy xuất, sau đó trả về các bằng chứng phù hợp. Các bằng chứng này có thể được chuyển đến mô hình Gemini để tổng hợp thành câu trả lời bằng ngôn ngữ tự nhiên.

Mục tiêu triển khai gồm:

- Cho phép người dùng thử nghiệm hệ thống trực tiếp trên trình duyệt mà không cần chạy riêng từng đoạn code trong notebook.
- Hỗ trợ lựa chọn lĩnh vực và phương pháp truy xuất KG-only, RAG-only hoặc KG+RAG.
- Hiển thị các bằng chứng truy xuất cùng thông tin nguồn, metadata, điểm liên quan và thời gian xử lý.
- Bổ sung khả năng tổng hợp câu trả lời bằng Gemini dựa trên các bằng chứng đã truy xuất, đồng thời giữ lại bằng chứng để người dùng kiểm tra.

- Tạo cơ sở để quan sát sự khác biệt giữa thời gian truy xuất và thời gian sinh câu trả lời.

Trong phiên bản này, Gemini không thay thế các phương pháp truy xuất mà hoạt động ở bước sau truy xuất. Mô hình nhận câu hỏi cùng một số bằng chứng được hệ thống xếp hạng, sau đó tạo câu trả lời dựa trên ngữ cảnh được cung cấp. Nếu không có bằng chứng phù hợp hoặc Gemini chưa được cấu hình, ứng dụng vẫn có thể thông báo trạng thái và hiển thị kết quả truy xuất.

Cần lưu ý rằng việc sử dụng Gemini không bảo đảm mọi câu trả lời đều chính xác. Người dùng cần đối chiếu nội dung được sinh với các bằng chứng hiển thị bên dưới. Dữ liệu chứng khoán và giá vàng trong hệ thống là dữ liệu lịch sử, không phải dữ liệu thị trường theo thời gian thực.

5.7.2. Kiến trúc và các thành phần triển khai

Ứng dụng được tổ chức thành các thành phần chính như sau:

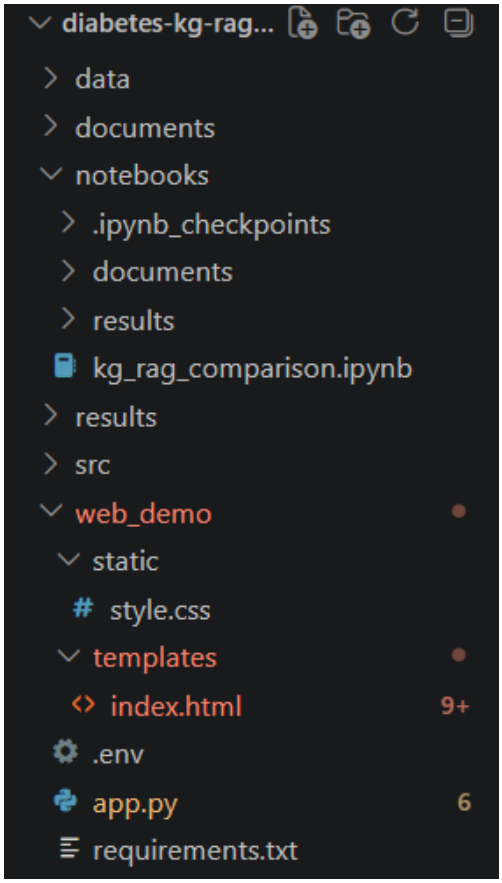
Thành phần	Vai trò
`app.py`	Backend Flask, nạp dữ liệu, xử lý bộ lọc, điều phối truy xuất, gọi Gemini và trả kết quả qua API.
`templates/index.html`	Xây dựng giao diện web, khu vực nhập câu hỏi, lựa chọn phương pháp và hiển thị câu trả lời cùng bằng chứng.
`static/style.css`	Định dạng bố cục, màu sắc, thẻ bằng chứng và khu vực câu trả lời tổng hợp.
`.env`	Lưu cấu hình kết nối Neo4j, Gemini API key và tên model được sử dụng.
`financial_knowledge_documents.csv`	Chứa các tài liệu tri thức tài chính phục vụ truy xuất chứng khoán và giá vàng.
`documents/diabetes/`	Chứa các tài liệu `.txt` và `.md` phục vụ truy xuất trên lĩnh vực Diabetes.
Neo4j	Lưu trữ đồ thị tri thức và cung cấp các nút, quan hệ hoặc dữ liệu tài chính khi backend truy vấn được cơ sở dữ liệu.
Gemini API	Tiếp nhận câu hỏi và các bằng chứng truy

	xuất để tạo câu trả lời bằng ngôn ngữ tự nhiên.

Backend sử dụng Flask để xây dựng các tuyến xử lý yêu cầu. Pandas hỗ trợ đọc dữ liệu tài chính từ CSV, trong khi scikit-learn cung cấp TF-IDF và độ tương đồng cosine cho nhánh RAG. Thư viện `neo4j` được sử dụng để kết nối với cơ sở dữ liệu đồ thị, còn `python-dotenv` hỗ trợ đọc các biến môi trường từ tệp `.env`.

Thư viện `google-genai` được sử dụng để gọi Gemini API. Tên model và API key được cấu hình thông qua biến môi trường thay vì ghi trực tiếp trong mã nguồn. Nhờ đó, có thể thay đổi model hoặc API key mà không cần sửa logic truy xuất.

Cấu trúc đường dẫn được xác định dựa trên vị trí của tệp `app.py`. Backend có thể truy cập các thư mục dữ liệu và kết quả nằm trong dự án thông qua đường dẫn tương đối được xây dựng từ vị trí này.



Hình 5.11. Cấu trúc thư mục triển khai ứng dụng KG-RAG tích hợp Gemini.

5.7.3. Backend và các API của hệ thống

Backend được triển khai bằng Flask và đảm nhiệm việc tiếp nhận yêu cầu từ giao diện web. Khi người dùng gửi câu hỏi, hệ thống kiểm tra lĩnh vực, phương pháp truy xuất, nội dung câu hỏi và các điều kiện lọc. Sau đó, backend gọi hàm truy xuất tương ứng, đo thời gian xử lý và thu thập danh sách bằng chứng.

Hệ thống hỗ trợ ba phương pháp truy xuất:

- **KG-only:** Xếp hạng các biểu diễn tri thức đã nạp vào `KG_DOCS` dựa trên mức độ trùng khớp từ khóa. Với dữ liệu Diabetes, các biểu diễn này có thể được tạo từ nút và quan hệ đọc từ Neo4j. Với dữ liệu tài chính, tài liệu CSV và dữ liệu đọc từ Neo4j có thể được đưa vào kho bằng chứng.
- **RAG-only:** Sử dụng TF-IDF và độ tương đồng cosine để xếp hạng tài liệu văn bản theo mức độ liên quan với câu hỏi.
- **KG+RAG:** Kết hợp thứ hạng từ hai nhánh bằng Reciprocal Rank Fusion (RRF), sau đó trả về danh sách bằng chứng tổng hợp.

Trong phiên bản demo hiện tại, KG-only không phải lúc nào cũng thực hiện graph traversal trực tiếp để giải quyết câu hỏi. Đối với dữ liệu tài chính, cùng nguồn CSV được nạp vào `KG_DOCS` và `RAG_DOCS`; vì vậy, cần hiểu kết quả KG-only là kết quả xếp hạng các biểu diễn tri thức dạng văn bản, không phải bằng chứng rằng mọi truy vấn đã khai thác cấu trúc quan hệ của đồ thị.

Sau bước truy xuất, backend có thể gửi câu hỏi và các bằng chứng thu được đến Gemini. Prompt yêu cầu mô hình chỉ sử dụng thông tin được cung cấp, không tự tạo số liệu hoặc ngày tháng, nêu rõ khi bằng chứng chưa đủ và dẫn chiếu `doc_id` để người dùng có thể đối chiếu nguồn. Các nội dung được truy xuất vẫn được giữ lại và hiển thị trên giao diện.

Ứng dụng cung cấp hai API chính:

API `/api/query`

API này tiếp nhận câu hỏi và trả về kết quả truy xuất, câu trả lời tổng hợp và các thông tin liên quan. Các trường đầu vào chính gồm:

- `domain`: lĩnh vực cần tra cứu, gồm `diabetes`, `stock` hoặc `gold`.
- `method`: phương pháp truy xuất, gồm `kg`, `rag` hoặc `hybrid`.
- `question`: nội dung câu hỏi.
- `top_k`: số lượng bằng chứng cần trả về.
- `year` và `ticker`: các điều kiện lọc tùy chọn khi phù hợp với lĩnh vực và dữ liệu.
- `generate_answer`: tùy chọn bật hoặc tắt bước tổng hợp câu trả lời.

Phản hồi JSON có các trường quan trọng như:

- `evidence`: danh sách bằng chứng đã xếp hạng.
- `answer`: câu trả lời được tổng hợp từ bằng chứng hoặc thông báo trạng thái nếu chưa thể tạo câu trả lời.
- `latency_ms`: thời gian truy xuất.
- `answer_latency_ms`: thời gian xử lý bước tạo câu trả lời.
- `result_count`: số lượng bằng chứng trả về.
- `filters`: các điều kiện lọc đã áp dụng.
- `note`: ghi chú về dữ liệu hoặc tình trạng truy xuất.
- `gemini_enabled` và `gemini_model`: thông tin cho biết Gemini có được khởi tạo và model nào đang được sử dụng.

Việc tách riêng `latency_ms` và `answer_latency_ms` giúp phân biệt chi phí thời gian của bước truy xuất với bước sinh câu trả lời. Thời gian tạo câu trả lời có thể thay đổi tùy vào độ dài bằng chứng, phản hồi của API và tình trạng hạn mức.

API `/api/health`

API này cung cấp thông tin kiểm tra trạng thái hệ thống, bao gồm trạng thái kết nối Neo4j, số lượng tài liệu theo từng lĩnh vực, sự tồn tại của nguồn dữ liệu và trạng thái cấu hình Gemini. API hỗ trợ kiểm tra ban đầu khi triển khai và xác định một số lỗi cấu hình thường gặp.

5.7.4. Giao diện người dùng

Giao diện web được thiết kế theo hướng tập trung vào việc tra cứu, tổng hợp và kiểm tra bằng chứng. Phần đầu trang hiển thị tên hệ thống Knowledge Graph & RAG cùng trạng thái kết nối cơ sở tri thức. Khu vực chính cho phép người dùng lựa chọn lĩnh vực, nhập câu hỏi, chọn phương pháp truy xuất và xác định số lượng bằng chứng cần hiển thị.

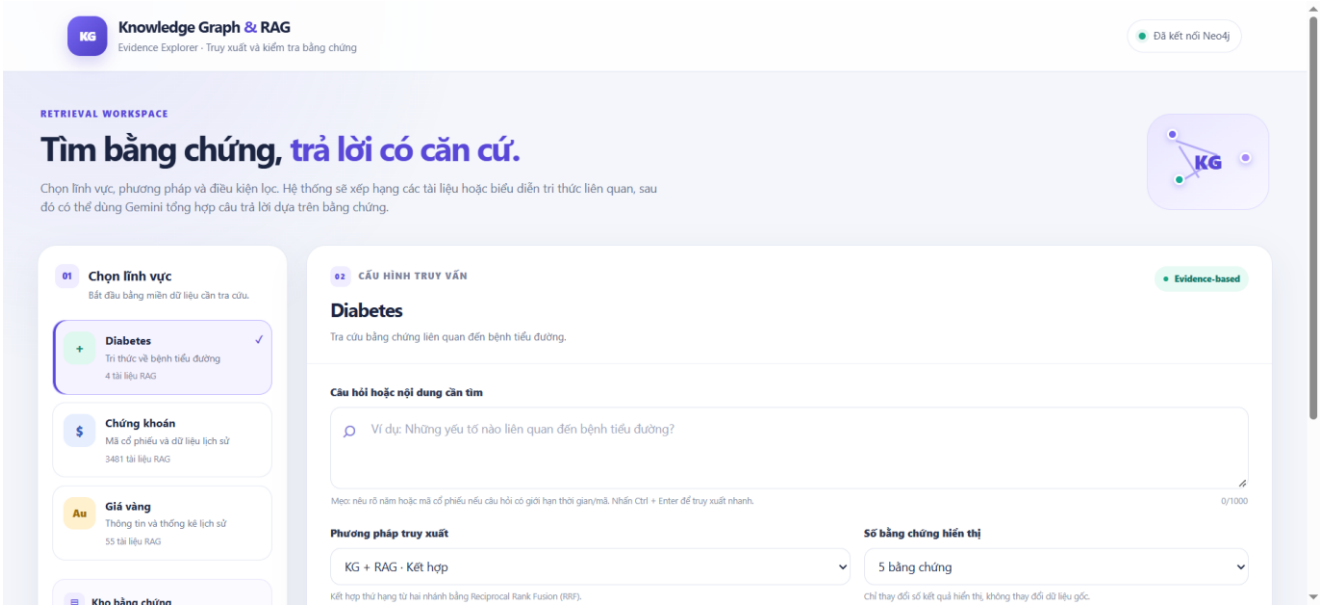
Hệ thống hỗ trợ ba lĩnh vực:

- **Diabetes:** Tra cứu thông tin liên quan đến bộ dữ liệu sức khỏe, các nút, quan hệ và kết quả thống kê đã chuẩn bị.
- **Chứng khoán:** Tra cứu bằng chứng từ dữ liệu cổ phiếu lịch sử.
- **Giá vàng:** Tra cứu bằng chứng từ bộ dữ liệu giá vàng lịch sử.

Người dùng có thể lựa chọn một trong ba phương pháp KG-only, RAG-only và KG+RAG. Giao diện cũng cung cấp các câu hỏi gợi ý để hỗ trợ thao tác thử nghiệm. Với dữ liệu tài chính, người dùng có thể nhập năm hoặc mã cổ phiếu để giới hạn phạm vi tra cứu khi phù hợp.

Sau khi gửi câu hỏi, giao diện hiển thị các thông tin về phương pháp đã chọn, thời gian truy xuất, số lượng kết quả, điều kiện lọc và ghi chú. Khu vực **Câu trả lời tổng hợp** trình bày nội dung Gemini tạo ra cùng thời gian của bước sinh câu trả lời. Bên dưới là danh sách bằng chứng gồm mã tài liệu, metadata, điểm liên quan và nội dung văn bản.

Việc hiển thị câu trả lời cùng bằng chứng giúp người dùng kiểm tra nội dung mà mô hình tổng hợp thay vì chỉ nhận một câu trả lời không có nguồn đối chiếu. Nếu không tìm được bằng chứng phù hợp, hệ thống thông báo tình trạng này thay vì tự ý thay bằng năm hoặc mã khác. Nếu Gemini chưa được cấu hình hoặc gặp lỗi API, khu vực câu trả lời sẽ thông báo trạng thái; các chức năng truy xuất bằng chứng vẫn có thể sử dụng độc lập.



Hình 5.12. Giao diện chính của hệ thống KG-RAG tích hợp Gemini.

5.7.5. Quy trình chạy và sử dụng ứng dụng

Để khởi chạy ứng dụng, cần chuẩn bị môi trường Python, cài đặt các thư viện và cấu hình các biến môi trường cần thiết. Nếu sử dụng chức năng tổng hợp câu trả lời, cần có Gemini API key hợp lệ và tên model được hỗ trợ. Nếu muốn truy vấn dữ liệu từ Neo4j, cần bảo đảm thông tin kết nối chính xác.

Bước 1: Cài đặt thư viện

Mở terminal tại thư mục dự án và chuyển vào thư mục chứa ứng dụng:

```
cd web_demo
```

Cài đặt các thư viện được khai báo trong `requirements.txt`:

```
python -m pip install -r requirements.txt
```

Trong danh sách thư viện cần có Flask, pandas, python-dotenv, scikit-learn, neo4j và google-genai. Thư viện `google-genai` phục vụ việc gọi Gemini API.

Bước 2: Cấu hình Neo4j và Gemini

Mở tệp `.env` trong thư mục `web_demo`. Giữ nguyên các biến Neo4j đang sử dụng và kiểm tra chúng có giá trị phù hợp:

```
NEO4J_URI=địa_chỉ_kết_nối
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=mật_khẩu_của_bạn
```

Bổ sung cấu hình Gemini:

```
GEMINI_API_KEY=API_KEY_CUA_BAN
GEMINI_MODEL=gemini-3.8-flash
```

Các giá trị trên chỉ minh họa cấu trúc cấu hình. Cần thay API key và thông tin kết nối bằng giá trị thực tế. Không đưa API key hoặc mật khẩu thật vào mã nguồn, báo cáo, ảnh chụp màn hình hay kho mã nguồn công khai. Không ghi đè tệp `.env` đang có chỉ để bổ sung cấu hình Gemini.

Bước 3: Khởi chạy ứng dụng

Từ thư mục `web_demo`, chạy:

```
python app.py
```

Khi khởi động, ứng dụng kiểm tra cấu hình Neo4j, nạp tài liệu và xây dựng chỉ mục RAG. Nếu cấu hình Gemini hợp lệ, backend khởi tạo Gemini client. Nếu Gemini chưa được cấu hình, ứng dụng ghi nhận trạng thái này và vẫn có thể phục vụ chức năng truy xuất bằng chứng.

Bước 4: Mở ứng dụng trên trình duyệt

Truy cập:

```
http://127.0.0.1:5000
```

Người dùng lựa chọn lĩnh vực, nhập câu hỏi và chọn phương pháp truy xuất. Sau khi gửi yêu cầu, ứng dụng hiển thị bằng chứng và câu trả lời tổng hợp nếu bước gọi Gemini thực hiện được.

Bước 5: Kiểm tra kết quả

Nên kiểm tra đồng thời nội dung câu trả lời, các `doc_id` được dẫn chiếu, bằng chứng thực tế và hai thông số thời gian. Nếu câu trả lời chưa được hỗ trợ đầy đủ bởi bằng chứng, cần ghi nhận giới hạn đó thay vì xem nội dung do mô hình sinh ra là kết luận chắc chắn.

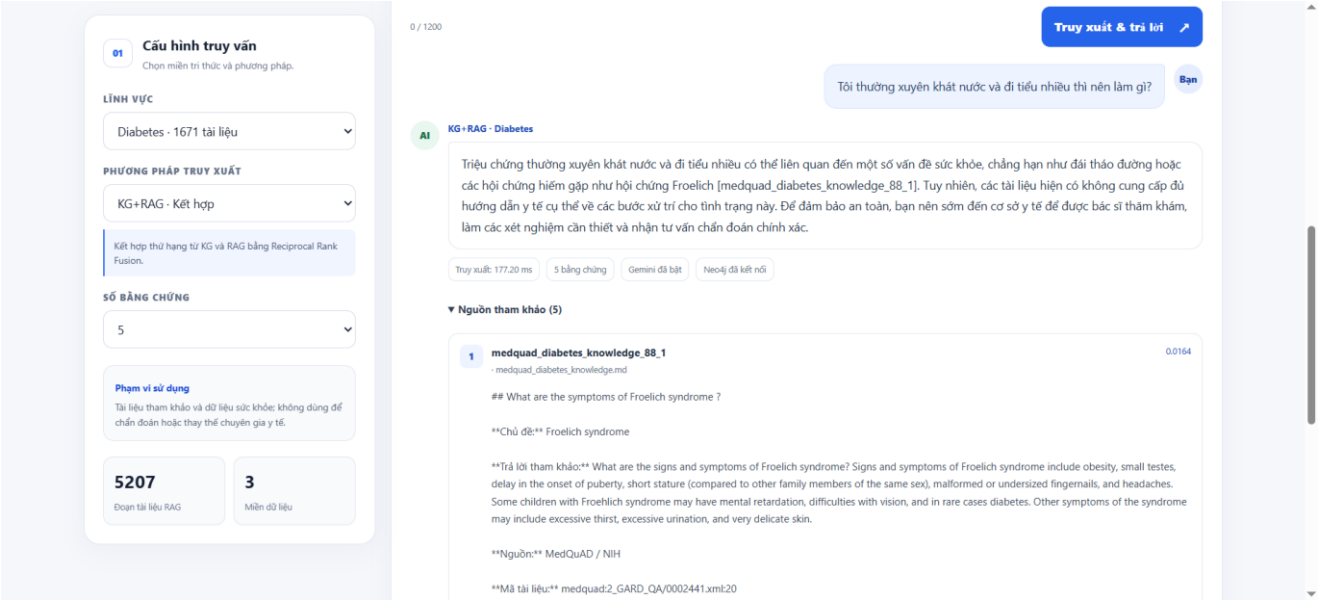
Quy trình trên triển khai ứng dụng trong môi trường cục bộ để thử nghiệm và trình diễn. Đây chưa phải triển khai công khai trên máy chủ Internet; việc đưa ứng dụng lên môi trường công khai cần có các biện pháp bảo mật, quản lý khóa API, kiểm soát truy cập và cấu hình máy chủ phù hợp.

5.7.6. Minh họa quá trình truy xuất và tổng hợp câu trả lời

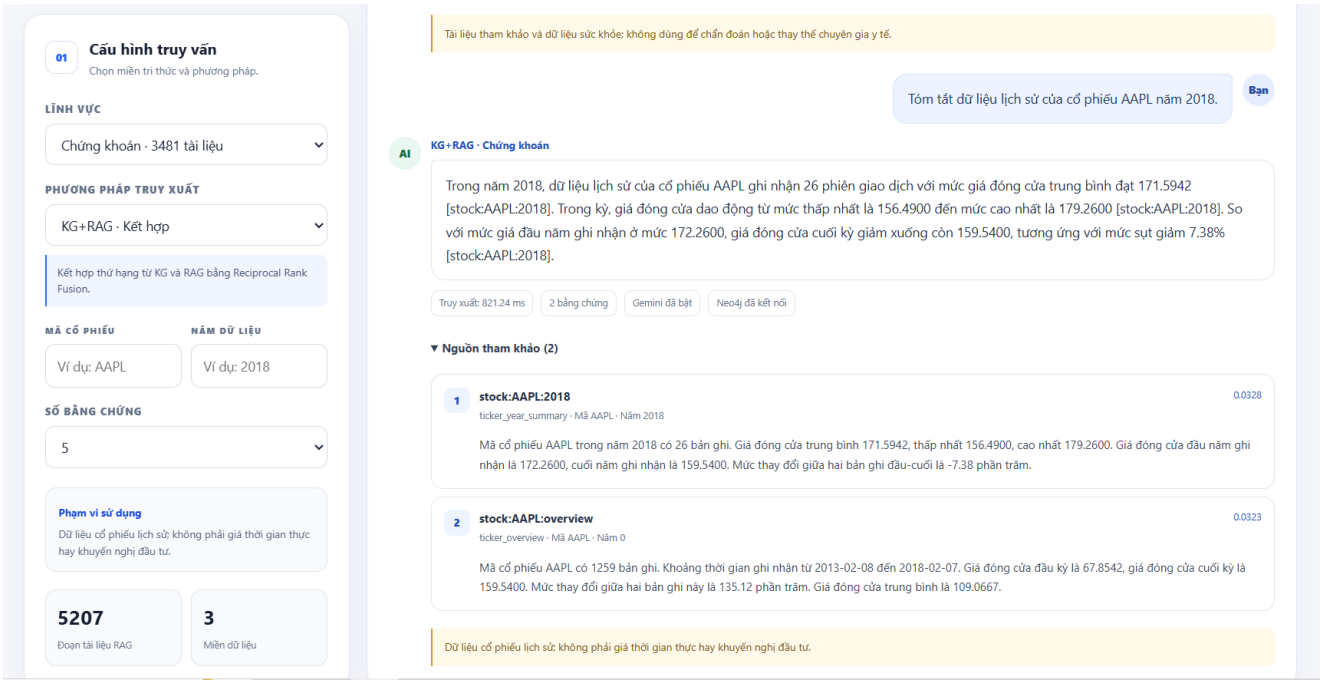
Sau khi khởi chạy ứng dụng, người dùng có thể thực hiện truy vấn theo quy trình sau:

- Chọn lĩnh vực cần tra cứu: Diabetes, Chứng khoán hoặc Giá vàng.
- Nhập câu hỏi vào ô nhập liệu hoặc lựa chọn câu hỏi gợi ý trên giao diện.
- Chọn phương pháp truy xuất: KG-only, RAG-only hoặc KG+RAG.
- Thiết lập điều kiện lọc phù hợp với lĩnh vực và dữ liệu, nếu có.
- Gửi yêu cầu để ứng dụng thực hiện truy xuất thông qua API `/api/query`.
- Kiểm tra câu trả lời do Gemini tổng hợp nếu chức năng này được cấu hình và hoạt động thành công.
- Đối chiếu câu trả lời với các bằng chứng truy xuất, bao gồm mã tài liệu, metadata và nội dung nguồn được hiển thị.
- Đánh giá mức độ phù hợp của câu trả lời dựa trên bằng chứng, đồng thời ghi nhận những nội dung chưa đủ dữ liệu để kết luận.

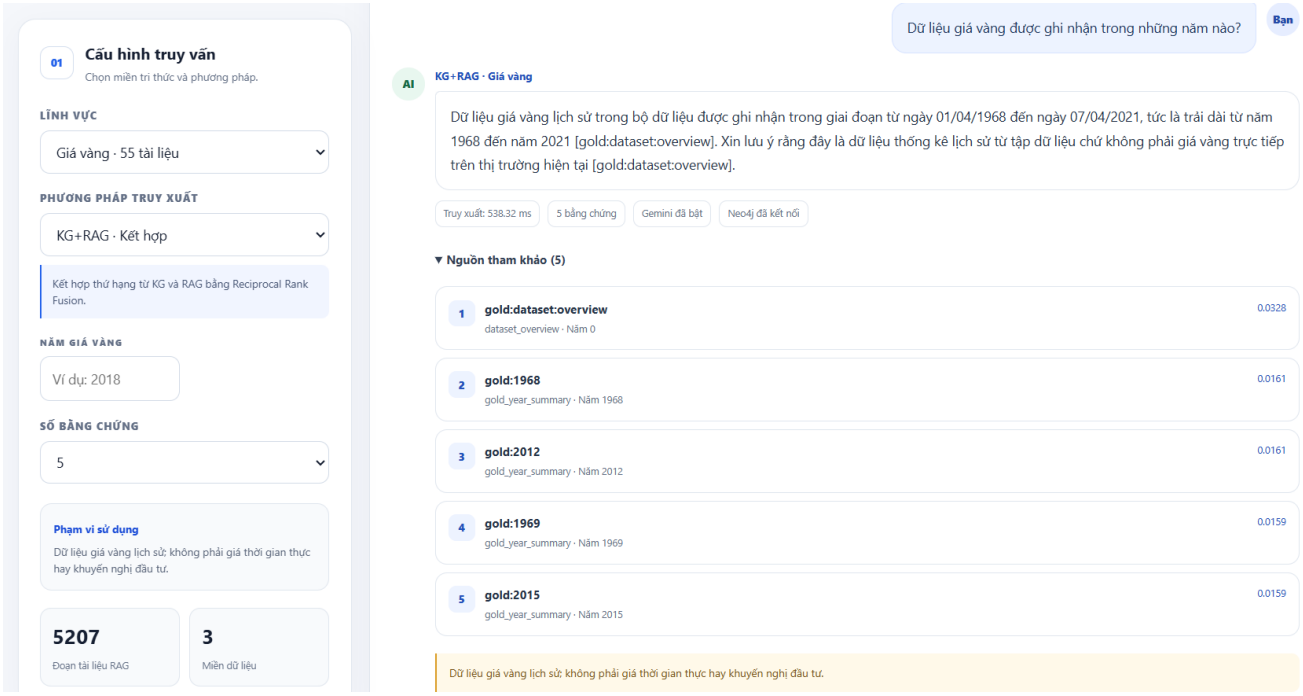
Để so sánh các phương pháp truy xuất trên ứng dụng, cần sử dụng cùng câu hỏi, cùng lĩnh vực và cùng điều kiện lọc. Các hình minh họa dưới đây thể hiện quá trình tra cứu trên ba lĩnh vực được hỗ trợ, từ việc nhập câu hỏi đến hiển thị câu trả lời và bằng chứng liên quan.



Hình 5.13. Kết quả truy xuất và tổng hợp câu trả lời trên dữ liệu Diabetes.



Hình 5.14. Kết quả truy xuất trên dữ liệu chứng khoán.



Hình 5.15. Kết quả truy xuất trên dữ liệu giá vàng.

5.7.7. Nhận xét về kết quả triển khai

Việc xây dựng ứng dụng web giúp tích hợp các thành phần thực nghiệm vào một giao diện thống nhất. Người dùng có thể lựa chọn lĩnh vực, nhập câu hỏi, thay đổi phương pháp truy xuất và kiểm tra bằng chứng mà không cần thao tác trực tiếp trên notebook.

Backend Flask tiếp nhận yêu cầu, áp dụng các điều kiện lọc và điều phối các nhánh truy xuất. Trong phiên bản ứng dụng hiện tại, TF-IDF và độ tương đồng cosine được sử dụng trong nhánh RAG. Nhánh KG-only xếp hạng các biểu diễn tri thức đã được nạp vào `KG_DOCS`, trong khi KG+RAG kết hợp thứ hạng từ hai nhánh truy xuất để tạo danh sách bằng chứng tổng hợp.

Việc tích hợp Gemini bổ sung bước chuyển từ bằng chứng truy xuất sang câu trả lời bằng ngôn ngữ tự nhiên. Câu trả lời được hiển thị cùng danh sách bằng chứng và thông tin thời gian xử lý, giúp người dùng có cơ sở đối chiếu nội dung được tổng hợp với nguồn dữ liệu. Việc phân biệt thời gian truy xuất và thời gian tạo câu trả lời cũng hỗ trợ quan sát chi phí thời gian của từng giai đoạn.

Bên cạnh ứng dụng web, nghiên cứu còn thực hiện thí nghiệm so sánh TF-IDF với Embedding trong notebook. Trên bộ gồm 20 câu hỏi đánh giá thuộc ba lĩnh vực Diabetes, chứng khoán và giá vàng, Embedding đạt Hit@5 bằng 100% và MRR bằng 77,67%, trong khi TF-IDF đạt Hit@5 bằng 95% và MRR bằng 75,00%. Tuy nhiên, thời gian truy xuất trung bình của Embedding cao hơn TF-IDF. Kết quả theo từng lĩnh vực cũng cho thấy ưu thế của mỗi phương pháp không đồng nhất trên tất cả các chỉ số.

Thí nghiệm Embedding hiện chỉ có vai trò so sánh phương pháp truy xuất trong notebook, chưa được tích hợp vào ứng dụng web. Vì vậy, các kết quả của thí nghiệm này không thay thế kết quả đánh giá KG-only, RAG-only và KG+RAG đã trình bày ở các mục trước. Việc tách riêng hai phần giúp duy trì tính nhất quán giữa kết quả thực nghiệm và mô tả triển khai.

Hệ thống vẫn có một số giới hạn. Chất lượng truy xuất phụ thuộc vào nội dung tài liệu, cách chuẩn bị tri thức, mức độ phù hợp giữa câu hỏi và dữ liệu cũng như cơ chế xếp hạng. KG-only trong bản demo chưa thực hiện truy vấn đồ thị trực tiếp cho mọi lĩnh vực; đối với dữ liệu tài chính, nguồn CSV được dùng để tạo các biểu diễn bằng chứng cho cả `KG_DOCS` và `RAG_DOCS`. Do đó, không nên diễn giải mọi kết quả của nhánh KG-only là kết quả của việc duyệt trực tiếp các quan hệ trong đồ thị.

Ngoài ra, câu trả lời do Gemini tạo ra vẫn có thể thiếu chính xác hoặc suy diễn vượt quá nguồn nếu bằng chứng không đầy đủ. Dữ liệu tài chính trong ứng dụng là dữ liệu lịch sử, không phải báo giá theo thời gian thực. Phạm vi câu trả lời cũng bị giới hạn bởi các tài liệu đã nạp, số lượng bằng chứng được chọn và tình trạng hoạt động của Gemini API. Vì vậy, ứng dụng phù hợp với mục đích thử nghiệm truy xuất và tổng hợp tri thức trong phạm vi đề tài, không thay thế hệ thống dự báo tài chính hoặc công cụ chẩn đoán y tế.

Nhìn chung, đề tài đã xây dựng được ứng dụng tương tác trên ba lĩnh vực Diabetes, chứng khoán và giá vàng, đồng thời thực hiện thí nghiệm bổ sung để so sánh TF-IDF với Embedding. Kết quả tạo cơ sở để tiếp tục nghiên cứu các phương pháp truy xuất phù hợp hơn,

cải thiện việc khai thác quan hệ trong đồ thị và đánh giá độ chính xác của câu trả lời dựa trên bằng chứng.

TÀI LIỆU THAM KHẢO

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS 2020). https://arxiv.org/abs/2005.11401

[2] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal rank fusion outperforms condorcet and individual rank learning methods*. Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 758–759. https://doi.org/10.1145/1571941.1572114

[3] Neo4j, Inc. (n.d.). *Cypher Manual*. https://neo4j.com/docs/cypher-manual/current/introduction/

[4] scikit-learn developers. (n.d.). *TfidfVectorizer*. scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[5] Google. (n.d.). *Generating content — Gemini API*. Google AI for Developers. https://ai.google.dev/api/generate-content

PHỤ LỤC

1. Bộ dữ liệu giá nhà

- Tên bộ dữ liệu: Miadul House Price.
- Nguồn tham khảo: Kaggle.
- Đường dẫn: https://www.kaggle.com/datasets/miadul/house-price-prediction-dataset/data
- Mục đích sử dụng: Dữ liệu mô tả các đặc điểm của bất động sản, được sử dụng để xây dựng mô hình hồi quy dự đoán giá nhà.

2. Bộ dữ liệu đánh giá sản phẩm thời trang

- Tên bộ dữ liệu: Women's Clothing Reviews.
- Nguồn tham khảo: Kaggle.
- Đường dẫn: https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews
- Mục đích sử dụng: Dữ liệu đánh giá sản phẩm thời trang của khách hàng, được sử dụng để phân tích nội dung nhận xét và xây dựng mô hình dự đoán khả năng khách hàng khuyến nghị sản phẩm.

3. Bộ dữ liệu chữ số viết tay MNIST

- Tên bộ dữ liệu: MNIST.
- Nguồn tham khảo được ghi trong báo cáo Assignment 05: Kaggle.
- Đường dẫn: https://www.kaggle.com/datasets/oddrationale/mnist-in-csv
- Mục đích sử dụng: Bộ dữ liệu ảnh chữ số viết tay từ 0 đến 9, với ảnh grayscale kích thước 28 × 28 pixel, được sử dụng để huấn luyện và đánh giá các mô hình CNN phân loại ảnh.

4. Bộ dữ liệu ảnh thời trang Fashion-MNIST

- Tên bộ dữ liệu: Fashion-MNIST.
- Nguồn tham khảo: Kaggle.
- Đường dẫn: https://www.kaggle.com/datasets/zalando-research/fashionmnist
- Mục đích sử dụng: Bộ dữ liệu ảnh sản phẩm thời trang gồm 10 lớp, được sử dụng để đánh giá khả năng phân loại ảnh và so sánh hiệu quả giữa các kiến trúc CNN.

5. Bộ dữ liệu Pima Indians Diabetes

- Tên bộ dữ liệu: Pima Indians Diabetes Dataset.
- Nguồn tham khảo: Kaggle.

- Đường dẫn: https://www.kaggle.com/datasets/jamaltariqcheema/pima-indians-diabetes-dataset/data
- Mục đích sử dụng: Dữ liệu dạng bảng gồm các chỉ số sức khỏe liên quan đến bệnh tiểu đường, được sử dụng để thử nghiệm các mô hình CNN một chiều (Conv1d) trên dữ liệu phi ảnh.

6. Bộ dữ liệu giao dịch bán lẻ trực tuyến

- Tên bộ dữ liệu: Online Retail Business.
- Tệp dữ liệu: `OnlineRetail.csv`.
- Nguồn tham khảo: Kaggle.
- Đường dẫn: https://www.kaggle.com/datasets/umerkk12/online-retail-business/data
- Nguồn tham khảo bổ sung: UCI Machine Learning Repository.
- Đường dẫn bổ sung: https://archive.ics.uci.edu/dataset/352/online+retail
- Mục đích sử dụng: Dữ liệu giao dịch bán lẻ trực tuyến được làm sạch và tổng hợp theo ngày thành các đặc trưng như doanh thu, số giao dịch, số khách hàng, số lượng hàng hóa và đơn giá trung bình. Chuỗi dữ liệu sau đó được sử dụng để huấn luyện RNN dự báo doanh thu.

7. Bộ dữ liệu Diabetes Health Indicators

- Tên bộ dữ liệu: Diabetes Health Indicators Dataset.
- Nguồn tham khảo: Kaggle.
- Đường dẫn: https://www.kaggle.com/datasets/mohankrishnathalla/diabetes-health-indicators-dataset
- Mục đích sử dụng: Dữ liệu sức khỏe và các chỉ số liên quan đến bệnh tiểu đường, được sử dụng để xây dựng cơ sở tri thức và thử nghiệm truy xuất thông tin.

8. Bộ dữ liệu chứng khoán S&P 500

- Tên bộ dữ liệu: S&P 500 Stock Data.
- Tệp dữ liệu: `all_stocks_5yr.csv`.
- Nguồn tham khảo: Kaggle.
- Đường dẫn: https://www.kaggle.com/camnugent/sandp500
- Mục đích sử dụng: Dữ liệu giá cổ phiếu lịch sử, bao gồm ngày giao dịch, giá mở cửa, giá cao nhất, giá thấp nhất, giá đóng cửa, khối lượng giao dịch và mã cổ phiếu.

9. Bộ dữ liệu giá vàng lịch sử

- Tên bộ dữ liệu: Gold and Silver Prices Dataset.
- Tệp dữ liệu: `gold_price.csv`.

- Nguồn tham khảo: Kaggle.
- Đường dẫn: https://www.kaggle.com/lbronchal/gold-and-silver-prices-dataset
- Mục đích sử dụng: Dữ liệu giá vàng lịch sử, được sử dụng để xây dựng cơ sở tri thức tài chính và thử nghiệm truy xuất thông tin về giá vàng theo thời gian.

10. Source code demo

Chương 2: https://github.com/tnd17/Assignment-02-PTHTTM-N05

CNN: https://github.com/tnd17/Assignment-05-PTHTTM-N05

KG-RAG: https://github.com/tnd17/KG-RAG-WEB-DEMO